

Politechnika Warszawska

WYDZIAŁ MECHANICZNY
ENERGETYKI I LOTNICTWA



Zakład Teorii Maszyn i Robotów

Praca dyplomowa inżynierska

na kierunku Robotyka i Automatyka
w specjalności Robotyka

Symulacja robota mobilnego do rozpoznawania tablic rejestracyjnych
samochodów

{Dawid Balewski}

Numer albumu 327687

promotor

Dr inż. Andrzej Kordecki

WARSZAWA 2026

Streszczenie

Symulacja robota mobilnego do rozpoznawania tablic rejestracyjnych samochodów

Praca opisuje proces opracowania symulacji autonomicznego robota mobilnego zdolnego do patrolowania parkingu, wykrywania pojazdów zaparkowanych z naruszeniem granic miejsca postojowego oraz odczytu ich tablic rejestracyjnych. W ramach pracy przedstawiono przegląd stanu wiedzy z zakresu robotyki mobilnej, nawigacji autonomicznej oraz wizji komputerowej, a także przeprowadzono badania skuteczności zaimplementowanego systemu.

Do realizacji projektu wykorzystano platformę robota TurtleBot4, którą zmodyfikowano pod kątem wymagań inspekcji parkingowej. Detekcja nieprawidłowego parkowania działa na podstawie analizy ciągłości linii parkingowej za pomocą kamery RGB oraz z wykorzystaniem segmentacji barwnej w przestrzeni HSV. Odczyt tablic rejestracyjnych zrealizowano za pomocą modułu ANPR łączącego architekturę YOLO do lokalizacji regionów tablic z biblioteką EasyOCR do rozpoznawania tekstu. Całość koordynowana jest przez nadrzędny węzeł sterujący, integrujący nawigację autonomiczną (Nav2) z modułami detekcyjnymi w ramach jednej misji inspekcyjnej.

Całość przetestowano na podstawie testów poszczególnych modułów oraz testów integracyjnych w środowisku symulacyjnym obejmującym Gazebo Harmonic jako symulator, ROS 2 Jazzy jako warstwę komunikacyjną i sterującą oraz RViz jako narzędzie wizualizacyjne, całość uruchomiona na systemie Ubuntu 24.04.

W ramach wniosków końcowych podsumowano przeprowadzone badania, omówiono ograniczenia zastosowanego podejścia oraz zaproponowano kierunki dalszego rozwoju platformy.

Słowa kluczowe: robot mobilny, autonomiczna nawigacja, SLAM, Nav2, ANPR, YOLO, EasyOCR, wizja komputerowa, symulacja, Gazebo Harmonic, ROS 2, TurtleBot4

Abstract

Mobile robot simulation for recognizing vehicle license plates

This thesis describes the development of a simulation of an autonomous mobile robot capable of patrolling a parking lot, detecting vehicles parked in violation of parking space boundaries, and reading their license plates. The work includes a literature review covering mobile robotics, autonomous navigation, and computer vision, as well as experimental evaluation of the implemented system.

The project was built around the TurtleBot4 platform, modified to meet the requirements of parking inspection. Incorrect parking detection was based on the analysis of parking line continuity in an RGB camera image using colour segmentation in the HSV colour space. License plate reading was performed by an ANPR module combining the YOLO architecture for plate region localisation with the EasyOCR library for text recognition. All components are coordinated by a master control node that integrates autonomous navigation (Nav2) with detection modules within a single inspection mission.

The system was validated through tests of individual modules and integrated end-to-end tests in a simulation environment comprising Gazebo Harmonic as the simulator, ROS 2 Jazzy as the communication and control layer, and RViz as the visualisation tool, all running on Ubuntu 24.04.

The conclusions summarise the conducted experiments, discuss the limitations of the adopted approach, and propose directions for further development of the platform.

Keywords: mobile robot, autonomous navigation, SLAM, Nav2, ANPR, YOLO, EasyOCR, computer vision, simulation, Gazebo Harmonic, ROS 2, TurtleBot4



„załącznik nr 3 do zarządzenia nr 24/2016 Rektora PW

Warszawa, 20.02.2026.

miejsowość i data

Dawid Balewski

.....
imię i nazwisko studenta

327687

.....
numer albumu

Robotyka i automatyka

.....
kierunek studiów

OŚWIADCZENIE

Świadomy/-a odpowiedzialności karnej za składanie fałszywych zeznań oświadczam, że niniejsza praca dyplomowa została napisana przeze mnie samodzielnie, pod opieką kierującego pracą dyplomową.

Jednocześnie oświadczam, że:

- niniejsza praca dyplomowa nie narusza praw autorskich w rozumieniu ustawy z dnia 4 lutego 1994 roku o prawie autorskim i prawach pokrewnych (Dz.U. z 2006 r. Nr 90, poz. 631 z późn. zm.) oraz dóbr osobistych chronionych prawem cywilnym,
- niniejsza praca dyplomowa nie zawiera danych i informacji, które uzyskałem/-am w sposób niedozwolony,
- niniejsza praca dyplomowa nie była wcześniej podstawą żadnej innej urzędowej procedury związanej z nadawaniem dyplomów lub tytułów zawodowych,
- wszystkie informacje umieszczone w niniejszej pracy, uzyskane ze źródeł pisanych i elektronicznych, zostały udokumentowane w wykazie literatury odpowiednimi odnośnikami,
- znam regulacje prawne Politechniki Warszawskiej w sprawie zarządzania prawami autorskimi i prawami pokrewnymi, prawami własności przemysłowej oraz zasadami komercjalizacji.

Oświadczam, że treść pracy dyplomowej w wersji drukowanej, treść pracy dyplomowej zawartej na nośniku elektronicznym (płycie kompaktowej) oraz treść pracy dyplomowej w module APD systemu USOS są identyczne.

.....
czytelny podpis studenta”

Spis treści

Streszczenie	3
Abstract	4
1 Wstęp	8
1.1 Analiza tematu	8
1.2 Cel pracy	8
1.3 Cele szczegółowe	8
1.3.1 Etapy realizacji projektu	9
1.4 Wartość edukacyjna projektu	9
2 Przegląd stanu wiedzy	11
2.1 ROS	11
2.1.1 Architektura ROS 2	11
2.1.2 Ewolucja ROS - różnice między ROS 1 a ROS 2	12
2.2 Gazebo	13
2.2.1 Porównanie	14
2.3 Roboty mobilne	16
2.3.1 TurtleBot4	16
2.3.2 Alternatywne platformy – ROSbot	17
2.3.3 ALPR Robot	18
2.4 RViz	18
2.5 SLAM	19
2.5.1 Źródła informacji wykorzystywane w SLAM	20
2.6 Mapa zajętości	21
2.7 Nav2	22
2.7.1 Realizacja trajektorii przez Nav2	22
2.7.2 Główne komponenty Nav2	23
2.7.3 Drzewo zachowań w Nav2	23
2.8 Wizja komputerowa	24
2.8.1 Klasyczna detekcja obiektów	24
2.8.2 Nowoczesne metody detekcji	25
2.9 OCR	27
2.9.1 Zasada działania systemów OCR	27
2.9.2 Konkretnie metody OCR	28
3 Realizacja projektu i weryfikacja działania	29
3.1 Przygotowanie środowiska	29
3.1.1 Założenia środowiskowe i dobór narzędzi	29
3.1.2 Warstwa sprzętowa	29
3.2 Budowa świata symulacji w Gazebo	30
3.2.1 Prototyp parkingu	30
3.2.2 Stworzenie wersji parkingu pod testy	30
3.3 Integracja robota TurtleBot4 ze światem własnym	31
3.3.1 Modyfikacje Robota	31
3.4 Modelowanie tablic rejestracyjnych w Gazebo	32
3.4.1 Założenia dotyczące tablic rejestracyjnych	32
3.4.2 Model tablicy w Gazebo	33
3.4.3 Dodanie tablic do pojazdów	33

3.5	Odczyt tablic rejestracyjnych przez robota	34
3.5.1	Wybrane rozwiązanie ANPR	34
3.5.2	Wstępne badania skuteczności wybranego rozwiązania	35
3.5.3	Węzeł ROS 2 do odczytu tablic rejestracyjnych	37
3.5.4	Metodyka doboru parametrów algorytmu	39
3.6	Algorytm detekcji nieprawidłowego parkowania	43
3.6.1	Założenia algorytmu i wybór podejścia	43
3.6.2	Skrypt testowy do analizy linii	43
3.6.3	Metodyka doboru parametrów algorytmu	44
3.6.4	Analiza i ograniczenia metody	45
3.7	Automatyzacja inspekcji parkingu z użyciem Nav2	45
3.7.1	Zmapowanie przestrzeni roboczej	46
3.7.2	Definicja punktów inspekcyjnych (waypoints)	46
3.7.3	Usługa sterująca realizacją punktów (Nav2)	47
3.7.4	Problemy z dokładnością pozycji i orientacji	47
3.7.5	Proces algorytmu dla pojedynczego punktu	48
3.7.6	Komunikacja między węzłami	49
3.7.7	Testy zintegrowanego systemu inspekcji	49
3.8	Decyzja o sposobie raportowania naruszeń	56
3.8.1	Brak jednoznacznego przypisania naruszenia do pojazdu w przypadku dwóch aut	56
3.8.2	Minimalnie inwazyjne podejście: zapis dowodu dla operatora	56
4	Podsumowanie	57
4.1	Wnioski końcowe	57
4.2	Propozycja dalszych prac	57
4.3	Podsumowanie	58
	Wykaz skrótów i symboli	63
	Spis rysunków	65
	Spis tabel	66
	Załączniki	67
	Pliki projektowe	67

Rozdział 1

Wstęp

1.1 Analiza tematu

Stale rosnąca liczba pojazdów w miastach prowadzi do nasilenia problemów związanych z organizacją ruchu i zarządzaniem przestrzenią parkingową. Nieprawidłowe parkowanie, obejmujące m.in. zajmowanie dwóch miejsc postojowych, blokowanie przestrzeni dla pieszych czy parkowanie poza wyznaczonymi strefami, stanowi nie tylko uciążliwość dla pozostałych użytkowników, lecz również generuje zagrożenie dla bezpieczeństwa ruchu drogowego. Tradycyjne metody egzekwowania przepisów parkingowych, polegające na bezpośrednim nadzorze przez służby porządkowe, wiążą się z wysokimi kosztami oraz ograniczoną skalowalnością - patrol ludzki nie jest w stanie objąć monitoringiem dużego obszaru w sposób ciągły.

Postęp w dziedzinie robotyki mobilnej oraz wizji komputerowej wspieranej sztuczną inteligencją otwiera możliwość automatyzacji tego typu zadań. Współczesne platformy robotyczne, wyposażone w czujniki LiDAR, kamery i jednostki obliczeniowe, są zdolne do autonomicznego przemieszczania się po wyznaczonym terenie, budowania mapy otoczenia i realizowania zadania w czasie rzeczywistym. Z kolei metody uczenia głębokiego, w szczególności architektury detekcji obiektów, umożliwiają niezawodne rozpoznawanie pojazdów oraz ich tablic rejestracyjnych bez konieczności stosowania infrastruktury stacjonarnej.

Praca łączy dwa rozwijające się obszary: robotykę mobilno-autonomiczną oraz nowoczesne metody wizji komputerowej w automatyzacji procesów monitoringu. Robotyka mobilna znajduje coraz szersze zastosowanie w logistyce, inspekcji infrastruktury, rolnictwie precyzyjnym i usługach terenowych. Natomiast algorytmy działające na podstawie sztucznej inteligencji wspierają automatyzację analizy danych sensorycznych, klasyfikację obiektów oraz podejmowanie decyzji na podstawie informacji wizyjnych. Połączenie obu tych obszarów w kontekście monitoringu parkingowego stanowi praktyczny przykład systemu, w którym autonomiczna nawigacja i percepcja¹ komputerowa współpracują w jednym systemie.

1.2 Cel pracy

Celem pracy jest opracowanie symulacji autonomicznego robota mobilnego w środowisku Gazebo, zdolnego do patrolowania wyznaczonego obszaru parkingowego, wykrywania pojazdów nieprawidłowo zaparkowanych oraz odczytu ich tablic rejestracyjnych. W ramach pracy przeprowadzono implementację i integrację modułów nawigacji autonomicznej, percepcji wizyjnej oraz logiki inspekcyjnej, a następnie zweryfikowano działanie kompletnego systemu w środowisku symulacyjnym.

Oprócz celu technicznego realizacji projektu, ważnym aspektem jest praktyczne opanowanie istotnych narzędzi i technologii stosowanych we współczesnej robotyce mobilnej. Kompetencje obejmujące konfigurację środowisk symulacyjnych, implementację algorytmów nawigacji i percepcji oraz integrację systemów wielomodułowych są fundamentem pracy inżyniera w branży robotycznej.

1.3 Cele szczegółowe

Cele szczegółowe pracy obejmują:

¹Percepcja - proces aktywnej interpretacji danych sensorycznych (wizyjnych, laserowych) w celu zrozumienia otoczenia i podejmowania decyzji.

- Konfigurację środowiska symulacyjnego umożliwiającego realizację projektu.
- Budowę świata symulacyjnego odwzorowującego parking z wyznaczonymi miejscami postojowymi oraz modelami pojazdów.
- Dobór i adaptację platformy robota mobilnego pod kątem wymagań projektu.
- Implementację autonomicznej nawigacji robota po wyznaczonej trasie inspekcyjnej z wykorzystaniem algorytmów mapowania i lokalizacji.
- Opracowanie modułu automatycznego rozpoznawania tablic rejestracyjnych (Automatic Number Plate Recognition, ANPR).
- Opracowanie algorytmu detekcji nieprawidłowego parkowania.
- Integrację wszystkich modułów w jeden działający system end-to-end², realizujący pełen cykl inspekcji parkingu w trybie autonomicznym.

Zakres pracy obejmuje fazę projektowania, implementacji oraz testów poszczególnych modułów, jak i weryfikację zintegrowanego systemu w warunkach symulacji.

1.3.1 Etapy realizacji projektu

Realizacja projektu została podzielona na kolejne etapy, umożliwiające stopniowe opracowanie, integrację oraz weryfikację poszczególnych komponentów systemu.

1. W pierwszym etapie przeprowadzono analizę tematu oraz określono założenia projektowe. Zdefiniowano cel systemu, jego zakres funkcjonalny, a także dobrano narzędzia opisane w przeglądzie stanu wiedzy (rozdział 2).
2. Kolejnym etapem była konfiguracja spójnego środowiska programistycznego. Obejmowała ona instalację i uzgodnienie wersji systemu operacyjnego Ubuntu 24.04, frameworku³ ROS 2 Jazzy, symulatora Gazebo Harmonic oraz narzędzia wizualizacyjnego RViz.
3. Następnie zbudowano świat symulacyjny w Gazebo, odwzorowujący zamknięty parking z wyznaczonymi miejscami postojowymi, modelami pojazdów pobranych z repozytorium Gazebo Fuel oraz realistycznymi tablicami rejestracyjnymi generowanymi za pomocą autorskiego skryptu.
4. W kolejnym etapie zintegrowano platformę robota TurtleBot4 ze stworzonym światem symulacyjnym. Przeprowadzono modyfikacje obejmujące zmianę pozycji i orientacji sensorów (LiDAR, kamera), dostrojenie rozkładu masy oraz parametryzację czujników w plikach URDF/XACRO⁴.
5. Równolegle opracowano moduł automatycznego rozpoznawania tablic rejestracyjnych, działający na podstawie architektury YOLO do lokalizacji regionów tablic w obrazie oraz biblioteki EasyOCR do odczytu tekstu. Moduł zaimplementowano jako węzeł ROS 2 z walidacją formatu tablicy rejestracyjnej.
6. Następnie opracowano algorytm detekcji nieprawidłowego parkowania, polegający na analizie ciągłości linii parkingowej za pomocą kamery RGB oraz z wykorzystaniem segmentacji barwnej w przestrzeni HSV⁵.
7. W ostatnim etapie zintegrowano wszystkie moduły za pośrednictwem nadrzędnego węzła sterującego, który koordynuje nawigację autonomiczną (Nav2), detekcję naruszeń oraz odczyt tablic rejestracyjnych w ramach jednej misji inspekcyjnej. Przeprowadzono testy integracyjne, na podstawie których dokonano kalibracji parametrów i oceniono skuteczność systemu.

1.4 Wartość edukacyjna projektu

Realizacja niniejszego projektu wiązała się z koniecznością samodzielnego opanowania szeregu narzędzi i technologii, z którymi autor nie miał wcześniej doświadczenia. Przed rozpoczęciem pracy znajomość systemów operacyjnych z rodziny Linux ograniczała się do poziomu podstawowego, a środowiska takie jak ROS 2, Gazebo czy RViz były całkowicie nowe. Podobnie zagadnienia związane z autonomiczną nawigacją, algorytmami SLAM oraz detekcją obiektów z wykorzystaniem sieci neuronowych wymagały zapoznania się zarówno z podstawami teoretycznymi, jak i praktycznymi aspektami implementacji.

Dodatkowym aspektem edukacyjnym było zetknięcie się z metodyką pracy typową dla projektów robotycznych, w której poszczególne moduły systemu są rozwijane, testowane i dostrajane

²End-to-end - proces realizujący zadanie od danych surowych do wyniku końcowego bez etapów pośrednich sterowanych ręcznie

³Framework - szkielet do budowy aplikacji dostarczający zestaw bibliotek i narzędzi.

⁴XACRO - XML Macros (do plików URDF)

⁵HSV (Hue, Saturation, Value) - model opisu przestrzeni barw, który rozdziela informację o kolorze (odcieniu) od jego nasycenia i jasności. W wizji komputerowej jest odporniejszy na zmiany oświetlenia niż model RGB

niezależnie, a następnie integrowane w ramach wspólnej architektury. Opanowanie umiejętności rozwiązywania problemu w ten sposób jest niezbędne do efektywnej pracy przy większych projektach realizowanych w zespołach, a więc w warunkach typowych dla codziennej pracy inżyniera.

Rozdział 2

Przegląd stanu wiedzy

2.1 ROS

Projektowany system składa się z wielu niezależnych modułów realizujących różne zadania, najważniejsze to:

- lokalizacja i mapowanie,
- nawigacja autonomiczna,
- detekcja tablic rejestracyjnych,
- ocena poprawności parkowania pojazdów.

Aby połączyć te komponenty w spójny, działający proces, niezbędna jest warstwa komunikacyjna zapewniająca wymianę danych między nimi. Rolę tę pełni w projekcie ROS 2, którego architektura dodatkowo zapewnia przenośność kodu między środowiskiem symulacyjnym a fizyczną jednostką, pozwala to na opracowanie systemu w symulacji i późniejsze wdrożenie na fizycznym robocie bez istotnych zmian.

Robot Operating System (ROS) to framework typu open-source¹ zaprojektowany w celu uproszczenia tworzenia systemów robotycznych. Umożliwia on integrację dowolnych robotów oraz komponentów, zarówno komercyjnych, jak i projektowanych indywidualnie. Mimo swojej nazwy ROS nie jest pełnoprawnym systemem operacyjnym, takim jak Windows czy Linux. Zamiast tego pełni funkcję warstwy pośredniej, która znajduje się pomiędzy systemem operacyjnym robota a poszczególnymi procesami [1].

ROS został pierwotnie zaprojektowany jako zunifikowana platforma do budowy aplikacji przeznaczonych dla pojedynczych robotów. Oferuje standardowy zestaw narzędzi, bibliotek i konwencji, które wspierają rozwój szerokiego spektrum aplikacji robotycznych, od prostych prototypów po złożone systemy robotyczne. Jednym z istotnych założeń projektowych ROS było promowanie ponownego wykorzystania i modularności, co umożliwia programistom łatwe udostępnianie oraz łączenie komponentów oprogramowania, a tym samym ułatwia tworzenie zaawansowanych rozwiązań robotycznych, co więcej, pozwala uniknąć pisania wszystkiego od podstaw [2].

Dla lepszego zrozumienia ROS można porównać do cyfrowego układu nerwowego robota. To elastyczny framework, który zapewnia ustandaryzowany sposób komunikacji pomiędzy różnymi częściami robota (organami) [3].

2.1.1 Architektura ROS 2

Architektura ROS 2 bazuje na węzłach (node), które stanowią niezależne procesy realizujące określone zadania obliczeniowe. Kompletny system robotyczny składa się zazwyczaj z wielu współpracujących węzłów, z których każdy odpowiada za wybrany aspekt działania robota. Przykładowo, jeden węzeł może zajmować się przetwarzaniem obrazu z kamery, inny planowaniem trajektorii ruchu, a kolejny generowaniem sygnałów sterujących dla napędów. Taki modułowy model umożliwia niezależne projektowanie, uruchamianie oraz testowanie poszczególnych komponentów systemu [4]. Architekturę ROS 2 ilustruje rysunek 2.1.

Komunikacja pomiędzy węzłami realizowana jest poprzez procedury publikacji i subskrypcji (publish-subscribe) tematów. Temat (topic) stanowi nazwany kanał wymiany danych określonego typu. Węzły publikujące (publisher) przesyłają komunikaty do danego tematu, natomiast węzły subskrybujące (subscriber) odbierają te dane. Zastosowanie modelu publikacja-subskrypcja prowadzi

¹Open-source - model tworzenia oprogramowania, w którym kod źródłowy jest publicznie dostępny i może być używany oraz modyfikowany

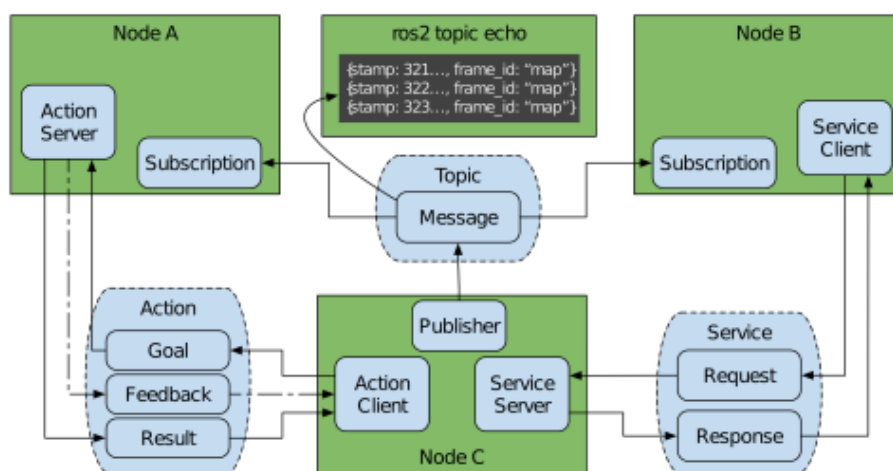
do luźnego powiązania elementów systemu, ponieważ węzeł publikujący nie musi posiadać wiedzy o odbiorcach, a węzeł subskrybujący nie jest zależny od źródła danych [4].

Struktura danych przesyłanych pomiędzy węzłami definiowana jest przez wiadomości (messages). ROS 2 udostępnia zestaw standardowych typów wiadomości przeznaczonych do opisu typowych danych robotycznych, takich jak odczyty z czujników, pozycje i orientacje obiektów czy obrazy. Przykładem jest wiadomość `geometry_msgs/Twist`, która opisuje składowe prędkości liniowej i kątowej wykorzystywane do sterowania robotami mobilnymi. W przypadku bardziej specyficznych zastosowań możliwe jest również definiowanie własnych typów wiadomości [4].

Zrozumienie relacji pomiędzy węzłami, tematami i wiadomościami stanowi podstawę pracy z ROS 2. Tematy można porównać do kanałów nadawczych, które transmitują określony rodzaj informacji, natomiast węzły pełnią rolę nadajników i odbiorników, publikując dane lub odbierając interesujące je wiadomości [4].

W ROS 2 dostępny jest również wzorzec komunikacji typu żądanie-odpowiedź (request-response), określany jako usługi (services). Taki model komunikacji umożliwia jednoznaczne powiązanie wysłanego żądania z otrzymaną odpowiedzią, co jest szczególnie przydatne w sytuacjach wymagających potwierdzenia wykonania zadania lub poprawnego odebrania polecenia. Usługi są przypisane do węzłów, co ułatwia organizację systemu [5].

Unikalnym sposobem komunikacji w ROS 2 są akcje (actions). Akcje stanowią asynchroniczny, zorientowany na cel interfejs komunikacyjny, który obejmuje wysłanie żądania, odebranie odpowiedzi, cykliczne przekazywanie informacji zwrotnej oraz możliwość anulowania realizowanego zadania. Sposób ten jest wykorzystywany głównie w przypadku zadań długotrwałych, takich jak autonomiczna nawigacja lub manipulacja, jednak znajduje zastosowanie również w innych scenariuszach. Tak jak usługi, akcje są nieblokujące oraz logicznie przypisane do węzłów systemu [5].



Rysunek 2.1: Architektura wymiany danych w ROS 2 [5]

2.1.2 Ewolucja ROS - różnice między ROS 1 a ROS 2

ROS 2 powstał jako odpowiedź na ograniczenia pierwszej wersji architektury ROS (ROS 1), różnice między kolejnymi wersjami w ujęciu ogólnym pokazuje rysunek 2.2. Trzy główne obszary zmian to: architekturę systemu, dostępne funkcjonalności oraz narzędzia i ekosystem.

Architektura

ROS 1 wykorzystuje architekturę typu master-slave² oraz sposób komunikacji bazujący na XML-RPC³ W ROS 2 zastosowano standard Data Distribution Service⁴ (DDS), który został zaprojektowany z myślą o wysokiej wydajności, niezawodności, niskich opóźnieniach oraz skalowalności systemów rozproszonych. DDS umożliwia także konfigurację parametrów jakości usług (QoS⁵), co pozwala precyzyjnie kontrolować sposób przesyłania danych, obejmując one m.in. niezawodność transmisji, wymagania czasowe oraz priorytety komunikatów. Standard XML-RPC sprawdza się w prostych wywołaniach procedur zdalnych, jednak większa złożoność DDS pozwala na skuteczniejsze

²Master-slave - architektura, w której jeden proces (master) sprawuje kontrolę nad pozostałymi (slave).

³XML-RPC (Extensible Markup Language Remote Procedure Call) - protokół zdalnego wywoływania procedur, wykorzystujący język XML do kodowania danych oraz protokół HTTP do ich transportu.

⁴DDS - standard komunikacji publish-subscribe, często używany w systemach czasu rzeczywistego.

⁵QoS - zestaw polityk jakości komunikacji, które pozwalają kontrolować zachowanie transportu danych.

wsparcie systemów czasu rzeczywistego. Ponadto rozproszony charakter DDS eliminuje pojedyncze punkty awarii w komunikacji, które były charakterystyczne dla architektury ROS 1 [6].

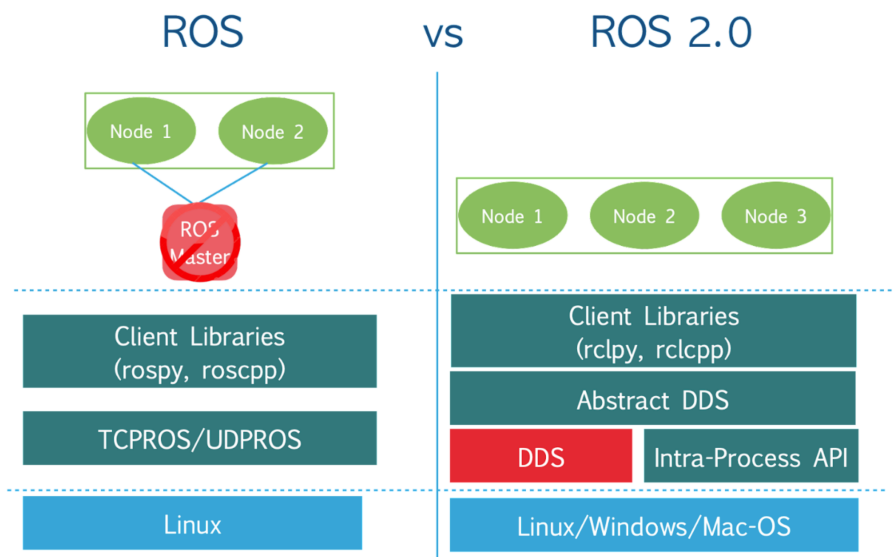
Funkcjonalności

W przeciwieństwie do ROS 1, ROS 2 umożliwia rzeczywiste równoległe wykonywanie wielu węzłów, co pozwala efektywnie wykorzystywać nowoczesne procesory wielordzeniowe. Połączenie standardu DDS, konfiguracji QoS oraz obsługi wielowątkowości sprawia, że ROS 2 znacznie lepiej nadaje się do zastosowań czasu rzeczywistego, w szczególności tam, gdzie wymagane są deterministyczne opóźnienia i wysoka przewidywalność komunikacji [6].

Narzędzia i ekosystem

ROS 2 nie jest wstecznie kompatybilny z ROS 1, co oznacza, że pakiety przeznaczone dla ROS 1 wymagają dostosowania lub ponownej implementacji. Chociaż stanowi to istotne ograniczenie, narzędzia takie jak ROSbridge umożliwiają częściowe łagodzenie problemów migracyjnych.

Podczas gdy ROS 1 był projektowany głównie z myślą o systemie Ubuntu, ROS 2 oferuje wsparcie dla wielu systemów operacyjnych, w tym macOS, Windows oraz różnych dystrybucji systemu Linux [6].



Rysunek 2.2: Porównanie ROS 1 i ROS 2 [2]

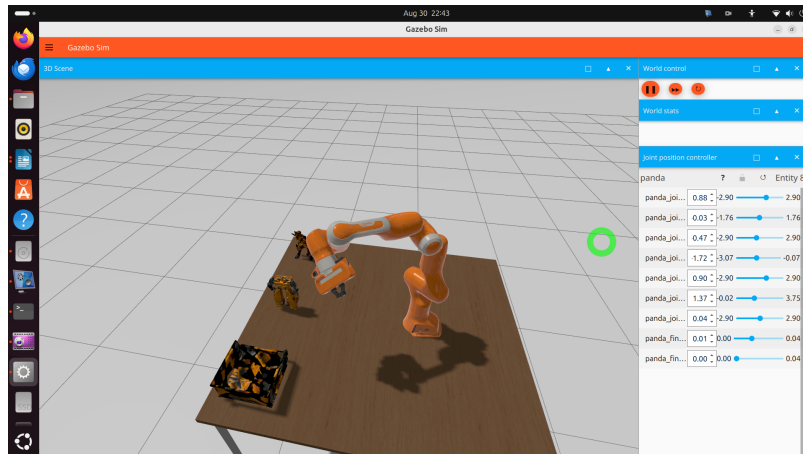
2.2 Gazebo

W niniejszej pracy jako środowisko symulacyjne wybrano Gazebo, silnik symulacyjny zintegrowany z ROS 2, umożliwiający modelowanie zarówno robota za pomocą plików URDF/XACRO, jak i otoczenia, w którym robot operuje (pliki SDF⁶), przykładowy świat stworzony za pomocą omówionych plików pokazuje rysunek 2.3. Silnik fizyczny Gazebo odwzorowuje dynamikę ciał sztywnych, grawitację, czy kolizje. Czujniki symulowane za pomocą wbudowanych wtyczek generują dane zbliżone do tych, które dostarczałyby rzeczywiste sensory. Algorytmy sterowania, nawigacji i percepcji uruchamiane w symulacji korzystają z tych samych interfejsów ROS 2 (topic, service, action) co oznacza, że w przypadku przeniesienia symulacji na fizycznego robota jedyną różnicą jest źródło danych [7]. Dzięki temu można testować system bez konieczności dysponowania rzeczywistym robotem, a także na bezpieczną weryfikację scenariuszy, które w warunkach fizycznych byłyby trudne, kosztowne lub ryzykowne.

Wykorzystanie symulacji komputerowych stanowi dobrą praktykę w procesie projektowania nowych konstrukcji robotycznych oraz algorytmów sterowania, jeszcze przed implementacją i uruchomieniem kodu na rzeczywistych systemach robotycznych. Wraz ze wzrostem dynamiki oraz złożoności robotów i towarzyszącego im oprogramowania, narzędzia symulacyjne są coraz częściej stosowane

⁶SDF (Simulation Description Format) - format opisu obiektów oparty na języku XML, wykorzystywany przez symulator Gazebo do definiowania geometrii, właściwości fizycznych oraz wizualnych elementów otoczenia i modeli.

do odwzorowywania realistycznego ruchu trójwymiarowych modeli robotów w zróżnicowanych środowiskach wirtualnych. W ostatnich latach liczba dostępnych narzędzi do symulacji dynamiki robotów znacząco wzrosła, co jest bezpośrednio związane z rozwojem mocy obliczeniowej oraz obniżeniem kosztów sprzętu komputerowego [8, 9]. Jednocześnie porównanie dostępnych symulatorów pozostaje zadaniem skomplikowanym, ponieważ poszczególne środowiska symulacyjne różnią się pod względem jakościowych cech funkcjonalnych i często wymagają dostosowania do rozwiązywania specyficznych problemów. Większość z nich nie była projektowana jako narzędzia ogólnego przeznaczenia [10].



Rysunek 2.3: Przykładowa symulacja w Gazebo [11]

2.2.1 Porównanie

Dobór odpowiedniego symulatora robotycznego zależy od wielu kryteriów, a badacze oraz praktycy, kierując się odmiennymi potrzebami i celami, stosują różne miary oceny jego przydatności [12]. Przeanalizowano badania porównujące różne silniki symulacyjne, w pracy zaprezentowano wyniki dwóch z nich:

1. *How to pick a mobile robot simulator: a quantitative comparison of CoppeliaSim, Gazebo, MORSE and Webots with a focus on accuracy of motion* - Andrew Farley, Jie Wang, Joshua A. Marshall [12].
2. *Robotics simulation – a comparison of two state-of-the-art solutions* - Maximilian Zwingel, Christopher May, Matthias Kalenberg, Jörg Franke [13].

Pierwsze skupia się na porównaniu czterech popularnych symulatorów robotów: CoppeliaSim (wcześniej znany jako V-REP), Gazebo, MORSE oraz Webots. Druga natomiast porównuje Gazebo oraz NVIDIA Isaac Sim w odniesieniu do bazujących na ROS robotów mobilnych.

Wyniki pierwszych badań pokazano na rysunku 2.4, kryteria punktacji dla niektórych metryk przedstawia tabela 2.1. Niedokładność odometrii wyznaczono porównując pozycję i orientację robota raportowaną przez symulator z danymi zmierzonymi na fizycznym robocie. Użyte wzory do wyznaczenia wartości znajdujących się na rysunku 2.4, a nie opisanych przez tabelę 2.1 wymagałyby szerokiego omówienia, dlatego je pominięto, dostępne są w badaniu.

Tabela 2.1: Kryteria oceny cech jakościowych symulatorów (na podstawie [12])

Kryterium	Opis
Darmowy	Symulator bezpłatny: 1 pkt, płatny: 0 pkt
Open source	Otwarty kod źródłowy: 1 pkt, zamknięty: 0 pkt
Kompatybilność z ROS	Oficjalne wsparcie ROS: 1 pkt; plugin instalowany ręcznie: 0.8 pkt; brak: 0 pkt
Języki programowania	C/C++, Python, MATLAB i więcej: 1 pkt; C/C++ i Python: 0.667 pkt; C/C++ lub Python: 0.333 pkt; brak: 0 pkt
Interfejs użytkownika	Pełna kontrola (start, stop, krokowanie, dodawanie modeli): 1 pkt; częściowa: 0.667 pkt; tylko podgląd: 0.333 pkt; brak UI: 0 pkt
Obsługa formatów modeli	URDF, SDF i więcej: 1 pkt; URDF lub SDF: 0.5 pkt; przez plugin: 0.25 pkt; brak: 0 pkt
Silniki fizyczne	Więcej niż jeden silnik: 1 pkt; jeden silnik: 0 pkt

Metric Name	Weight	CoppeliaSim	Gazebo	MORSE	Webots
Free To Use	4	1.000	1.000	1.000	1.000
Open Source	2	0	1.000	1.000	1.000
ROS Integration	6	0.800	1.000	1.000	0.800
Programming Languages	3	1.000	0.667	0.333	1.000
UI Functionality	6	1.000	1.000	0.333	1.000
Model Format Support	4	1.000	1.000	0	0.250
Physics Engine Support	3	1.000	1.000	0	0
Real Time Factor	4	0.914	1.000	0.789	0.849
Average Load CPU Efficiency	2	0.364	0.174	0.333	1.000
Intense Load CPU Efficiency	2	0.583	0.304	0.583	1.000
IMU Angular Velocity Accuracy	10	0.875	1.000	0.792	0.867
IMU Linear Acceleration Accuracy	10	1.000	0.713	0.424	0.736
Total	56	49.100	49.087	32.148	44.226

Rysunek 2.4: Porównanie czterech popularnych symulatorów robotów: CoppeliaSim (wcześniej znany jako V-REP), Gazebo, MORSE oraz Webots. [12]

W pierwszym badaniu stwierdzono:

Najwyższą dokładność odwzorowania dla odometrii uzyskał symulator CoppeliaSim. Oprogramowanie to oferuje również najszersze wsparcie pod względem liczby obsługiwanych języków programowania, silników fizycznych oraz typów modeli, co czyni je najbardziej wszechstronnym spośród analizowanych narzędzi. Gazebo uzyskał jedynie nieznacznie niższe wyniki w porównaniu z CoppeliaSim, co pozwala uznać go za bardzo dobrą alternatywę w zastosowaniach symulacyjnych [12].

Dla badania porównującego Gazebo z NVIDIA Isaac Sim (NIS) autorzy stwierdzają, że NIS nie jest jeszcze gotowy, aby zastąpić Gazebo jako główne narzędzie symulacyjne dla robotów mobilnych wykorzystujących ROS. W momencie pisania artykułu NIS stawiał przed użytkownikiem wiele przeszkód. Część ograniczeń można obejść opracowując własne rozwiązania, jednak rzadko są one optymalne. Gazebo pod względem symulacji fizyki wciąż pozostaje na porównywalnym poziomie z NIS. Niemniej jednak w czasie użytkowania autorzy zaobserwowali znaczące postępy w rozwoju NIS i dostrzegają w nim duży potencjał. Jeśli użytkownik ma dostęp do systemu zdolnego do uruchomienia NIS, wypróbowanie go może być opłacalne w zależności od rodzaju projektu (patrz rysunek 2.5) [13].

Demand	Better suitable simulator
Physics simulation	Gazebo
Evaluation of vision-based algorithms	NVIDIA Isaac Sim
Evaluation of optical-/laser-based algorithms	Gazebo (until NIS utilizes raytracing for laser simulation)
Project duration	Short- & medium-term: Gazebo Long-term: NVIDIA Isaac Sim
Simulation of UAVs or UAVs	Gazebo
Availability of moving objects and persons	Gazebo

Table 4: Simulator recommendations for different demands

Rysunek 2.5: Gazebo czy NIS dla danych zastosowań [13].

2.3 Roboty mobilne

Przeprowadzenie symulacji w środowisku ROS 2 wymaga wcześniejszego opracowania modelu robota - symulacja nie jest procesem abstrakcyjnym i musi działać na podstawie określonej struktury mechanicznej, modelu kinematycznego oraz założeń dotyczących napędów i czujników. Model taki może stanowić zarówno konstrukcję czysto koncepcyjną, jak i odwzorowanie fizycznego robota. W obu przypadkach konieczne jest świadome dobranie komponentów, które często bazują na rzeczywistych podzespołach dostępnych na rynku.

W ROS 2 opis robota realizowany jest za pomocą formatu URDF (Unified Robot Description Format), który definiuje geometrię, kinematykę, właściwości fizyczne oraz konfigurację czujników i napędów. Tak zdefiniowany model stanowi punkt wejścia zarówno dla symulatora Gazebo, jak i dla pozostałych komponentów systemu - od wizualizacji w RViz po planowanie trajektorii w Nav2 [14]. Podejście to wpisuje się w koncepcję cyfrowego bliźniaka (Digital Twin), czyli wirtualnej repliki systemu fizycznego, której celem jest możliwie wierne odwzorowanie struktury, dynamiki i zachowania rzeczywistego obiektu [15].

W sekcji 2.3.1 i 2.3.2 omówiono wybrane platformy robotyczne, które mogłyby stanowić bazę sprzętową dla realizowanego projektu, a także istniejące rozwiązanie przemysłowe (sekcja 2.3.3) realizujące zbliżone zadanie - automatyczną inspekcję parkingów z rozpoznawaniem tablic rejestracyjnych. Analiza tych platform pozwala na porównanie dostępnych podejść oraz uzasadnienie wyboru docelowej platformy zastosowanej w projekcie.

2.3.1 TurtleBot4

TurtleBot4 (TB4) jest mobilną platformą badawczo-edukacyjną zaprojektowaną z myślą o pracy w środowisku ROS 2. Stanowi rozwinięcie wcześniejszych wersji TurtleBota i jest obecnie jedną z najczęściej wykorzystywanych platform do badań nad autonomiczną nawigacją, SLAM oraz percepcją wizyjną. Platforma TurtleBot dostarczana jest wraz z wstępnie skonfigurowanym i zainstalowanym środowiskiem Gazebo, szczegółową dokumentacją użytkownika, przykładowymi implementacjami oraz modelem symulacyjnym. Umożliwia to szybkie rozpoczęcie pracy oraz rozwijanie aplikacji robotycznych. Dzięki rozbudowanym materiałom edukacyjnym i samouczkom użytkownik może efektywnie rozpocząć naukę oraz eksperymenty z wykorzystaniem systemu. Platforma ta stanowi element dynamicznie rozwijającego się, otwartoźródłowego ekosystemu ROS, skupiającego szeroką społeczność deweloperów i badaczy [16].

Dokładna specyfikacja TurtleBot4

TurtleBot4 dostępny jest w dwóch wersjach, dokładna specyfikacja każdej z nich została przedstawiona na rysunku 2.6.

Konfiguracja sprzętowa pozwala na realizację typowych zadań robotyki mobilnej.

- LiDAR RPLiDAR-A1 z zasięgiem do 12 m i pełnym skanem 360° umożliwia budowanie map otoczenia (SLAM) oraz wykrywanie przeszkód w czasie rzeczywistym,
- Kamera OAK-D-PRO, łącząca sensor RGB o rozdzielczości 4K z parą kamer stereo i projektorem IR, jest dobrą bazą sprzętową dla detekcji i rozpoznawania obiektów, zapewnia też percepcję głębi,
- Wbudowana jednostka IMU (żyroskop i akcelerometr 3-osiowy) wraz z enkoderami kół dostarcza danych do estymacji pozycji i orientacji robota, które po fuzji sensorycznej pozwalają na dokładną lokalizację,
- Czujniki podczerwieni oraz strefy zderzakowe stanowią dodatkową warstwę bezpieczeństwa, zapobiegając kolizjom i spadkom z krawędzi.

Całość zarządzana jest przez komputer Raspberry Pi 4B z systemem Ubuntu i frameworkiem ROS 2, co zapewnia pełną kompatybilność z pakietami nawigacyjnymi takimi jak Nav2 oraz narzędziami do mapowania i lokalizacji [16].

Komunikacja między komponentami realizowana jest z wykorzystaniem standardu DDS, co zapewnia skalowalność, odporność na awarie oraz możliwość integracji z rozproszonymi systemami robotycznymi [16].

TECHNICAL SPECIFICATIONS



	TurtleBot 4	TurtleBot 4 Lite
SIZE AND WEIGHT		
EXTERNAL DIMENSIONS (LxWxH)	341 x 339 x 351 mm [13.4 x 13.3 x 13.8 in]	341 x 339 x 192 mm [13.4 x 13.3 x 7.5 in]
WEIGHT	3.9 kg [8.6 lbs]	3.3 kg [7.2 lbs]
WHEELS (Diameter)	72 mm [0.55 in]	
GROUND CLEARANCE	4.5 mm [0.17 in]	
SPEED AND PERFORMANCE		
MAX PAYLOAD	9 Kg - Default 15 kg - Custom Configuration	
MAX SPEED	0.31 m/s (safe mode), 0.46 m/s (cliff sensors disabled)	
MAX ROTATIONAL SPEED	1.90 rad/s	
BATTERY AND POWER SYSTEM		
CHEMISTRY	26 Wh Lithium Ion (14.4V nominal) Rechargeable	
CHARGE TIME	2.5 hrs	
OPERATINGTME	2.5 - 4.0 hrs (load dependent)	
USER POWER	VBAT @ 300mA, 12V @300mA, 5V @ 500mA 3.3V @ 250mA	VBAT @ 1.9A (14.4V nominal) Low current 5V and 3.3V via Raspeberry Pi GPIO
DOCKING	Base station for docked charging	
SENSORS		
2D LIDAR	RPLIDAR-A1 0.15-12m range; 8kHz sampling rate; 360 degree angular range; 1 degree angular resolution.	
CAMERA	OAK-D-PRO 4K RGB auto focus camera (IMX378) Mono stereo camera pair (OV9282) IMU, Spatial AI processor IR Laser Dot Projector & Illumination LED	OAK-D-LITE 4K RGB auto focus camera (IMX214) Mono stereo camera pair (OV251) Spatial AI processor
OTHER SENSORS	2x front bumper zones, 2x wheel encoders, 4x IR cliff sensors, 6x IR obstacle sensors, 1x downward optical flow sensor for odometry, 1x 3D gyroscope, 1x 3D accelerometer, 1x battery level monitor	
OTHER ACTUATORS	2x Drive Motors, 6x RGB LED Ring 1x Speaker, 5 Status LEDs 2 User LEDs, 128x64 OLED Display	2x Drive Motors, 6x RGB LED Ring 1x Speaker
COMPUTERS		
COMPUTER	Raspberry Pi 4B [4 GB]	
SOFTWARE	Ubuntu 20.04, ROS 2	

Support Open Source

TurtleBot 4 is an open source platform and you can get the designs and software at clearpathrobotics.com/turtlebot. You're encouraged to try it out, extend it, or build your own. For every TurtleBot 4 shipped, a portion of the proceeds goes to the Open Robotics for the continued support of the development, distribution, and adoption of open-source software in robotics research and education.

© Clearpath Robotics, Inc. All Rights Reserved. Clearpath Robotics, and clearpathrobotics.com are trademarks of Clearpath Robotics. All other product and company names listed are trademarks or trade names of their respective companies. † Accessible USB and user power ports, top mounting plate are only available in the TurtleBot 4 Standard model.



Rysunek 2.6: Karta katalogowa TurtleBot4 [16]

2.3.2 Alternatywne platformy – ROSbot

Alternatywną platformą mobilną o zbliżonych możliwościach jest ROSbot, opracowany przez polską firmę Husarion. Platforma, dostępna w wariantach ROSbot 3 (rysunek 2.7) oraz ROSbot 3 PRO, również przeznaczona jest do badań, edukacji i prototypowania systemów autonomicznych bazujących na ROS i ROS 2 [17].

ROSbot posiada zbliżone wyposażenie sensoryczne co TurtleBot4, czyli:

- **LiDAR 2D** (np. SLAMTEC C1 lub S2),
- **Kamerę RGB-D** (np. OAK-D Lite lub OAK-D PRO),
- **IMU** (np. BNO055),
- **Czujniki Time-of-Flight** do wykrywania przeszkód w bliskim otoczeniu,

- **Enkodery kół**

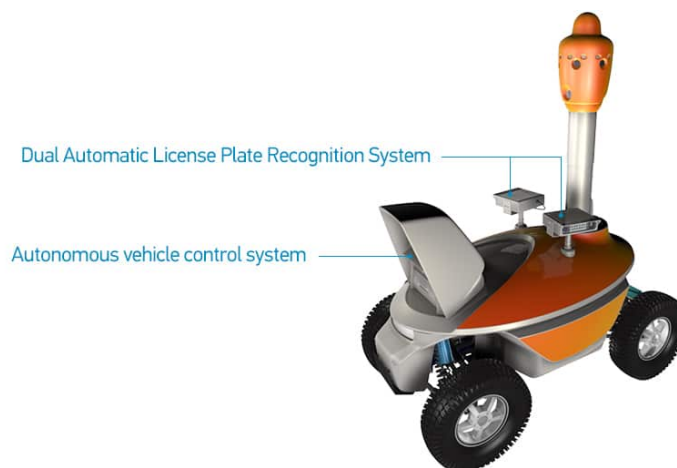
Podobnie jak TurtleBot4, platforma natywnie wspiera integrację z istotnymi narzędziami ekosystemu ROS 2, takimi jak SLAM Toolbox, Nav2, RViz czy Gazebo, a komunikacja realizowana jest poprzez architekturę DDS [17]. Wybór między tymi platformami zależy głównie od preferencji użytkownika oraz specyficznych wymagań projektu.



Rysunek 2.7: ROSbot [17]

2.3.3 ALPR Robot

Robot S5 ALPR⁷ firmy SMP Robotics (rysunek 2.8) jest przemysłową platformą mobilną przeznaczoną do automatycznego rozpoznawania tablic rejestracyjnych podczas patrolowania parkingów i innych obszarów postojowych. Zgodnie z informacjami producenta robot porusza się autonomicznie z prędkością zbliżoną do marszu pieszego, omija przeszkody, rejestruje tablice rejestracyjne wraz z lokalizacją pojazdów oraz może pełnić funkcję mobilnego systemu nadzoru wizyjnego dzięki zestawowi kamer zapewniających widok panoramiczny 360° oraz transmisji danych przez Wi-Fi. Wersja DUO umożliwia jednoczesny odczyt tablic po obu stronach robota, a rozwiązanie może być wykorzystywane m.in. do monitoringu parkingów podziemnych, rejestracji naruszeń zasad parkowania oraz tworzenia bazy lokalizacji pojazdów [18]. Jest to zamknięte rozwiązanie komercyjne o charakterze przemysłowym, producent nie udostępnia szczegółowej dokumentacji technicznej, kodu źródłowego ani środowiska symulacyjnego, co uniemożliwia swobodne testowanie, modyfikowanie i rozwijanie platformy bez jej zakupu.



Rysunek 2.8: ALPR Robot [18]

2.4 RViz

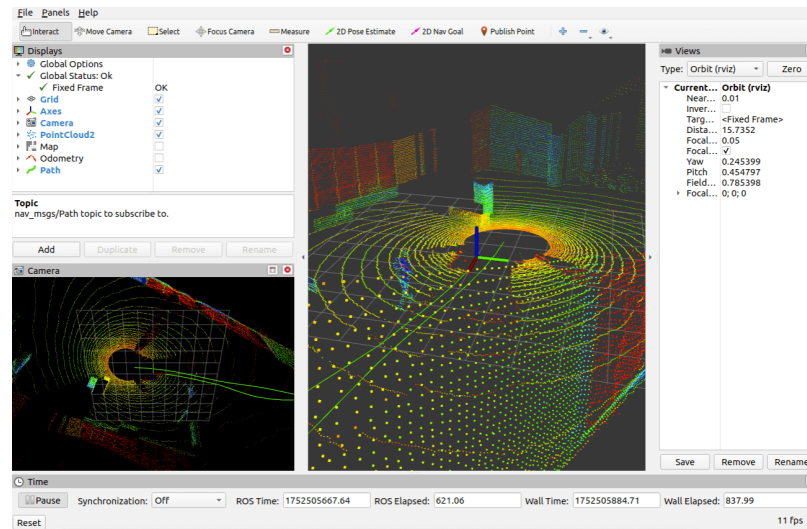
W procesie rozwoju systemów robotycznych istotną rolę odgrywają narzędzia wizualizacji i analizy danych, które umożliwiają monitorowanie, debugowanie oraz interpretację złożonych da-

⁷ALPR (Automatic License Plate Recognition) - termin bliźniaczy do ANPR, nazywa się tak systemy do automatycznego odczytywania tablic rejestracyjnych

nych sensorycznych (rysunek 2.9). Platformy takie jak RViz, Foxglove oraz Rerun stanowią element warstwy wizualizacji w architekturze obserwowalności systemu. Umożliwiają one pracę zarówno z danymi przesyłanymi w czasie rzeczywistym, jak i z zapisami archiwalnymi. Skuteczne działanie tych narzędzi wymaga terminowego oraz odpowiednio ustrukturyzowanego dostępu do strumieni danych [19].

Każda z wymienionych platform pełni odmienną funkcję w procesie projektowania i testowania systemów robotycznych. RViz (ROS Visualization) jest klasycznym narzędziem trójwymiarowej wizualizacji przeznaczonym dla ekosystemu ROS i jest powszechnie wykorzystywany do monitorowania pracy robota oraz analizy jego działania w czasie rzeczywistym [19].

Dobór odpowiedniego narzędzia zależy od kontekstu zastosowania. W przypadku pracy w środowisku ROS, szczególnie podczas uruchamiania i debugowania systemu w czasie rzeczywistym, RViz stanowi podstawowe narzędzie wizualizacyjne, które pomaga w tworzeniu symulacji [19, 20].



Rysunek 2.9: Interfejs graficzny RViz [19]

Aplikacja pozwala na wizualizację modeli robotów opisanych w formacie URDF, danych z czujników takich jak LiDAR, kamera czy IMU, a także trajektorii ruchu, map tworzonych w procesie SLAM, transformacji pomiędzy układami współrzędnych (TF) oraz wielu innych danych [21].

RViz działa jako system wyświetlaczy (display plugins), które subskrybują wybrane tematy ROS i interpretują ich zawartość w postaci graficznej. Dzięki temu możliwe jest jednoczesne monitorowanie wielu strumieni danych w czasie rzeczywistym, co znacząco ułatwia debugowanie oraz analizę zachowania systemu robotycznego [22].

W niniejszym projekcie RViz użyto jako główne narzędzie wizualizacyjne. Możliwość jednoczesnego podglądu wszystkich interesujących nas informacji z czujników w jednym oknie pozwala na bieżącą weryfikację działania poszczególnych modułów systemu. Bezpośrednia integracja z ROS 2 oraz brak konieczności dodatkowej konfiguracji poza wyborem odpowiednich tematów sprawiają, że RViz jest odpowiednim narzędziem dla tego projektu.

2.5 SLAM

W niniejszej pracy zakłada się, że robot powinien móc działać również w nowym, wcześniej nieopisanym środowisku. Potrzebny jest więc mechanizm, który pozwoli mu mapować środowisko podczas ruchu i równolegle określać własną pozycję.

Problem jednoczesnej lokalizacji i mapowania (SLAM, ang. Simultaneous Localization and Mapping) dotyczy zagadnienia, czy możliwe jest umieszczenie mobilnego robota w nieznanym lokalizacji, w nieznanym środowisku, a następnie umożliwienie mu stopniowego budowania spójnej mapy otoczenia przy jednoczesnym wyznaczaniu własnego położenia względem tej mapy. Rozwiązanie problemu SLAM przez długi czas było postrzegane jako jedno z istotnych wyzwań robotyki mobilnej, ponieważ jego skuteczna realizacja stanowi fundament dla osiągnięcia rzeczywistej autonomii systemów robotycznych [23].

Problem SLAM wymaga zdefiniowania odpowiednich reprezentacji zarówno dla modelu obserwacji, jak i modelu ruchu [23]. W tym celu pojazd musi być wyposażony w system sensoryczny

umożliwiający wykonywanie pomiarów względnego położenia pomiędzy charakterystycznymi punktami otoczenia a samym robotem [24].

2.5.1 Źródła informacji wykorzystywane w SLAM

Wielu badaczy oraz zespołów naukowych zajmowało się i nadal zajmuje problematyką SLAM. Najczęściej wykorzystywane czujniki w systemach tego typu można podzielić na trzy główne kategorie: systemy sonarowe, laserowe (LiDAR-SLAM) oraz wizyjne (vSLAM). Dodatkowo stosowane są inne źródła informacji sensorycznej w celu lepszego określenia stanu robota oraz percepcji otoczenia [25], takie jak kompas, czujniki podczerwieni czy system nawigacji satelitarnej GPS. Należy jednak pamiętać o tym, że wszystkie wymienione czujniki obarczone są określonymi błędami pomiarowymi, określanymi jako szum pomiarowy oraz posiadają ograniczenia zasięgu działania [24].

Warto zauważyć, że klasyczne podejście polegające na rozważaniu pojedynczych klas czujników jako głównego źródła danych jest dziś w praktyce rzadziej spotykane. W nowoczesnych systemach SLAM coraz częściej stosuje się algorytmy działające na zasadzie fuzji sensorycznej, które łączą informacje z wielu źródeł, aby zwiększyć odporność na zakłócenia i poprawić stabilność estymacji położenia oraz mapy [26]. Zarys działania algorytmów SLAM działających na zasadzie fuzji sensorycznej przedstawia rysunek 2.10.

LiDAR-SLAM

Roboty mobilne są powszechnie wykorzystywane w środowiskach wewnętrznych. W takich zastosowaniach czujniki 2D LiDAR stały się standardowym rozwiązaniem w nawigacji oraz lokalizacji, głównie ze względu na wysoką dokładność pomiaru odległości oraz stosunkowo niewielką ilość generowanych danych. Wraz ze wzrostem zapotrzebowania na pracę robotów w środowiskach zewnętrznych, systemy te zaczęły być wykorzystywane w coraz bardziej złożonych i otwartych przestrzeniach [27, 28]. W jego wyniku wieloliniowe skanery 3D LiDAR zyskały znaczną popularność i zaczęły być szeroko stosowane w aplikacjach terenowych.

Systemy SLAM bazujące na czujnikach laserowych wykorzystują pomiary odległości do rekonstrukcji geometrii otoczenia oraz estymacji trajektorii robota. W klasycznych podejściach 2D LiDAR generuje skany w postaci zbioru punktów reprezentujących odległości od przeszkód w określonych kierunkach. Na podstawie kolejnych skanów realizowane jest dopasowanie skanów (scan matching), które pozwala wyznaczyć względne przemieszczenie robota [29].

W przypadku systemów 3D LiDAR wykorzystywana jest gęsta chmura punktów, co pozwala na dokładniejsze odwzorowanie środowiska, jednak wiąże się ze zwiększoną złożonością obliczeniową [26].

vSLAM

System vSLAM jest bardziej zaawansowany technicznie w porównaniu z metodami SLAM działającymi na podstawie innych czujników. Wynika to z faktu, że kamery pozyskują informacje wizualne z ograniczonego pola widzenia, podczas gdy w robotyce mobilnej powszechnie stosowane są skanery laserowe oferujące obserwację w zakresie zbliżonym do 360°, co zapewnia pełniejsze pokrycie otoczenia.

Struktura systemu składa się zasadniczo z trzech podstawowych modułów:

1. inicjalizacji,
2. śledzenia (tracking),
3. mapowania (mapping).

Rozpoczęcie działania systemu vSLAM wymaga zdefiniowania układu współrzędnych, w którym estymowana będzie pozycja kamery oraz rekonstruowana struktura trójwymiarowa nieznanego środowiska. W etapie inicjalizacji wyznaczany jest globalny układ odniesienia, a następnie rekonstruowany jest fragment otoczenia, tworzący początkową mapę w tym układzie [30].

Po zakończeniu inicjalizacji realizowane są równoległe procesy śledzenia oraz mapowania, umożliwiające ciągłą estymację pozycji kamery. W module śledzenia aktualna mapa jest wykorzystywana do wyznaczenia pozycji kamery względem tej mapy poprzez analizę obrazu. W tym celu wyznaczane są odpowiadające sobie punkty 2D–3D pomiędzy obrazem a mapą, uzyskane na podstawie dopasowania cech lub ich śledzenia w kolejnych klatkach. Następnie pozycja kamery obliczana jest poprzez rozwiązywanie problemu Perspective-n-Point (PnP) [31, 32].

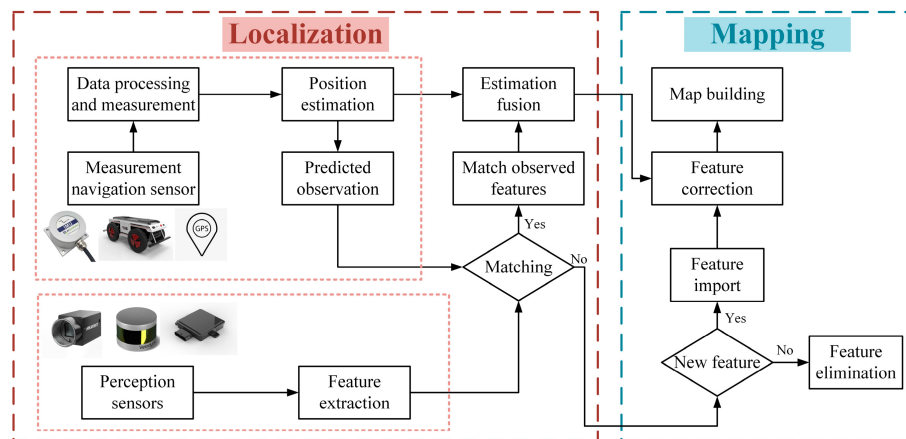
W większości algorytmów vSLAM zakłada się, że parametry wewnętrzne kamery są wcześniej skalibrowane i znane. W konsekwencji estymowana pozycja kamery odpowiada parametrom zewnętrznym, określającym translację i rotację kamery w globalnym układzie współrzędnych. Mo-

duł mapowania odpowiada za rozszerzanie mapy poprzez rekonstrukcję trójwymiarowej struktury środowiska w chwilach, gdy kamera obserwuje obszary wcześniej nieujęte w mapie [30].

Wielosensoryczny SLAM

Jednym z najczęściej stosowanych podejść jest integracja skanera LiDAR z jednostką inercyjną IMU, znana jako LiDAR-Inertial Odometry (LIO). W tego typu systemach pomiary przyspieszeń i prędkości kątowych umożliwiają poprawę estymacji krótkoterminowego ruchu, natomiast dane LiDAR wykorzystywane są do korekty dryfu⁸ oraz budowy mapy. [33, 26].

Często spotykaną, ulepszoną wersją klasycznego vSLAM jest podejście visual-inertial (VIO visual-inertial SLAM), w którym dane z kamery są łączone z pomiarami IMU. Dzięki temu możliwe jest stabilniejsze śledzenie ruchu w krótkim horyzoncie czasowym oraz ograniczenie problemów typowych dla metod bazujących wyłącznie na obrazie (np. rozmycie, szybkie obroty, chwilowy brak cech w scenie), co w praktyce poprawia odporność i dokładność estymacji [34].

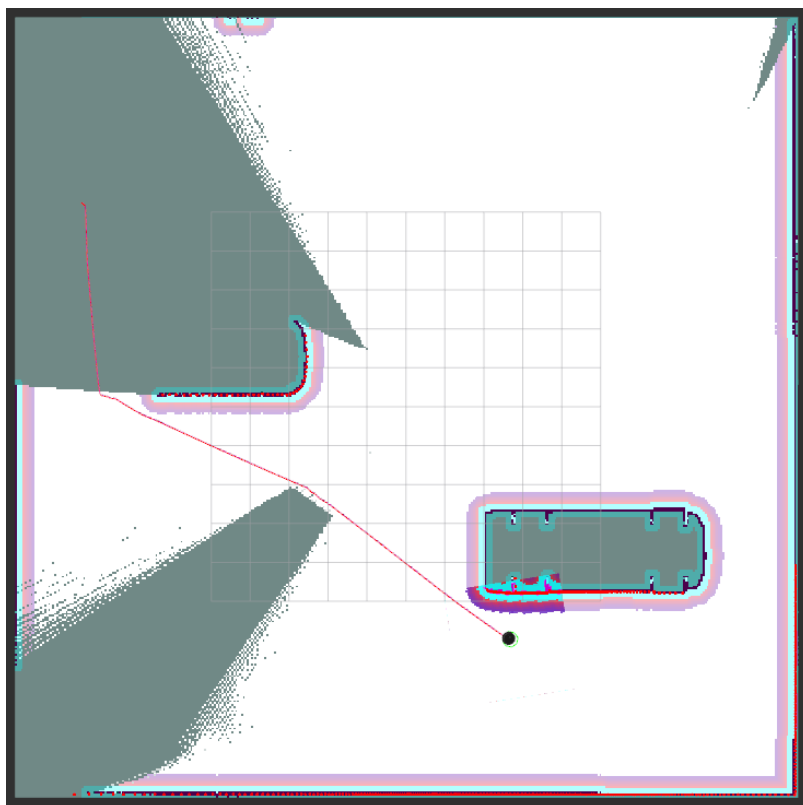


Rysunek 2.10: Schemat blokowy ilustrujący działanie algorytmów SLAM działających na podstawie fuzji sensorycznej [26].

2.6 Mapa zajętości

Dla nawigacji mobilnej w środowisku ROS 2 podstawową reprezentacją przestrzeni roboczej robota jest mapa zajętości (occupancy grid map), budowana najczęściej za pomocą SLAM. Jest to dwuwymiarowa siatka komórek o zadanej rozdzielczości, w której każdej komórce przypisana jest wartość liczbową określającą prawdopodobieństwo, że dany fragment przestrzeni jest zajęty przez przeszkodę. W wizualizacji mapy (np. w narzędziu RViz) poszczególne stany komórek odwzorowane są kolorami: obszary wolne przedstawiane są jako białe, przeszkody jako czarne, a przestrzeń nieznana jako szara (rysunek 2.11). Taka reprezentacja umożliwia algorytmom nawigacyjnym planowanie ścieżki prowadzącej wyłącznie przez obszary wolne, unikanie przeszkód oznaczonych jako zajęte oraz podejmowanie decyzji o eksploracji obszarów dotychczas nieznanymi. Mapa zajętości jest aktualizowana na bieżąco na podstawie danych z czujników (np. LiDAR-IMU), co pozwala na odzwierciedlenie zmian zachodzących w otoczeniu robota w czasie rzeczywistym [35].

⁸Dryf (drift) - zjawisko narastania błędów pomiarowych w czasie, prowadzące do coraz większych rozbieżności między pozycją estymowaną a rzeczywistą.



Rysunek 2.11: Mapa zajętości budowana w czasie rzeczywistym przez algorytm SLAM

2.7 Nav2

W niniejszym projekcie robot ma poruszać się autonomicznie - samodzielnie wyznaczać trasę do zadanego celu i zrealizować ją z uwzględnieniem przeszkód stałych, jak i dynamicznych, bez ingerencji operatora. Funkcjonalność tę zapewnia stos (stack) nawigacyjny Nav2 w ROS 2, który integruje lokalizację, planowanie ruchu oraz sterowanie napędami w zamkniętej pętli, umożliwiając wykonywanie zadań typu „jedź do wskazanego punktu” w mapie lub w trakcie jej tworzenia używając SLAM.

Nav2 stanowi najnowsze oprogramowanie do nawigacji robotów mobilnych w środowisku ROS 2, rozwijające możliwości oferowane wcześniej przez Nav1 w systemie ROS 1. Jedną z najistotniejszych zmian w Nav2 jest zastosowanie architektury drzewa zachowań (Behavior Tree, BT) do realizacji zadań nawigacyjnych, w miejsce automatu stanów skończonych wykorzystywanego w Nav1. Zastosowanie drzewa zachowań zwiększa modularność, elastyczność oraz przejrzystość działania systemu. Ułatwia to rozszerzanie funkcjonalności poprzez dodawanie nowych zachowań, takich jak procedury odzyskiwania, oczekiwanie na lokalizację czy reagowanie na przeszkody dynamiczne [36, 37]. Architekturę stosu Nav2 ilustruje rysunek 2.12.

Architektura BT w Nav2 umożliwia zarówno sekwencyjne, jak i równoległe wykonywanie zaawansowanej logiki nawigacyjnej. Pozwala to na implementację złożonych zachowań bez konieczności modyfikacji rdzenia stosu nawigacyjnego, co stanowi istotną przewagę względem Nav1 [36, 37].

2.7.1 Realizacja trajektorii przez Nav2

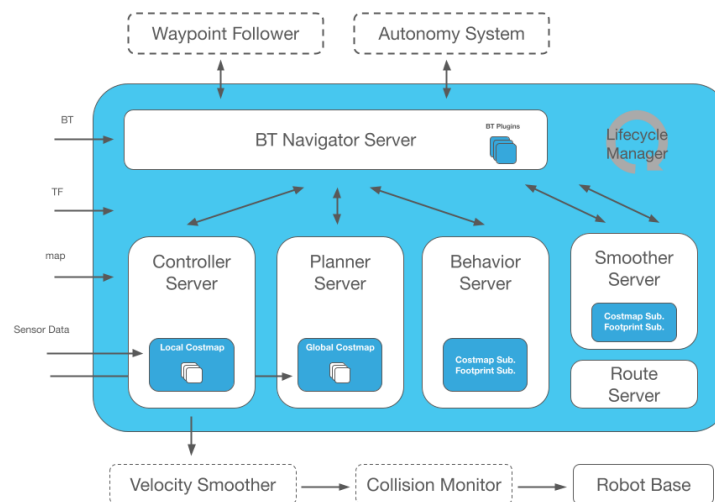
Moduł Nav2 realizuje nawigację robota na podstawie zestawu map kosztów (costmaps), które stanowią dwuwymiarową siatkową reprezentację środowiska - każdej komórce przypisana jest wartość kosztu odzwierciedlająca stopień przejeźdźności danego obszaru. Istotne znaczenie mają dwie mapy: globalna mapa kosztów (global costmap) oraz lokalna mapa kosztów (local costmap). Mapa globalna obejmuje całe znane środowisko i wykorzystywana jest przez planer globalny do wyznaczenia ogólnej trasy od pozycji robota do zadanego punktu nawigacyjnego. Mapa lokalna obejmuje najbliższe otoczenie robota i jest aktualizowana na bieżąco na podstawie danych z czujników - odpowiada za nią planer lokalny, którego zadaniem jest realizacja trasy z uwzględnieniem przeszkód wykrywanych w czasie rzeczywistym. Obie mapy budowane są z warstw (layers), które łączą różne

źródła informacji: warstwa statyczna (static layer) importuje mapę zajętości wygenerowaną wcześniej przez algorytm SLAM, warstwa przeszkód (obstacle layer) integruje bieżące odczyty z czujników, a warstwa inflacji (inflation layer) rozszerza obszary zajęte o strefę buforową, na rysunku 2.11 oznaczona kolorem błękitny i fioletowy. Za jej pomocą planer utrzymuje bezpieczny odstęp od przeszkód. Współdziałanie tych warstw pozwala robotowi wyznaczać trasę do zadanego celu z jednoczesnym omijaniem zarówno znanych, jak i dynamicznie wykrywanych przeszkód [36].

2.7.2 Główne komponenty Nav2

Do podstawowych elementów systemu Nav2 należą [36, 37]:

- **Map Server** – odpowiada za wczytanie mapy statycznej, na podstawie której tworzona jest dwuwymiarowa mapa kosztów (costmap). Costmap stanowi dynamiczną, warstwową reprezentację siatkową otoczenia robota, integrującą dane z mapy statycznej oraz bieżących pomiarów czujników. Wykorzystywana jest przez moduły planowania i sterowania do omijania przeszkód oraz wyznaczania bezpiecznej trajektorii.
- **Lifecycle Manager** – nadzoruje proces uruchamiania, konfiguracji, aktywacji oraz wyłączania wszystkich węzłów i serwerów Nav2. Zapewnia prawidłową sekwencję przejść pomiędzy stanami, co zwiększa stabilność oraz przewidywalność działania systemu.
- **Planner Server** – odpowiada za wyznaczenie globalnej ścieżki pomiędzy aktualną pozycją robota a celem. Na podstawie pozycji początkowej i docelowej analizuje mapę oraz wyznacza bezpieczną i optymalną trajektorię, unikając obszarów zatłoczonych lub niedostępnych.
- **Controller Server** – generuje komendy prędkości na podstawie punktów trasy dostarczonych przez Planner Server. Monitoruje postęp robota i dynamicznie koryguje tor ruchu, zapewniając bezpieczne i precyzyjne poruszanie się nawet w środowisku dynamicznym.
- **Smoother** – moduł wygładzający globalną trajektorię. Jego zadaniem jest eliminacja ostrych zakrętów i nieregularności w planowanej ścieżce, tak aby ruch robota był bardziej płynny i wykonalny.
- **Recovery Server** – implementuje strategie odzyskiwania w sytuacjach niepowodzenia nawigacji. W przypadku utknięcia robota lub braku możliwości wyznaczenia trasy wykonywane są zdefiniowane procedury, takie jak cofanie, obrót czy czyszczenie mapy kosztów.



Rysunek 2.12: Schemat ilustrujący architekturę Nav2 [36]

2.7.3 Drzewo zachowań w Nav2

Moduł Behavior Tree Navigator w Nav2 odpowiada za aktywację oraz zarządzanie wykonaniem serwerów planowania, sterowania oraz odzyskiwania. System bazuje na modelu drzewa zachowań, który steruje realizacją zadań nawigacyjnych. Struktura drzewa definiowana jest w pliku XML, co umożliwia łatwą konfigurację logiki nawigacyjnej poprzez zastosowanie węzłów sterujących przepływem oraz węzłów warunkowych. W przypadku niepowodzenia planowania uruchamiane są odpowiednie procedury odzyskiwania. Metoda ta działa na zasadzie analizy statusu węzłów akcji oraz elementów sterujących drzewa zachowań, dzięki czemu system reaguje adekwatnie na awarię konkretnego komponentu [36, 37].

2.8 Wizja komputerowa

W kontekście niniejszej pracy wizja komputerowa została wykorzystana jako narzędzie percepcji robota mobilnego podczas inspekcji parkingu. Jej rolą jest pozyskanie z obrazu informacji, które są istotne dla zadań, które robot ma wykonać: ocenić poprawność zaparkowanego pojazdu, odczytać treść tablicy rejestracyjnej.

Człowiek w naturalny sposób postrzega trójwymiarową strukturę otaczającego świata. Wystarczy spojrzeć na wazon z kwiatami stojący na stole, aby uświadomić sobie, jak wyraźne jest wrażenie głębi i przestrzenności. Na podstawie subtelnych zmian oświetlenia oraz cieni padających na powierzchnię płatków możliwe jest rozpoznanie ich kształtu, struktury oraz przezroczystości. Wizja komputerowa jest dziedziną informatyki, której celem jest umożliwienie komputerom analizy oraz interpretacji obrazów w sposób zbliżony do ludzkiego. Na podstawie jednego lub wielu obrazów system próbuje odtworzyć informacje o strukturze przestrzennej sceny, kształcie obiektów, ich położeniu, oświetleniu czy kolorze [38].

Zagadnienie to ma charakter problemu odwrotnego, ponieważ na podstawie dwuwymiarowego obrazu, stanowiącego rzut rzeczywistej sceny, należy wnioskować o jej trójwymiarowej budowie. W praktyce oznacza to konieczność stosowania modeli matematycznych opisujących propagację światła oraz metod probabilistycznych pozwalających radzić sobie z niejednoznacznością danych. Mimo że człowiek wykonuje ten proces w sposób naturalny i intuicyjny, opracowanie algorytmów osiągających porównywalny poziom interpretacji pozostaje złożonym wyzwaniem [38].

2.8.1 Klasyczna detekcja obiektów

Aby uzyskać pełniejsze zrozumienie obrazu, nie wystarczy jedynie przypisać mu określoną klasę, lecz konieczne jest również precyzyjne określenie położenia oraz kategorii obiektów znajdujących się w scenie. Zadanie to określane jest jako detekcja obiektów (object detection) [39]. Obejmuje ono szereg podzadań, takich jak detekcja twarzy [40], detekcja pieszych [41] czy analiza struktury szkieletowej [42, 43].

Detekcja obiektów stanowi jedno z podstawowych zagadnień wizji komputerowej, dostarczając informacji niezbędnych do semantycznej interpretacji obrazów i sekwencji wideo. Znajduje zastosowanie m.in. w klasyfikacji obrazów [44, 45], analizie zachowań człowieka [46], rozpoznawaniu twarzy [47] oraz systemach autonomicznej jazdy [48, 49, 43].

Problem detekcji obiektów polega na jednoczesnym rozwiązaniu dwóch zadań: lokalizacji obiektów w obrazie (object localization) oraz ich klasyfikacji (object classification). Klasyczne podejście do detekcji można podzielić na trzy główne etapy: ekstrakcja obszarów zainteresowania, ekstrakcję cech oraz klasyfikację [43].

Ekstrakcja obszarów zainteresowania (ROI)

Ponieważ obiekty mogą występować w różnych miejscach obrazu oraz charakteryzować się zróżnicowanymi rozmiarami i proporcjami, stosowano strategię przesuwne okna (sliding window) w wielu skalach. Metoda ta umożliwia przeszukanie całego obrazu, jednak generuje bardzo dużą liczbę okien kandydackich, co prowadzi do wysokiej złożoności obliczeniowej oraz powstawania liczących, redundantnych regionów [43].

Ekstrakcja cech

W celu rozróżniania obiektów konieczne jest wyznaczenie reprezentacji opisującej ich właściwości wizualne. Do najczęściej stosowanych deskryptorów należą SIFT [50], HOG [51] oraz cechy Haar [52]. Pomimo skuteczności tych metod, ręczne projektowanie deskryptorów cech nie pozwalało na pełne odwzorowanie zróżnicowania obiektów w zmiennych warunkach oświetlenia i tła [43].

Klasyfikacja

Na ostatnim etapie wykorzystywano klasyfikatory, takie jak maszyna wektorów nośnych (SVM) [53], AdaBoost [54] czy model deformowalnych części (DPM) [39]. Szczególnie model DPM umożliwiał reprezentację obiektu jako zbioru powiązanych części z uwzględnieniem kosztu deformacji, co pozwalało na skuteczniejsze radzenie sobie ze zmianami pozy i kształtu [43].

2.8.2 Nowoczesne metody detekcji

Pomimo znaczącego postępu, klasyczne metody detekcji miały istotne ograniczenia wynikające z dużej zmienności punktu widzenia, zasłonięć oraz warunków oświetleniowych. Rozwój głębokich sieci neuronowych oraz metod uczenia reprezentacji znacząco wpłynął na dalszy rozwój detekcji obiektów, prowadząc do powstania architektur takich jak R-CNN (rysunek 2.13) [55], Faster R-CNN [56] oraz YOLO [57, 43]. Natomiast najnowsze podejścia bazujące na sieciach neuronowych wykorzystujących architekturę transformerów, takich jak DETR.

Konwolucyjne sieci neuronowe - CNN

Konwolucyjne sieci neuronowe (Convolutional Neural Networks, CNN) stanowią szczególną klasę sztucznych sieci neuronowych, które wykazują wysoką skuteczność w analizie danych o strukturze przestrzennej, takich jak obrazy, sygnały mowy czy sygnały audio. Ich architektura została zaprojektowana w taki sposób, aby umożliwić automatyczne wydobywanie cech z danych wejściowych bez konieczności ręcznego projektowania deskryptorów [58].

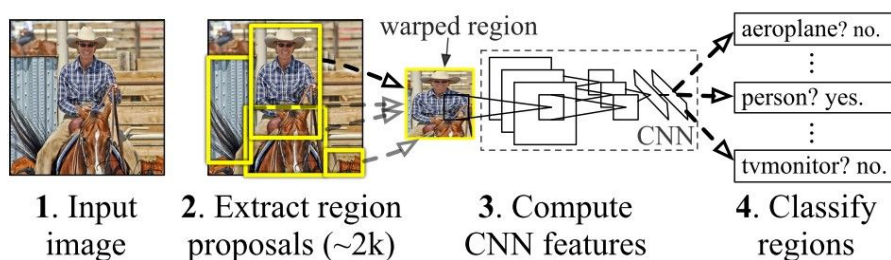
Typowa architektura CNN składa się z trzech podstawowych rodzajów warstw:

- warstw konwolucyjnych,
- warstw łączących (pooling),
- warstw w pełni połączonych (fully connected).

Warstwa konwolucyjna stanowi podstawowy element sieci CNN. Jej zadaniem jest wykrywanie lokalnych wzorców w obrazie poprzez zastosowanie filtrów (jąder konwolucyjnych), które przesuwane są po całej przestrzeni wejściowej. W początkowych warstwach sieć uczy się identyfikować proste cechy, takie jak krawędzie, kontrasty czy zmiany kolorów. W kolejnych, głębszych warstwach rozpoznawane są coraz bardziej złożone struktury, na przykład fragmenty obiektów, a ostatecznie całe obiekty. Wraz ze wzrostem liczby warstw rośnie poziom abstrakcji reprezentacji [58].

Warstwa łącząca (pooling) odpowiada za redukcję wymiarowości danych, zmniejszając liczbę parametrów oraz złożoność obliczeniową modelu. Operacja ta polega na agregacji wartości w określonym obszarze wejścia, na przykład poprzez wybór wartości maksymalnej (max pooling) lub średniej (average pooling). W przeciwieństwie do warstwy konwolucyjnej, warstwa pooling nie zawiera uczonych wag, lecz pełni funkcję kompresji i uogólnienia reprezentacji cech [58].

Warstwa w pełni połączona (fully connected, FC) stanowi zazwyczaj końcowy etap sieci. W tej warstwie każdy neuron jest połączony z wszystkimi neuronami warstwy poprzedniej. Jej zadaniem jest przeprowadzenie klasyfikacji na podstawie cech wyekstrahowanych przez wcześniejsze warstwy konwolucyjne i pooling. W warstwach konwolucyjnych i pooling często stosowana jest funkcja aktywacji ReLU, natomiast w końcowej warstwie klasyfikacyjnej powszechnie wykorzystuje się funkcję softmax, która generuje rozkład prawdopodobieństwa przynależności do poszczególnych klas [58].

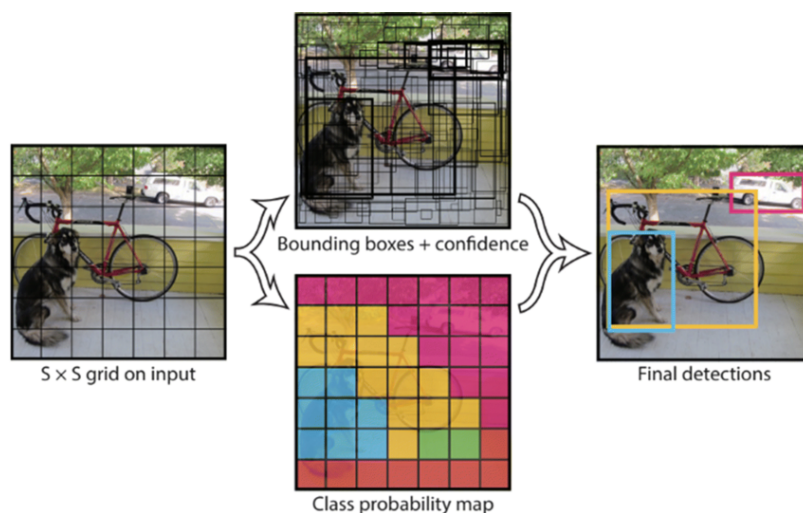


Rysunek 2.13: Algorytm detekcji R-CNN [59]

YOLO

W klasycznych architekturach detekcji obiektów bazujących na sieciach konwolucyjnych stosuje się algorytm generowania propozycji regionów (region proposal). W pierwszym etapie wyznaczane są potencjalne obszary zawierające obiekty w postaci ramek ograniczających (bounding boxes), a następnie dla każdego z nich wykonywana jest klasyfikacja. Po etapie klasyfikacji przeprowadza się dodatkowe przetwarzanie końcowe (post-processing), którego celem jest doprecyzowanie położenia ramek, eliminacja zduplikowanych detekcji oraz ponowne przeskalowanie ocen wiarygodności z uwzględnieniem kontekstu sceny [55]. Tak złożony, wieloetapowy potok przetwarzania charakteryzuje się znaczną złożonością obliczeniową oraz trudnością optymalizacji, ponieważ każdy z jego komponentów musi być trenowany oddzielnie [57].

Architektura YOLOv1 (You Only Look Once) stanowi uproszczone i zintegrowane podejście do detekcji obiektów. W przeciwieństwie do metod wieloetapowych, wykorzystuje pojedynczą sieć konwolucyjną, która jednocześnie przewiduje współrzędne wielu ramek ograniczających (bounding boxes) oraz prawdopodobieństwa przynależności do poszczególnych klas. Model YOLO trenowany jest na pełnych obrazach i bezpośrednio optymalizuje jakość detekcji jako problem regresji. Dzięki temu nie wymaga złożonego, wieloetapowego potoku przetwarzania. W fazie testowej wystarczy jedno przejście obrazu przez sieć neuronową, aby uzyskać wynik detekcji [57] Algorytm detekcji został pokazany na rysunku 2.14.



Rysunek 2.14: Algorytm detekcji YOLO [57]

Jedną z głównych zalet YOLO jest bardzo wysoka szybkość działania. Ponieważ detekcja traktowana jest jako zadanie regresji, nie ma potrzeby generowania propozycji regionów ani oddzielnego klasyfikowania każdego obszaru. Upraszcza to architekturę oraz umożliwia pracę w czasie rzeczywistym. Kolejną istotną cechą jest globalne rozumowanie o obrazie. W przeciwieństwie do metod bazujących na przesuwym oknie lub propozycjach regionów, YOLO analizuje cały obraz jednocześnie zarówno w fazie treningu, jak i wnioskowania. Pozwala to na uwzględnienie kontekstu sceny, co przekłada się na lepsze modelowanie zależności pomiędzy obiektami. Dodatkowo YOLO uczy się reprezentacji o dobrej zdolności generalizacji. W badaniach wykazano, że model trenowany na obrazach naturalnych zachowuje wysoką skuteczność także w przypadku innych domen, na przykład grafiki czy ilustracji, przewyższając klasyczne metody takie jak DPM czy R-CNN. Dzięki temu architektura ta wykazuje większą odporność na zmiany środowiska oraz nieoczekiwane dane wejściowe [57].

Ewolucja rodziny YOLO. YOLO nie jest pojedynczym modelem, lecz rodziną kolejnych wersji rozwijanych na przestrzeni lat. Począwszy od oryginalnego YOLOv1, w kolejnych iteracjach wprowadzano zmiany architektoniczne i treningowe ukierunkowane na poprawę dokładności, szybkości oraz stabilności działania w zastosowaniach czasu rzeczywistego. Ewolucja obejmuje m.in. modyfikacje sposobu predykcji ramek, usprawnienia wieloskalowe oraz stopniowe odchodzenie od klasycznego, wieloetapowego post-processingu w stronę bardziej zintegrowanych rozwiązań [60].

DETR

Nowszą klasę metod detekcji obiektów stanowią modele bazujące na architekturze transformerów, z których jednym z najbardziej rozpoznawalnych jest DETR (DEtection TRansformer). W przeciwieństwie do klasycznych detektorów CNN, które działają na podstawie etapów generowania propozycji regionów, doboru ramek bazowych oraz procedurach post-processingu, DETR formułuje detekcję jako problem predykcji zbioru obiektów i realizuje ją w sposób end-to-end. Model wykorzystuje algorytm uwagi oraz zbiór tzw. zapytań obiektowych (object queries), a dopasowanie predykcji do obiektów referencyjnych realizowane jest poprzez dwudzielne dopasowanie (Hungarian matching) [61].

2.9 OCR

OCR (Optical Character Recognition) jest technologią umożliwiającą automatyczne przekształcanie obrazu zawierającego tekst w postać możliwą do odczytu maszynowego. Proces ten polega na ekstrakcji informacji tekstowej z zeskanowanych dokumentów, obrazów z kamer lub plików PDF zawierających wyłącznie grafikę. Systemy OCR wykorzystują zarówno elementy sprzętowe, jak i programowe. Urządzenia takie jak skanery optyczne odpowiadają za pozyskanie obrazu dokumentu, natomiast właściwe rozpoznanie znaków realizowane jest przez algorytmy przetwarzania obrazu i modele uczenia maszynowego. Współczesne systemy OCR coraz częściej wykorzystują metody sztucznej inteligencji, w tym techniki inteligentnego rozpoznawania znaków ICR (Intelligent Character Recognition), umożliwiające identyfikację różnych języków, krojów pisma oraz tekstu odręcznego. Technologia ta znajduje szerokie zastosowanie w digitalizacji dokumentów, sektorze zdrowotnym, bankowym, przemysłowym czy automatycznym odczytywaniu tablic rejestracyjnych [62, 63].

W systemach automatycznego rozpoznawania tablic rejestracyjnych (ANPR/ALPR) metody OCR stanowią typowe rozwiązanie dla etapu właściwego odczytu znaków (rysunek 2.15).



Rysunek 2.15: Odczyt numeru tablicy rejestracyjnej za pomocą OCR [64]

2.9.1 Zasada działania systemów OCR

U podstaw działania technologii OCR leży próba odwzorowania sposobu, w jaki człowiek odczytuje tekst. System analizuje strukturę obrazu, identyfikuje znaki, a następnie przekształca je w tekst zakodowany w postaci cyfrowej. Proces ten, mimo pozornej prostoty, obejmuje kilka istotnych etapów [65].

Wstępne przetwarzanie obrazu (pre-processing). Na tym etapie obraz wejściowy przygotowywany jest do dalszej analizy. Stosuje się techniki takie jak redukcja szumu, korekcja nachylenia (deskew), poprawa kontrastu czy binaryzacja, której celem jest oddzielenie tekstu od tła. Odpowiednie przygotowanie obrazu ma istotny wpływ na skuteczność dalszego rozpoznawania.

Segmentacja znaków. Po przetworzeniu wstępnym obraz dzielony jest na pojedyncze znaki lub symbole graficzne. Segmentacja stanowi istotny etap procesu, ponieważ umożliwia analizę każdego znaku oddzielnie, co zwiększa dokładność rozpoznania.

Ekstrakcja cech. W kolejnym kroku wyznaczane są charakterystyczne cechy każdego znaku, takie jak linie, łuki, pętle czy proporcje geometryczne. Cechy te stanowią podstawę do różnicowania poszczególnych symboli.

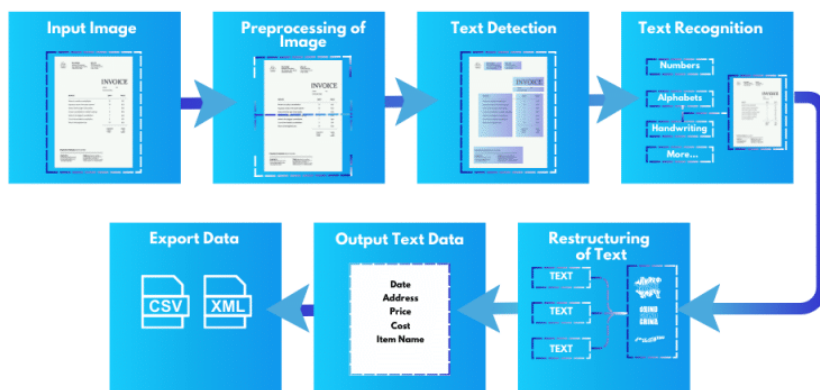
Rozpoznawanie znaków. Wyodrębnione cechy porównywane są z bazą wzorców lub przetwarzane przez modele uczenia maszynowego, w tym sieci neuronowe. Algorytm określa najbardziej prawdopodobne dopasowanie dla każdego znaku.

Post-processing. Ostatni etap obejmuje korektę rozpoznanego tekstu. Wykorzystuje się słowniki, analizę kontekstową, reguły gramatyczne lub dodatkowe modele statystyczne w celu eliminacji błędów, poprawy odstępów oraz zwiększenia ogólnej dokładności systemu.

Uogólniony proces działania OCR ilustruje rysunek 2.16.

How OCR Works

AlgoDocs



Rysunek 2.16: Uogólniony algorytm działania OCR [65]

2.9.2 Konkretnie metody OCR

W praktyce dostępnych jest wiele narzędzi OCR, różniących się podejściem oraz wymaganiami obliczeniowymi. Przykładowe, często wykorzystywane rozwiązania to:

- **Tesseract OCR** - otwartoźródłowy silnik OCR rozwijany pierwotnie w HP, a później udostępniony jako projekt open source. Działa w klasycznym potoku: na podstawie analizy składowych połączonych (konturów) grupuje elementy obrazu w grupy, organizuje je w linie i słowa, a następnie wykonuje rozpoznawanie znaków. W artykule opisano podejście dwupassowe z adaptacją, gdzie poprawnie rozpoznane fragmenty z pierwszego przejścia służą do dostosowania klasyfikatora, a drugie przejście służy do poprawy trudniejszych przypadków. Istotnym elementem są też algorytmy segmentacji znaków i radzenia sobie ze zlepianiami (np. „chopping” i dobór najlepszego podziału), co pomaga przy gorszej jakości obrazu [66].
- **EasyOCR** - działa w typowym, dwustopniowym potoku - detekcja tekstu i rozpoznanie. Najpierw moduł detekcji (CRAFT, Character Region Awareness for Text Detection) lokalizuje na obrazie obszary zawierające tekst, wyznaczając ich położenie (ramki lub wielokąty) dopiero te wycinki są przekazywane dalej. W drugim kroku wykonywane jest właściwe OCR, zwykle modelem typu CRNN: część konwolucyjna wydobywa cechy z obrazu, część sekwencyjna modeluje kolejność znaków, a dekodery zamienia predykcje na końcowy ciąg znaków. Wynikiem są rozpoznane napisy wraz z położeniem i oceną pewności, co jest wygodne np. w ANPR, gdzie interesuje nas zarówno tekst tablicy, jak i jej lokalizacja w kadrze [67].

Rozdział 3

Realizacja projektu i weryfikacja działania

3.1 Przygotowanie środowiska

W projektach realizowanych w środowisku Linux krytycznym problemem jest zgodność wersji oprogramowania i bibliotek. System operacyjny, środowisko języka Python, ROS, Gazebo posiadają własne cykle wydań i zależności, które nie zawsze są ze sobą spójne. W konsekwencji aktualizacja jednego komponentu może wymusić zmianę wersji innych bibliotek lub doprowadzić do błędów uruchomieniowych. Z tego względu istotne dla projektu było ustalenie i utrzymanie spójnego zestawu wersji oprogramowania.

3.1.1 Założenia środowiskowe i dobór narzędzi

System operacyjny użyty w projekcie to Ubuntu 24.04 LTS (Long term support). System ten jest obecnym standardem w robotyce akademickiej, dostosowanie się do standardu zapewnia łatwe odtworzenie projektu na innym sprzęcie, szeroko dostępną dokumentację, dużą dostępność narzędzi, oprogramowania oraz wieloletnie wsparcie. Ułatwia to pracę, co jest szczególnie ważne, zważając na fakt braku doświadczenia w systemach operacyjnych z serii Linux.

Mimo coraz większego wsparcia Windowsa dla ROS 2, to Ubuntu pozostaje domyślnym i najczęściej rekomendowanym OS dla ROS 2. Wersja nazwana Jazzy Jalisco wydana w 2024 roku posiada długoterminowe wsparcie oraz jest obecnie uważana za Tier 1, ponadto była wielokrotnie testowana i rozwijana na Ubuntu 24.04. Część symulacyjna projektu jest tworzona przy użyciu Gazebo Harmonic, które jest rekomendowaną linią Gazebo dla ROS 2 Jazzy na Ubuntu 24.04, zapewniającą spójność wersji i kompatybilność zależności.

Użyte wersje narzędzi i bibliotek:

- **System operacyjny:** Ubuntu 24.04 LTS (Noble)
- **Framework robotyczny:** ROS 2 Jazzy (Jazzy Jalisco)
- **Symulator:** Gazebo Harmonic v8.10
- **Platforma robota w symulacji:** TurtleBot4
- **OCR:** EasyOCR
- **Biblioteka numeryczna:** NumPy 1.26.4 (zastosowano obniżenie wersji z 2.4.2 z uwagi na kompatybilność z mostkiem ROS oraz EasyOCR)
- **Przetwarzanie obrazu:** OpenCV 4.8.1 (ostatnia wersja zgodna z NumPy 1.26.x)

3.1.2 Warstwa sprzętowa

Warstwę sprzętową projektu stanowi laptop Lenovo Legion 5 15IAH7 wyposażony w procesor Intel Core i7-12700H (architektura Alder Lake-H, 14 rdzeni / 20 wątków), przeznaczoną kartę graficzną NVIDIA GeForce RTX 3050 Ti Laptop GPU oraz 16 GB pamięci RAM DDR5. Taka konfiguracja jest przeznaczona do zastosowań o podwyższonych wymaganiach obliczeniowych, co ma znaczenie z punktu widzenia tego projektu - szczególnie ze względu na jednoczesne uruchamianie wielu procesów ROS 2 (obciążenie CPU) oraz renderowanie świata symulacyjnego w Gazebo i wizualizacji w RViz (obciążenie GPU).

Laptop posiada konfigurację grafiki hybrydowej, czyli zintegrowany układ graficzny Intel (iGPU) oraz przeznaczoną kartę graficzną NVIDIA (dGPU). W przypadku wybranego systemu

operacyjnego konfiguracja hybrydowa powodowała problem z prawidłowym przypisaniem karty graficznej, który objawiał się nieoczekiwanym zamykaniem Gazebo podczas renderowania symulacji. Z tego powodu zdecydowano się wyłączyć jednostkę zintegrowaną, używając wyłącznie przeznaczonej karty graficznej o wyższej wydajności. Konsekwencją takiego podejścia może być większy pobór mocy i krótszy czas pracy na baterii, jednak priorytetem była stabilność symulacji oraz bezawaryjność warstwy sprzętowej.

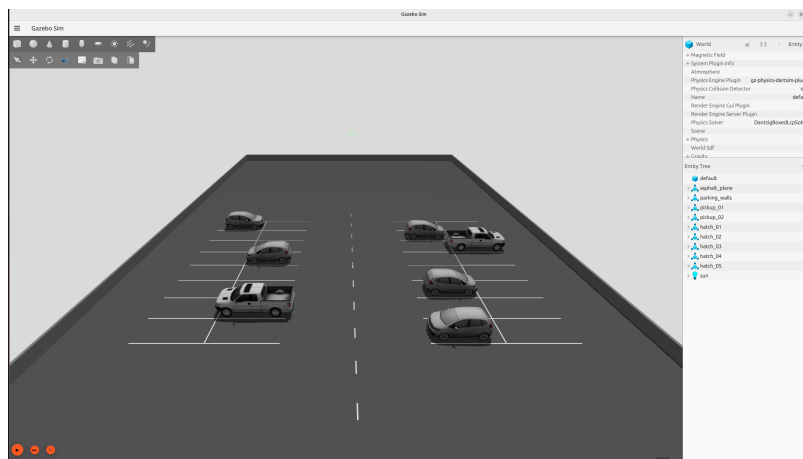
3.2 Budowa świata symulacji w Gazebo

3.2.1 Prototyp parkingu

W pierwszej kolejności przystąpiono do zbudowania świata, w którym będzie odbywać się symulacja. Celem było odwzorowanie zamkniętego asfaltowego parkingu, w którym miejsca parkingowe wytyczone przez białe linie rozmieszczone są pod kątem prostym do osi jezdni, z dostępem do światła słonecznego realizowanego w Gazebo za pomocą obiektu `light`. Płaszczyzna asfaltu posiada warstwę wizualną (`visual`) jak i kolizyjną (`collision`), która zapewnia poprawne zachowanie dynamiczne obiektu w symulacji - obiekty nie przenikają się między sobą oraz ich potencjalne kolizje są zgodne z zasadami dynamiki. Linie parkingowe - ciągle oraz linie wyznaczające oś jezdni posiadają jedynie warstwę wizualną, aby zminimalizować możliwe błędy przy renderowaniu świata oraz poruszaniu się robota po tym świecie. Miejsce parkingowe ma wymiary zgodne z typowymi dla miejsc parkingowych prostokątów, czyli 2,5 metra szerokości oraz 5 metrów długości.

Modele pojazdów (Pickup, Hatchback)

Jedną z zalet użycia Gazebo jest duża liczba wcześniej stworzonych i gotowych do wykorzystania modeli jak i światów. Ze strony Gazebo Fuel, którą można znaleźć pod linkiem: <https://app.gazebosim.org/dashboard> pobrano modele samochodów *Pickup* oraz *Hatchback* dostarczone przez OpenRobotics, modele te są doskonale odwzorowane względem ich prawdziwych odpowiedników, zarówno pod względem wymiarów czy siatki wizualnej. Co ważne, pobierając pliki modelu uzyskuje się dostęp do ich kodu, który można modyfikować na własne potrzeby. Modele te zostały rozmieszczone w miejscach parkingowych, aby przetestować wstępne założenia dotyczące wymiarów miejsc. Świat uzupełniony o pojazdy można zobaczyć na rysunku 3.1.



Rysunek 3.1: Parking wraz z modelami samochodów

3.2.2 Stworzenie wersji parkingu pod testy

Pierwsza wersja parkingu widoczna na rysunku 3.1, mimo że spełniała swoje założenia, nie była najbardziej efektywna, problematyczne okazało się przemieszczanie stosunkowo wolnego robota jak i mapowanie przez algorytm SLAM tak dużej przestrzeni, dlatego zdecydowano się na stworzenie pomniejszonej wersji, przedstawionej na rysunku 3.2, która będzie używana do testowania wykonania kolejnych zadań przez Robota.



Rysunek 3.2: Wersja parkingu wykorzystywana podczas badań

3.3 Integracja robota TurtleBot4 ze światem własnym

Stworzenie własnego modelu robota wraz z plikami służącymi do uruchomienia symulacji, obsługi węzłów, napędów czy sensorów wymaga wielu godzin pracy, szczegółowej wiedzy i dobrego zaznajomienia z konstrukcją robotów mobilnych oraz środowiska, w którym tworzony jest robot. Dla pewnych specyficznych wymagań projektowych niemożliwe byłoby użycie gotowej platformy robota, jednak dla naszego projektu można znaleźć wiele takich platform, które spełniają wymagania projektu.

Po zaznajomieniu się ze specyfikacją, opisem oraz opiniami dotyczącymi platform robotów mobilnych typu open-source dostępnych w internecie wybór padł na TurtleBot4 w wersji Standard, którego producentem jest firma Clearpath. Robot ten jest typową mobilną platformą badawczo-edukacyjną zaprojektowaną z myślą o pracy w środowisku ROS 2. Obecnie jest to jedna z najczęściej wykorzystywanych platform do badań nad autonomiczną nawigacją, SLAM oraz percepcją wizyjną [16]. Wyróżnia się obszerną dokumentacją, dużą liczbą plug-inów oraz zaawansowanym symulatorem.

3.3.1 Modyfikacje Robotu

Wraz z całym workspace¹ TB4 uzyskuje się dostęp do zestawu plików URDF/XACRO opisujących robota. Wstępne testy wykazały, że domyślna konfiguracja (rysunek 3.3a) opisana w sekcji 2.3.1 nie jest funkcjonalna w naszym środowisku. Finalna wersja (rysunek 3.3b) została osiągnięta poprzez zmiany:

1. **Pozycji LiDAR:** czujnik oryginalnie zamontowany pozwalał na wykrycie jedynie kół samochodów, co jest niewystarczające dla założeń projektu. Testy wykazały, że wymagane jest podniesienie o około 30 cm aby LiDAR był w stanie bezproblemowo wykryć cały obrys modelu samochodu, w tym celu czujnik został przeniesiony na płytę montażową.
2. **Pozycji kamery:** pierwotnie kamera była w pozycji czołowej, natomiast takie ustawienie nie było najbardziej efektywne do inspekcji prawidłowo zaparkowanych samochodów - wymagałoby obrotu robota w stronę miejsca parkingowego. Stwierdzono, że prostym, a jednocześnie skutecznym podejściem do detekcji nieprawidłowo zaparkowanych pojazdów jest analiza ciągłości linii parkingowej, tj. weryfikacja, czy linia została przerwana przez pojazd. Szczegółowe omówienie zalet i wad takiego rozwiązania przedstawiono w sekcji 3.6.1. W celu zapewnienia odpowiedniej widoczności linii parkingowych zmodyfikowano położenie i orientację kamery: podniesiono ją o 70 cm względem pierwotnej konfiguracji oraz obrócono o $+90^\circ$ zgodnie z prawoskrętnym układem współrzędnych. Realizacją mechaniczną tych zmian jest przymocowanie kamery do drugiej płyty montażowej, którą osadzono na słupkach.
3. **Rozkładu wagi:** Gazebo bardzo dobrze odwzorowuje dynamikę modeli, po powyższych zmianach środek ciężkości robota znacząco przesunął się do góry, co spowodowało, że przy przyspieszaniu przód robota unosił się. Zachowanie to jest zdecydowanie niepożądane, robot traci stabilizację, a gwałtowne ruchy zakłócają działanie i mogą uszkodzić wrażliwą na wstrząsy

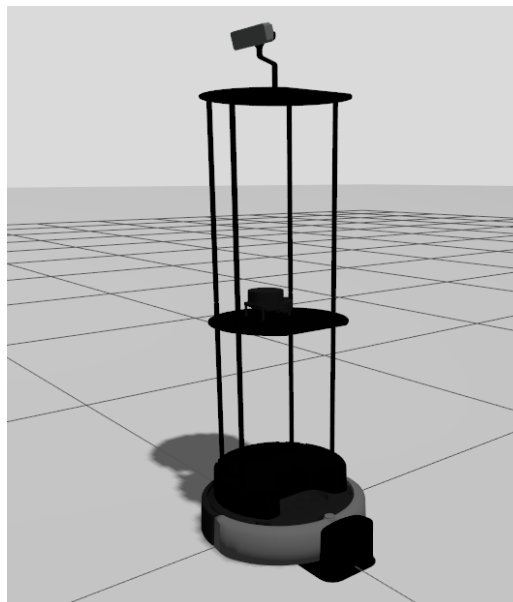
¹Workspace (środowisko robocze) – uporządkowany katalog projektu, w którym znajdują się pliki źródłowe, konfiguracje oraz artefakty budowania i uruchamiania. Dla ROS 2 najczęściej jest to struktura (np. `ros2_ws`) zawierająca m.in. katalog `src` z pakietami, a po zbudowaniu także `build`, `install` i `log`.

elektronikę znajdującą się w sensorach. W oryginalnej konstrukcji TB4 uwzględniono blok obciążający, który jednak dla nowej konfiguracji okazał się niewystarczający. Zmieniono masę bloku z 0.06 kg na 1.0 kg, zmiana ta w realnym modelu może się odbyć poprzez zmianę materiału z którego wykonany jest blok obciążający lub jego geometrii.

4. **Parametryzacja sensorów:** poprzez pliki URDF/XACRO można zmodyfikować parametry działania sensorów w symulacji, dla lepszego działania LiDAR: lekko zwiększono promień minimalny, odległość od jakiej wykrywana jest przeszkoda, aby nie traktował słupków na których utrzymywane są płyty montażowe jako przeszkody oraz zwiększono rozdzielczość kamery dla uzyskania lepszego obrazu potrzebnego do odczytania tablic rejestracyjnych oraz detekcji linii parkingowych.



(a) Domyślna [16]



(b) Zmodyfikowana

Rysunek 3.3: Porównanie konfiguracji TB4.

3.4 Modelowanie tablic rejestracyjnych w Gazebo

Tablice rejestracyjne stanowią istotny element identyfikacyjny pojazdu. Posiadają one ustandaryzowaną strukturę, wymiary oraz założenia, które jednak różnią się w zależności od kraju lub regionu. Nawet w obrębie jednego kraju można znaleźć wyjątki tak jak chociażby w Polsce tablicami niestandardowymi, a dopuszczonymi do użytku są tablice:

- **Dla samochodów powyżej 25 lat:** określanych mianem klasyków, żółte tło zamiast białego.
- **Personalizowane:** mające inną strukturę znaków niż standardowe, jednak wciąż znormalizowane.
- **Dwurzędowe:** potocznie nazywane amerykańskimi.
- **Historyczne:** rejestracje sprzed 2000 roku, posiadają czarne tło i białe znaki.
- **Dyplomatyczne:** niebieskie tło zastępuje białe.
- **Dla samochodów elektrycznych:** z zielonym tłem zamiast białego.

W kontekście odczytywania tablic rejestracyjnych poprzez metody ANPR im lepiej zdefiniujemy strukturę jaką może mieć tablica tym pewniejszy wynik otrzymujemy. Mamy zatem wybór, czy chcemy mieć sztywno zdefiniowaną strukturę, w konsekwencji mniej uniwersalny algorytm, czy na odwrót.

3.4.1 Założenia dotyczące tablic rejestracyjnych

Założenia dotyczące tablic rejestracyjnych występujących w projekcie, na których testowane były w późniejszym etapie metody OCR to:

- **Wymiary** - 520 x 110 mm zgodne z obowiązującymi normami stosowanymi w Polsce oraz w większości krajów Unii Europejskiej.
- **Jednorzędowość.**
- **Od pięcioletniego do dziewięcioletniego ciągu alfanumerycznego.**

- **Stały format graficzny:** zbliżony do tablic europejskich (biały obszar, niebieski pasek wraz ze skrótem kraju, czarne znaki).

Konfiguracja ta nie jest najbardziej uniwersalną, szczególnie jeżeli weźmiemy pod uwagę rozwiązania z całego świata, jednak pokrywa zdecydowaną większość przypadków tablic rejestracyjnych, które można zaobserwować na co dzień w Polsce.

3.4.2 Model tablicy w Gazebo

Dla modułu rozpoznawania tablic rejestracyjnych (ANPR) konieczne było uzyskanie realistycznych tablic widocznych w symulacji Gazebo. Model powstaje jako zestaw: tekstura, którą będzie obraz (plik PNG) oraz definicja ciała modelu w formacie SDF, dzięki temu można połączyć tablicę wraz z modelem samochodu poprzez zlinkowanie ścieżki do pliku SDF tablicy rejestracyjnej w pliku SDF pojazdu oraz określenie pozycji tablicy. Rozwiązanie to zapewnia, że gdy już raz dobrze określimy pozycję modelu tablicy względem danego modelu pojazdu to staje się integralną częścią pojazdu i zmienia pozycję wraz z nim. Jeżeli chcemy zmienić tablicę, wystarczy, że zmienimy ścieżkę do pliku.

Generacja tekstury tablicy (PNG)

Pierwszym krokiem jest utworzenie tekstury, która odwzorowuje wygląd tablicy. Tekstura generowana jest za pomocą biblioteki Pillow/PIL dostępnej w języku Python, która umożliwia generowanie powtarzalnych grafik o stałym układzie i proporcjach. Jest to istotne, ponieważ niejednolite tekstury - różniące się na przykład rozmiarem czcionki, odstępami między znakami czy proporcjami tła - mogłyby negatywnie wpływać na skuteczność odczytu tablic przez system ANPR, wprowadzając dodatkową zmienność niezwiązaną z badanym problemem. Rysunek 3.4 przedstawia przykładowy plik PNG użyty jako tekstura ciała tablicy.



Rysunek 3.4: Plik PNG tablicy rejestracyjnej użyty jako tekstura

Budowa modelu 3D w SDF

Kolejny krok polega na utworzeniu ciała modelu SDF, który wykorzystuje wygenerowaną grafikę jako teksturę w definicji materiału:

- **Warstwa wizualna (visual):** prostopadłościan (box) o wymiarach odpowiadających tablicy rzeczywistej, z przypisaną teksturą jako mapą albedo.
- **Warstwa kolizji (collision):** identyczna bryła jak dla warstwy wizualnej, w naszym przypadku mało istotna
- **Parametry fizyczne (inertial):** masa oraz macierz bezwładności w bardzo uproszczonej wersji, dla naszego przypadku nieistotne.

Skrypt tworzący tablice

Aby zautomatyzować generowanie tablic rejestracyjnych, stworzono skrypt w języku Python, który łączy wyżej wymienione procesy. Podaje się żądany zestaw znaków, którym ma być opisana tablica, np. „WWA 2121”, a skrypt automatycznie tworzy odpowiadającą mu grafikę oraz kompletną definicję modelu do środowiska Gazebo. Każdy zestaw plików jest jednoznacznie identyfikowany na podstawie unikalnej nazwy modelu wynikającej z treści tablicy.

3.4.3 Dodanie tablic do pojazdów

Pozycje tablic rejestracyjnych na poszczególnych modelach pojazdów dobrano iteracyjnie, dostosowując wartości współrzędnych do geometrii każdego z nich. Uzupełnione modele pojazdów prezentuje rysunek 3.5. Tablice zostały nieznacznie odsunięte od pojazdu, aby wyeliminować ryzyko wzajemnego przenikania się modeli, które mogłoby zakłócać odczyt kamery. Pozycję ustawiono jako statyczną, dzięki czemu zachowują przypisaną pozycję przez cały czas trwania symulacji.



(a) Hatchback



(b) Pickup

Rysunek 3.5: Modele pojazdów z dodanymi modelami tablic rejestracyjnych.

3.5 Odczyt tablic rejestracyjnych przez robota

Systemy automatycznego odczytu tablic rejestracyjnych (ANPR) są obecnie powszechnie stosowane w wielu rozwiązaniach. Można je spotkać m.in. w kontroli wjazdu na parkingi i tereny zamknięte, automatycznym poborze opłat (tolling), monitoringu ruchu oraz egzekwowaniu przepisów, w tym w coraz częściej spotykanych systemach odcinkowego pomiaru prędkości.

Przeanalizowano i porównano ogólnodostępne podejścia, które można wykorzystać do realizacji odczytu tablic rejestracyjnych (ANPR) w robocie. Wzięto pod uwagę zarówno klasyczne silniki OCR, takie jak TesseractOCR [68], jak i rozwiązania działające na podstawie uczenia głębokiego, np. EasyOCR w połączeniu z detekcją YOLOv11 [69]. Sprawdzono również możliwość wykorzystania agentów LLM² poprzez API³ jako elementu pomocniczego [70]. Z uwagi na to, że symulacja ma być niejako prototypem fizycznego robota, uznano że rozwiązania wymagające stałego połączenia z internetem mogą być problematyczne (opóźnienia, zakłócenia transmisji, brak dostępu do sieci), dlatego zostały odrzucone w pierwszej kolejności.

3.5.1 Wybrane rozwiązanie ANPR

Ostatecznie zdecydowano się na wykorzystanie gotowego rozwiązania ANPR dostarczonego przez Ultralytics [69], bazującego na architekturze YOLOv11 (krótki opis przedstawiono poniżej), w połączeniu z biblioteką EasyOCR, opisaną w sekcji 2.9.2. Podejście to dzieli detekcję na dwa główne kroki: w pierwszym kroku model YOLOv11, wytrenowany na zbiorze danych zawierającym tablice rejestracyjne, lokalizuje w obrazie regiony odpowiadające tablicom (bounding boxes), a następnie biblioteka EasyOCR, odczytuje tekst wyłącznie z wyciętych fragmentów obrazu (ROI). Dzięki temu EasyOCR nie przetwarza całego kadru, lecz jedynie obszary wskazane przez detektor, co zwiększa zarówno szybkość, jak i precyzję rozpoznawania.

Zdecydowano się na to rozwiązanie ze względu na:

- brak konieczności tworzenia własnej treningowej bazy danych,
- baza danych została stworzona na podstawie rzeczywistych obiektów - jeżeli system poradzi sobie w symulacji, powinien osiągnąć porównywalną skuteczność w warunkach rzeczywistych,
- wykorzystanie architektury YOLOv11, co pozwala na precyzyjną lokalizację tablic w obrazie niezależnie od ich położenia, skali i orientacji,
- prosty interfejs API biblioteki EasyOCR niewymagający szczególnej znajomości sieci neuronowych,
- brak konieczności zaawansowanego preprocessingu obrazu - model YOLOv11 dostarcza gotowe ROI, a EasyOCR przetwarza je bezpośrednio,
- możliwość pracy w trybie offline bez konieczności łączności z siecią,
- dobra dokładność rozpoznawania dla tekstów w różnych warunkach oświetleniowych,
- oba projekty są aktywnie rozwijane, a modele są regularnie aktualizowane,
- licencje open-source pozwalające na swobodne wykorzystanie w projektach badawczych.

²LLM Large Language Model - wielkoskalowy model językowy szkolony na ogromnych zbiorach danych, zdolny do rozumienia i generowania tekstu w języku naturalnym oraz wykonywania złożonych zadań logicznych np. ChatGPT.

³Application Programming Interface - zestaw funkcji opisujących jak korzystać z danej biblioteki.

Główną zaletą zastosowanego połączenia jest rozdzielenie detekcji tablicy od odczytu tekstu na dwa niezależne etapy. Model YOLOv11 odpowiada wyłącznie za znalezienie obszarów, o wysokim prawdopodobieństwie że znajduje się w nich tablica rejestracyjna. Następnie EasyOCR, dzięki wykorzystaniu głębokich sieci neuronowych (CRAFT do detekcji znaków oraz CRNN do ich rozpoznawania), odczytuje tekst z dostarczonych fragmentów obrazu. Dzięki takiemu podziałowi zadań unika się przetwarzania całego obrazu przez OCR, co w podejściach bez detektora oraz odpowiednich zabezpieczeń odczytu prowadziło do fałszywych detekcji, między innymi nazw modeli pojazdów, oznaczeń parkingowych lub innych tekstów znajdujących się na obrazie.

YOLOv11

YOLOv11 (często opisywany w dokumentacji Ultralytics jako YOLO11) to generacja modeli do detekcji w czasie rzeczywistym wydana przez Ultralytics we wrześniu 2024, nastawiona na lepszy kompromis dokładność–szybkość oraz łatwe wdrażanie w praktycznych pipeline’ach⁴ (gotowe tryby treningu i inferencji). W porównaniu do wcześniejszych modeli tej linii kładzie nacisk na usprawnienia ekstrakcji cech i efektywności obliczeniowej [71]. Szczegółowy opis dostępny na stronie <https://docs.ultralytics.com/models/yolo11/>

3.5.2 Wstępne badania skuteczności wybranego rozwiązania

Przed integracją rozwiązania z robotem przeprowadzono wstępne testy mające na celu weryfikację skuteczności połączenia YOLOv11 i EasyOCR w warunkach symulacji Gazebo.

Oryginalne rozwiązanie Ultralytics zostało zaprojektowane do pracy z plikami wideo. Na potrzeby testów wstępnych zmodyfikowano skrypt, dostosowując go do przetwarzania pojedynczych zdjęć w formacie PNG. Modyfikacja polegała na zastąpieniu pętli odczytującej klatki wideo jednorazowym załadowaniem obrazu z pliku, przy zachowaniu pozostałej logiki detekcji i rozpoznawania.

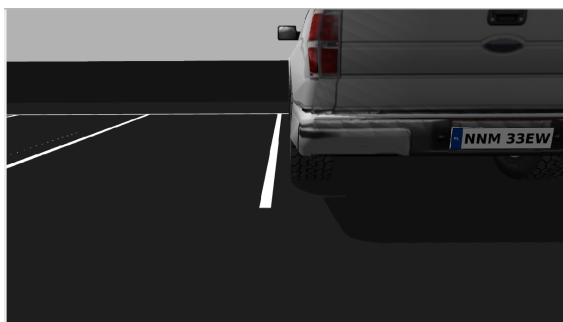
Przygotowanie bazy zdjęć

W celu stworzenia bazy testowej wykonano serię 20 zdjęć tablicy rejestracyjnej w różnych konfiguracjach umiejscowienia jej względem kamery.

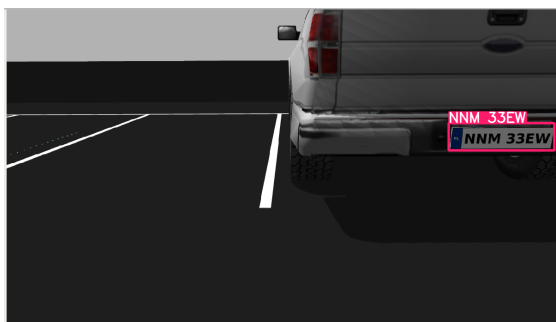
1. Robot został przemieszczony do różnych lokalizacji w otoczeniu, z których widoczna była tablica rejestracyjna pojazdów.
2. Dla każdej pozycji robota wykonano zrzut ekranu wizualizacji obrazu z kamery w narzędziu RViz, subskrybując topic `/oakd/rgb/preview/image_raw`.
3. Zdjęcia obejmowały różne konfiguracje przestrzenne, w tym:
 - różne odległości od tablicy,
 - zmienne kąty patrzenia względem płaszczyzny tablicy.

Cztery przykładowe zdjęcia z bazy testowej zostały przedstawione na rysunku 3.6.

⁴Pipeline - ciąg połączonych elementów przetwarzających dane w sposób sekwencyjny.



(a) próbka t1



(b) przetworzona próbka t1



(c) próbka t9



(d) przetworzona próbka t9



(e) próbka t12



(f) przetworzona próbka t12



(g) próbka t18



(h) przetworzona próbka t18

Rysunek 3.6: Cztery przykładowe próbki zdjęć wraz z wynikiem użyte w testach.

Wyniki testów

Wyniki testów przedstawiono w tabeli 3.1. Algorytm wykazał bardzo wysoką skuteczność, osiągając 100% poprawnych rozpoznań w przygotowanej bazie testowej.

Tabela 3.1: Wyniki testów rozpoznawania tablic rejestracyjnych

Metryka	Wartość
Liczba zdjęć testowych	20
Poprawnie rozpoznane tablice	20
Błędnie rozpoznane tablice	0
Tablice nierozpoznane	0
Skuteczność	100%

Analiza i ograniczenia

Pomimo osiągnięcia 100% skuteczności, należy zaznaczyć, że baza zdjęć testowych zawierała głównie próbki bez przypadków krytycznych:

- tablice w pełni widoczne, bez przesłonięć,
- dobre warunki oświetleniowe (brak silnych prześwieśleń),
- stosunkowo niewielkie kąty patrzenia względem płaszczyzny tablicy,
- tablica w stanie technicznym umożliwiającym łatwy odczyt (bez zabrudzeń, uszkodzeń).
- brak napisów umieszczonych w regionach mogących przypominać tablicę rejestracyjną

W rzeczywistych warunkach eksploatacji system może napotkać na przypadki trudniejsze. Mimo tych ograniczeń, uzyskane wyniki potwierdziły, że wybrana metoda jest wyborem spełniającym wymagania projektu i można o nią oprzeć system rozpoznawania tablic rejestracyjnych w robocie.

3.5.3 Węzeł ROS 2 do odczytu tablic rejestracyjnych

W celu integracji rozwiązania ANPR z robotem zaimplementowano węzeł ROS 2 o nazwie `anpr_node`. Węzeł został napisany w języku Python i realizuje detekcję tablic, dzieląc proces na trzy etapy, pierwszym z nich jest detekcja regionów tablic przez YOLOv11, następnie odczyt tekstu z wyciętych fragmentów obrazu (EasyOCR) oraz końcowo walidacja formatu numeru rejestracyjnego. Proces rozpoznawania tablicy rejestracyjnej z obrazu opisuje schemat blokowy przedstawiony na rysunku 3.7, ilustrujący kolejne kroki przetwarzania danych od odebrania obrazu z kamery aż do publikacji wyniku.

Pierwszym krokiem jest subskrypcja obrazu z kamery poprzez topic `/oakd/rgb/preview/image_raw`. Aby nie obciążać procesora, nie analizowana jest każda klatka, lecz co N -ta (parametr `process_every_n_frames`, domyślnie $N = 15$). Wybrana klatka jest następnie konwertowana z formatu wiadomości ROS 2 do macierzy OpenCV przy użyciu biblioteki `cv_bridge`, taka konwersja jest wymagana, z uwagi, że zarówno YOLOv11, jak i EasyOCR operują na macierzach NumPy, a nie na surowym formacie `sensor_msgs/msg/Image`.

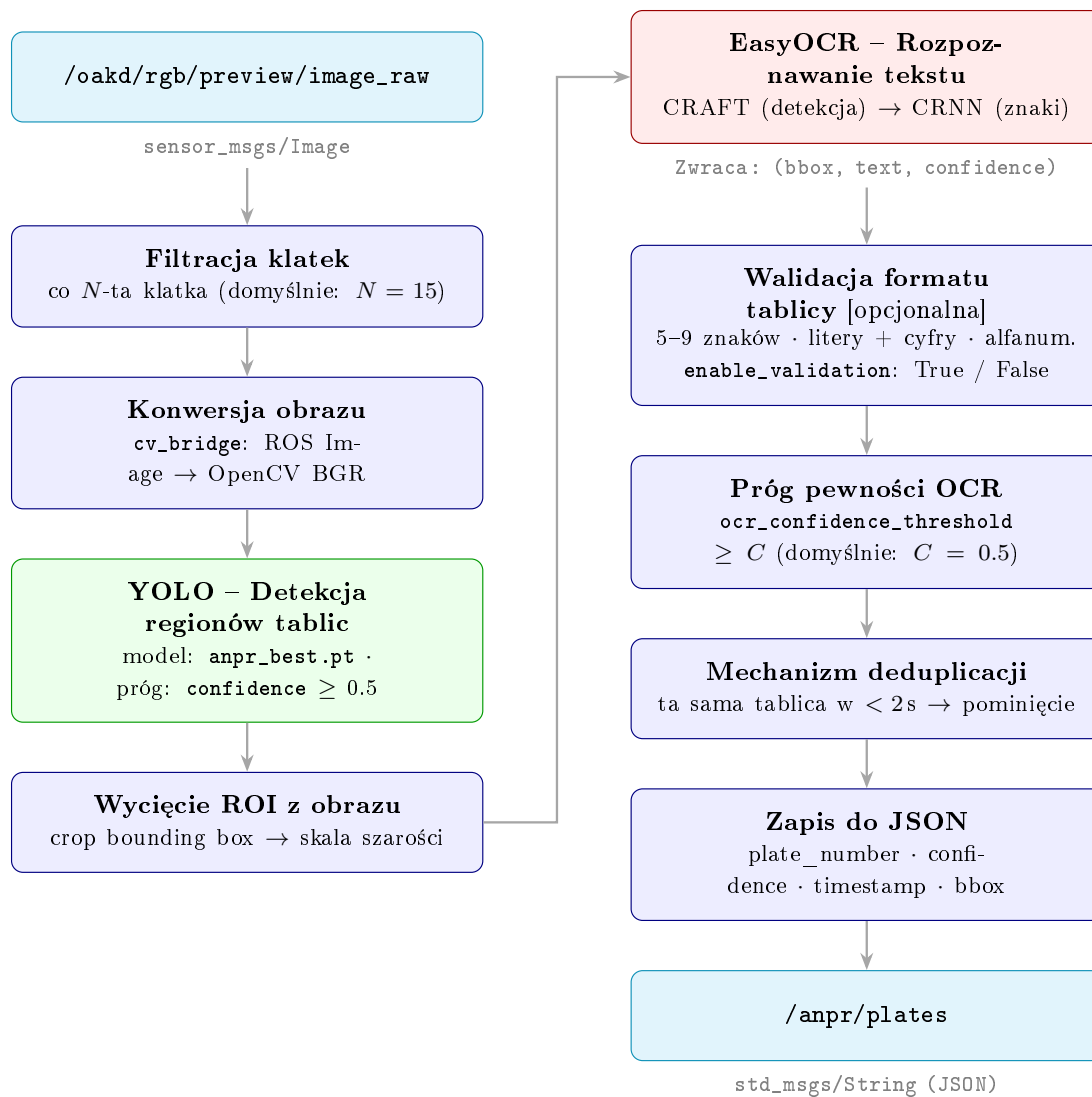
W pierwszym etapie przetwarzania obraz trafia do modelu YOLOv11, który lokalizuje w kadrze regiony odpowiadające tablicom rejestracyjnym. Model zwraca listę ramek otaczających (*bounding boxes*) wraz z poziomem pewności detekcji. Akceptowane są wyłącznie detekcje, których pewność przekracza próg Y (parametr `confidence`, domyślnie $Y = 0,5$). Dzięki temu do dalszego przetwarzania trafiają jedynie fragmenty obrazu z wysokim prawdopodobieństwem zawierania tablicy, a EasyOCR nie musi przeszukiwać całego kadru.

W drugim etapie, dla każdego wykrytego regionu, wycinany jest ROI na podstawie współrzędnych bounding box zwróconych przez YOLOv11. Wycięty fragment jest konwertowany do skali szarości, a następnie przetwarzany przez bibliotekę EasyOCR, która rozpoznaje tekst obecny w ROI, zwracając odczytane znaki oraz poziom pewności odczytu (*OCR confidence*).

Trzeci etap stanowi opcjonalna walidacja formatu, realizowana za pomocą metody `is_valid_plate_format()`, którą można włączać i wyłączać (domyślnie włączona). Metoda ta sprawdza, czy odczytany tekst spełnia kryteria numeru rejestracyjnego założone w sekcji 3.4.1 tj. zawiera od 5 do 9 znaków alfanumerycznych, w tym zarówno litery, jak i cyfry. Metoda ta pozwala zabezpieczyć się przed wykrywaniem znaków umieszczonych w miejscach przypominających tablice, ale niebędące nimi.

Aby tekst został uznany za poprawnie odczytany, musi dodatkowo przekroczyć próg pewności OCR C (parametr `ocr_confidence_threshold`, domyślnie $C = 0,5$). Zaimplementowano również mechanizm deduplikacji, który zapobiega wielokrotnemu publikowaniu tej samej tablicy w krótkim czasie. Detekcja spełniająca wszystkie powyżej wymienione kryteria jest zapisywana w formacie JSON⁵ (zawierającym pola: `plate_number`, `confidence`, `timestamp`, `bbox`) i publikowana do topicu `/anpr/plates` jako wiadomość `std_msgs/String`.

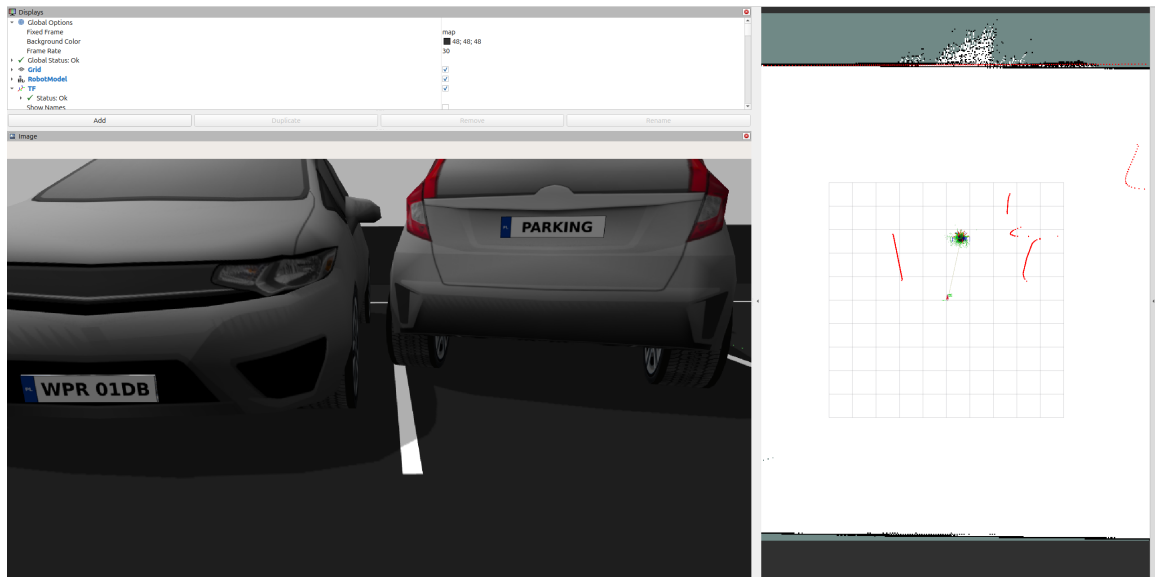
⁵JavaScript Object Notation (JSON) - tekstowy format wymiany danych, popularny w API i konfiguracji



Rysunek 3.7: Schemat blokowy węzła odpowiadającego za detekcję i odczyt tablic rejestracyjnych.

Walidacja metody *is_valid_plate_format()*

Przeprowadzona walidacja potwierdziła krytyczną rolę metody *is_valid_plate_format()* w procesie rozpoznawania tablic rejestracyjnych jej pominięcie skutkuje znacznym wzrostem błędnych detekcji co przedstawia rysunek 3.8.



(a) Konfiguracja świata, użyta do walidacji.

```
^Cbalek@balek-Legion-5-15IAH7:~$ ros2 topic echo anpr/plates
data: '{"plates": []}'
---
data: '{"plates": [{"plate_number": "WPR 01DB", "confidence": 0.9873, "timestamp": 5511.891, "bbox": {"x": 399, "y": 306, "width": 181, ...}}]}'
---
data: '{"plates": []}'
---
data: '{"plates": []}'
---
data: '{"plates": []}'
---
data: '{"plates": [{"plate_number": "WPR 01DB", "confidence": 0.9873, "timestamp": 5513.871, "bbox": {"x": 399, "y": 306, "width": 181, ...}}]}'
```

(b) Publikowane wiadomości przez węzeł *anpr* node z włączoną metodą *is_valid_plate_format()*.

```
---
data: '{"plates": [{"plate_number": "1B", "confidence": 0.9915, "timestamp": 7108.893, "bbox": {"x": 0, "y": 331, "width": 92, "height": ...}}]}'
---
data: '{"plates": [{"plate_number": "WPR 01DB", "confidence": 0.8299, "timestamp": 7109.388, "bbox": {"x": 32, "y": 317, "width": 217, ...}}]}'
---
data: '{"plates": [{"plate_number": "WPR 01DB", "confidence": 0.6524, "timestamp": 7109.883, "bbox": {"x": 43, "y": 317, "width": 214, ...}}]}'
---
data: '{"plates": [{"plate_number": "DB", "confidence": 0.9997, "timestamp": 7110.378, "bbox": {"x": 0, "y": 336, "width": 74, "height": ...}}]}'
---
data: '{"plates": [{"plate_number": "PARKING", "confidence": 0.9994, "timestamp": 7110.873, "bbox": {"x": 478, "y": 102, "width": 130, ...}}]}'
```

(c) Publikowane wiadomości przez węzeł *anpr* node z wyłączoną metodą *is_valid_plate_format()*.

Rysunek 3.8: Scenariusz dla walidacji metody *is_valid_plate_format()*

3.5.4 Metodyka doboru parametrów algorytmu

Celem badań było wyznaczenie optymalnych wartości progów pewności detekcji YOLOv11 (*confidence*) oraz pewności odczytu OCR (*ocr_confidence_threshold*) w warunkach zbliżonych do docelowego trybu pracy robota, tj. podczas jego ruchu wzdłuż zaparkowanych pojazdów.

Dobór progu pewności OCR.

W celu wyznaczenia minimalnego progu pewności OCR, przy którym system zwraca poprawne odczyty, przeprowadzono przejazd robota wzdłuż czterech zaparkowanych pojazdów, widocznych na rysunku 3.9 z progiem ustawionym na wartość minimalną ($C = 0,1$) oraz wyłączoną deduplikacją. Analizując logi systemu, zidentyfikowano najniższą wartość pewności OCR, dla której odczyt tablicy był w pełni zgodny z wartością referencyjną (ground truth, GT). Wartość ta wyniosła $C \approx 0,5$, co przyjęto jako dolną granicę akceptowalnego progu w dalszych krokach.



Rysunek 3.9: Scenariusz w Gazebo dla doboru progu pewności OCR.

Dobór progu pewności detekcji YOLOv11.

W celu zbadania wpływu progu pewności modelu YOLOv11 na jakość detekcji tablic rejestracyjnych przygotowano środowisko testowe składające się z pojedynczego pojazdu umieszczonego na parkingu w czterech konfiguracjach przestrzennych widocznych na rysunku 3.10. Dla każdej konfiguracji robot poruszał się po stałej trasie, od osi jednej linii parkingowej - punkt (0 ; 0) do osi drugiej - punkt (-2,5 ; 0).



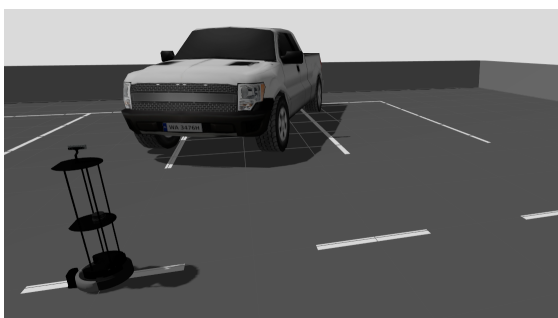
(a) Konfiguracja nr. 1



(b) Konfiguracja nr. 2



(c) Konfiguracja nr. 3



(d) Konfiguracja nr. 4

Rysunek 3.10: Konfiguracje przestrzenne pojazdu w testach ANPR

Dla każdej z czterech konfiguracji wykonano trzy przejazdy, zmieniając wyłącznie próg pewności YOLOv11, przyjmując wartości: $Y \in \{0,8; 0,5; 0,3\}$. Łącznie przeprowadzono 12 przejazdów, dla których analizowano pierwsze 32 klatki. Pozostałe parametry były stałe dla wszystkich prób:

- `ocr_confidence_threshold = 0,5`,
- `process_every_n_frames = 10`,

- `enable_validation = true`,
- `dedup_interval_sec = 0,0` (deduplikacja wyłączona).

Zastosowane metryki. Dla każdej detekcji obliczono:

- **Odległość edycyjną Levenshteina** (d_L) - minimalną liczbę operacji edycji wymaganą do przekształcenia odczytanego tekstu w wartość referencyjną.
- **Character Error Rate** (CER) - stosunek odległości Levenshteina do długości ciągu referencyjnego:

$$CER = \frac{d_L(\hat{y}, y)}{|y|} \quad (3.1)$$

gdzie $\hat{y}$ oznacza odczytany tekst, a y - wartość referencyjną.

- **TP** (True Positive) - klatka, na której system poprawnie wykrył i odczytał tablicę ($d_L = 0$),
- **FP** (False Positive) - klatka, na której system wykrył tablicę, ale odczyt był błędny ($d_L > 0$),
- **FN** (False Negative) - klatka, na której tablica była widoczna, ale system jej nie wykrył,
- **Precision** = $\frac{TP}{TP+FP}$ - udział poprawnych odczytów wśród wszystkich detekcji,
- **Recall** = $\frac{TP}{TP+FN}$ - udział poprawnie wykrytych tablic wśród wszystkich klatek zawierających tablicę.

Z uwagi na to, że każda przetwarzana klatka zawierała widoczną tablicę rejestracyjną, metryka TN (True Negative) nie występuje w przedstawionym zestawieniu.

Wyniki

Zbiorcze wyniki 12 przejazdów przedstawiono w tabeli 3.2.

Tabela 3.2: Wyniki ewaluacji modułu ANPR dla 12 przejazdów

Konfig.	Y	Detekcje	TP	FP	FN	Precision	Recall	Śr. CER
K1	0,8	7	7	0	25	100,0%	21,9%	0,0%
K1	0,5	20	20	0	12	100,0%	62,5%	0,0%
K1	0,3	32	25	7	0	78,1%	100,0%	3,1%
K2	0,8	6	6	0	26	100,0%	18,8%	0,0%
K2	0,5	18	18	0	14	100,0%	56,3%	0,0%
K2	0,3	28	23	5	4	82,1%	85,2%	2,4%
K3	0,8	5	5	0	27	100,0%	15,6%	0,0%
K3	0,5	16	15	1	16	93,8%	48,4%	0,7%
K3	0,3	30	22	8	2	73,3%	91,7%	3,7%
K4	0,8	4	4	0	28	100,0%	12,5%	0,0%
K4	0,5	14	13	1	18	92,9%	41,9%	0,8%
K4	0,3	26	19	7	6	73,1%	76,0%	3,9%

Szczegółowe zestawienie wszystkich błędnych odczytów wraz z obliczonymi metrykami przedstawiono w tabeli 3.3.

Tabela 3.3: Szczegółowe zestawienie błędnych odczytów tablic rejestracyjnych

Konfig.	Y	Odczyt	GT	d_L	CER
K1	0,3	WA 3476	WA 3476H	1	11,1%
K1	0,3	WA 3476	WA 3476H	1	11,1%
K1	0,3	WA 3476	WA 3476H	1	11,1%
K1	0,3	WA 3476	WA 3476H	1	11,1%
K1	0,3	WA 3476	WA 3476H	1	11,1%
K1	0,3	WA 3476_	WA 3476H	1	11,1%
K1	0,3	WA 34761_	WA 3476H	2	22,2%
K2	0,3	PZ 5599	PZ 55999	1	11,1%
K2	0,3	PZ 5599	PZ 55999	1	11,1%
K2	0,3	PZ 55999_	PZ 55999	1	11,1%
K2	0,3	PZ 559	PZ 55999	2	22,2%
K2	0,3	PZ 5S999	PZ 55999	1	11,1%
K3	0,5	WA 3476	WA 3476H	1	11,1%
K3	0,3	WA 3476	WA 3476H	1	11,1%
K3	0,3	WA 3476	WA 3476H	1	11,1%
K3	0,3	WA 3476	WA 3476H	1	11,1%
K3	0,3	WA 347	WA 3476H	2	22,2%
K3	0,3	WA 34761	WA 3476H	1	11,1%
K3	0,3	WA 3476_	WA 3476H	1	11,1%
K3	0,3	WA 3A76H	WA 3476H	1	11,1%
K4	0,5	A 3476H	WA 3476H	1	11,1%
K4	0,3	A 3476	WA 3476H	2	22,2%
K4	0,3	WA 3476	WA 3476H	1	11,1%
K4	0,3	WA 3476	WA 3476H	1	11,1%
K4	0,3	WA 3A76H	WA 3476H	1	11,1%
K4	0,3	WA_3476H	WA 3476H	1	11,1%
K4	0,3	WA 3476	WA 3476H	1	11,1%

Analiza wyników

Na podstawie przeprowadzonych 12 przejazdów można sformułować następujące obserwacje:

Wpływ progu YOLOv11 na liczbę detekcji. Próg $Y = 0,8$ skutkuje najmniejszą liczbą detekcji, co oznacza, że model odrzuca wiele klatek, na których tablica jest widoczna, ale wykryta z niższą pewnością. Obniżenie progu do $Y = 0,5$ zwiększa liczbę detekcji, zachowując jednocześnie wysoką dokładność. Dalsze obniżenie do $Y = 0,3$ daje najwięcej detekcji, ale kosztem pojawienia się błędnych odczytów.

Charakterystyka błędów OCR. Analiza tabeli 3.3 wskazuje, że dominującym typem błędu jest utrata ostatniego znaku tablicy (np. odczyt WA 3476 zamiast WA 3476H). Błędy te występują na klatkach zarejestrowanych przy najmniej korzystnych kątach obserwacji, gdzie skrajne znaki tablicy są częściowo niewidoczne lub zniekształcone perspektywicznie.

Wpływ konfiguracji przestrzennej. Konfiguracje K3 i K4, w których pojazd był ustawiony bardziej pod kątem w stosunku do kamery wykazały niższą dokładność niż konfiguracje K1 i K2, - przy progu $Y = 0,5$ pojawiły się pojedyncze błędy ($d_L = 1$), a przy $Y = 0,3$ liczba błędnych detekcji wzrosła. Potwierdza to, że kąt obserwacji ma istotny wpływ na jakość rozpoznawania.

Wybór optymalnych parametrów.

Próg $Y = 0,5$ stanowi optymalny kompromis pomiędzy liczbą detekcji a dokładnością odczytu. Zapewnia wystarczająco dużą liczbę detekcji, aby system mógł poprawnie zidentyfikować tablicę, przy jednocześnie wysokim accuracy na poziomie 92,9-100% oraz średniego CER nieprzekraczającego 0,8%. W połączeniu z progiem OCR $C = 0,5$, wyznaczonym we wcześniejszym eksperymencie, wartości te przyjęto jako parametry domyślne modułu ANPR w dalszych testach integracyjnych.

3.6 Algorytm detekcji nieprawidłowego parkowania

Celem opracowanego algorytmu było jednoznaczne określenie, czy pojazd zaparkowany na danym miejscu narusza jego granice. W analizowanym środowisku przyjęto klasyczny układ parkingu z wyraźnie oznaczonymi liniami poziomymi koloru białego, wyznaczającymi granice miejsc postojowych.

3.6.1 Założenia algorytmu i wybór podejścia

Na etapie projektowym rozważono trzy różne podejścia do rozwiązania problemu, działające na podstawie:

1. **Sieci neuronowych** - podejście zakładało budowę zbioru danych zawierającego obrazy poprawnie oraz niepoprawnie zaparkowanych pojazdów, a następnie wykorzystanie sieci neuronowej do klasyfikacji stanu parkowania. Algorytm działałby na podstawie sieci neuronowej typu YOLO, umożliwiłoby to automatyczną ocenę zaparkowanego pojazdu na podstawie treningu sieci. Główną trudnością takiego podejścia jest przygotowanie odpowiednio liczego i zróżnicowanego zbioru danych treningowych oraz przeprowadzenie procesu uczenia, co w praktyce oznacza znaczące nakłady czasowe.
2. **Kamery głębi** - rozwiązanie opierało się na wcześniejszym zmapowaniu geometrii parkingu i wykorzystaniu kamery głębi do analizy przestrzennej. Każde miejsce mogłoby zostać opisane jako prostopadłościan, a przekroczenie jego granic byłoby wykrywane poprzez analizę chmury punktów. Metoda ta nie należy do najprostszych, wymaga stabilnej kalibracji czujnika głębi oraz dodatkowej warstwy przetwarzania danych przestrzennych.
3. **Analizy barwy** - koncepcja sprowadzała się do bezpośredniej analizy linii parkingowej widocznej w obrazie z kamery RGB. Zakładano, że w stałej konfiguracji środowiska położenie linii względem robota pozostaje przewidywalne. W konsekwencji możliwe było wyznaczenie konkretnego wycinka obrazu, w którym zawierałaby się linia. Na podstawie analizy kolorów pikseli w wyznaczonym obszarze podejmowana byłaby decyzja, czy linia jest widoczna, czy została zasłonięta przez pojazd.

Opcję 1 (sieci neuronowe) odrzucono w związku ze znaczącymi nakładami czasowymi potrzebnymi do przygotowania zbioru danych treningowych oraz procesu uczenia, co wykaczało poza ramy czasowe projektu. Podejście z wykorzystaniem kamery głębi odrzucono z uwagi na zbyt wysoką złożoność kalibracji czujnika głębi oraz przetwarzania danych przestrzennych.

Ostatecznie wybrano opcję 3 (analiza barwy). Mimo swoich ograniczeń w postaci zależności od oświetlenia i niskiej uniwersalności, uznano, że metoda ta jest najłatwiejsza do wdrożenia, charakteryzuje się niewielką złożonością obliczeniową, prostą implementacją oraz wystarczającą skutecznością w znanych warunkach. Docelowo planowana jest implementacja rozwiązania działającego na podstawie sieci neuronowych (opcja 1), które zapewniłoby większą uniwersalność oraz łatwą adaptację do nowych warunków.

3.6.2 Skrypt testowy do analizy linii

Stworzono skrypt testowy, którego zadaniem jest znalezienie odpowiedniego ROI oraz wartości parametrów, decydujących o ocenie poprawności parkowania na podstawie widoczności i ciągłości linii ograniczającej miejsce postojowe. Implementację zrealizowano w języku Python z wykorzystaniem biblioteki *OpenCV* (moduł *cv2*), która zapewnia gotowe narzędzia do przetwarzania obrazu jak i konwersji przestrzeni barw. Aby osiągnąć powtarzalny wynik oraz w najbardziej efektywny sposób analizować linie, założono że kamera podczas analizy będzie znajdowała się na osi linii parkingowej, zawsze w tej samej odległości od jej końca i orientacji.

Algorytm wyznacza ROI w centralnej części kadru, którego położenie określane jest procentowo względem wymiarów obrazu. Następnie wycięty fragment konwertowany jest do przestrzeni HSV, przestrzeń ta lepiej analizuje odcienie białego oraz szarości niż przestrzeń RGB, co jest ważne wobec występujących w świetle cieni. Ocena poprawności parkowania na podstawie analizy ciągłości linii. Dla każdego wiersza ROI zliczana jest liczba pikseli spełniających kryterium barwy, której zakres określony jest w programie. Jeżeli udział wierszy zawierających linię (tj. takich, w których liczba pikseli przekracza ustalony próg) jest wystarczający, parkowanie uznawane jest za poprawne. W przeciwnym przypadku przyjmowane jest, że pojazd zasłonił linię i przekroczył obrys miejsca.

3.6.3 Metodyka doboru parametrów algorytmu

Proces kalibracji parametrów algorytmu przeprowadzono w warunkach kontrolowanych, zapewniających powtarzalność pomiarów. Kamery ustawiono w osi linii parkingowej, prostopadłe do jej przebiegu, w odległości odpowiadającej pozycji na osi linii przerywanych parkingu. W takiej konfiguracji analizowana linia znajdowała się w centralnej części kadru, końcowe wartości parametrów widoczne są w tabeli 3.4.

Dobór parametrów ROI

Parametry określające ROI wyznaczono na podstawie obrazu referencyjnego przedstawionego na rysunku 3.11a. W procesie iteracyjnym sprawdzano kolejne wartości parametrów `roi_x_start_pct`, `roi_y_end_pct`, `roi_x_center` oraz `roi_x_width_pct`, wizualizując zaznaczony obszar na obrazie. Wybrano wartości zapewniające pełne objęcie linii przez ROI, po czym dodano zapas bezpieczeństwa wynoszący 10% każdej wartości, aby uwzględnić niewielkie odchylenia pozycji robota.

Dobór progów przestrzeni HSV

Parametr `hsv_upper` określający górną granicę progowania w przestrzeni HSV ustawiono na wartość `[180, 30, 255]`, co odpowiada jasnym kolorom o niskim nasyceniu - białym i prawie białym odcieniom i nie modyfikowano go w trakcie kalibracji. Dla parametru `hsv_lower` ustalającego dolną granicę progowania przeprowadzono test z pojazdem rzucającym cień na linię (rysunek 3.11b). Składowe H i S ustawiono na wartość 0, co umożliwia detekcję odcieni szarości niezależnie od barwy. Składową V zmniejszano iteracyjnie, aż do uzyskania wartości `continuity_threshold` różnej od wartości referencyjnej z rysunku 3.11a. Po znalezieniu progowej wartości V dodano margines bezpieczeństwa, obniżając ją dodatkowo o 10%.

Dobór progu ciągłości linii

Parametr `continuity_threshold` wyznaczono na podstawie analizy różnych konfiguracji nieprawidłowo zaparkowanych pojazdów, w tym przypadków przedstawionych na rysunkach 3.11c i 3.11d. Dla każdej konfiguracji wyznaczono wartość wskaźnika ciągłości linii. Jako wartość progową przyjęto wartość uzyskaną dla przypadku o najmniejszym stopniu naruszenia (rysunek 3.11c), zwiększoną o margines 0,03, co zapewnia pewność detekcji przy jednoczesnym ograniczeniu liczby fałszywych alarmów.

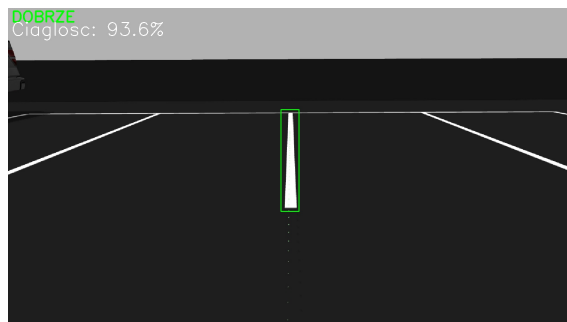
Dobór minimalnego udziału białych pikseli

Parametr `min_white_pct` określający minimalny procentowy udział pikseli linii w pojedynczym wierszu ROI ustawiono na stosunkowo niską wartość 0,05. Taki dobór ma na celu wykluczenie sytuacji, w których pojazd nieznacznie najjeżdża na linię, nie przekraczając jej całym obrysem, co nie powinno być traktowane jako naruszenie przepisów parkowania.

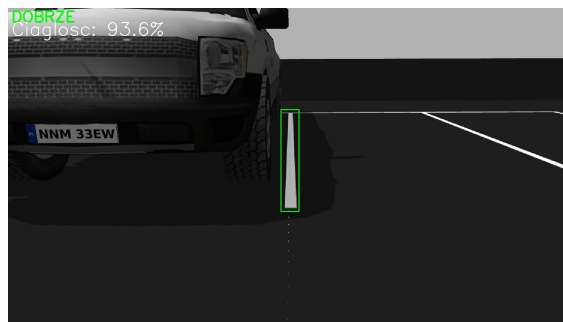
Tabela 3.4: Empirycznie dobrane parametry algorytmu analizy linii

Parametr	Wartość	Opis
<code>roi_y_start_pct</code>	0.32	Początek obszaru ROI w osi pionowej (procent wysokości obrazu)
<code>roi_y_end_pct</code>	0.64	Koniec obszaru ROI w osi pionowej
<code>roi_x_center</code>	0.50	Położenie środka ROI w osi poziomej
<code>roi_x_width_pct</code>	0.032	Szerokość analizowanego wycinka jako procent szerokości obrazu
<code>hsv_lower</code>	<code>[0, 0, 125]</code>	Dolna granica progowania koloru linii w przestrzeni HSV - jasny odcień szarego
<code>hsv_upper</code>	<code>[180, 30, 255]</code>	Górna granica progowania koloru linii w przestrzeni HSV - kolor biały
<code>min_white_pct</code>	0.05	Minimalny procentowy udział pikseli linii w pojedynczym wierszu ROI
<code>continuity_threshold</code>	0.85	Minimalny udział wierszy zawierających linię wymagany do uznania parkowania za poprawne

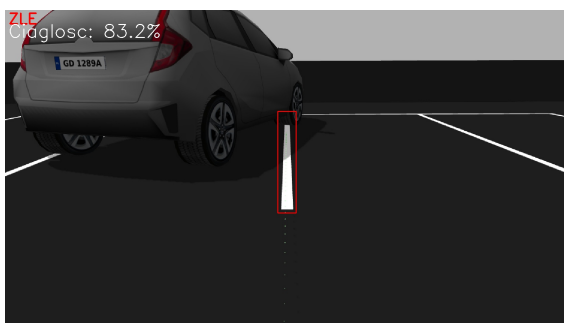
Przykładowe wyniki metody analizy ciągłości linii



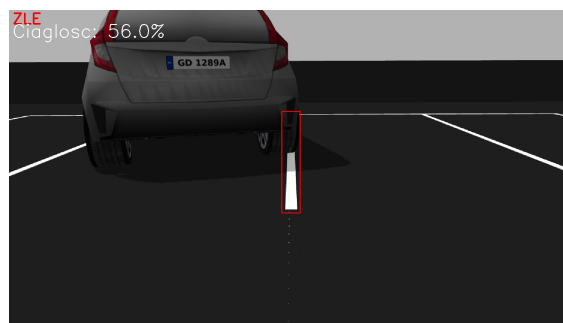
(a) Przypadek poprawnego parkowania – linia w pełni widoczna



(b) Przypadek poprawnego parkowania – pojazd zmienia barwę linii odczytaną przez kamerę



(c) Przypadek niepoprawnego parkowania - zasłonięcie linii daleko od kamery



(d) Przypadek niepoprawnego parkowania - zasłonięcie linii blisko kamery

Rysunek 3.11: Wyniki analizy parkowania, dla poszczególnych przypadków.

3.6.4 Analiza i ograniczenia metody

Przeprowadzone testy wykazały, że metoda działająca na podstawie analizy ciągłości linii jest wrażliwa na geometryczne położenie pojazdu względem kamery. Zbliżony stopień przekroczenia obrysu miejsca postojowego może generować różne wartości wskaźnika ciągłości w zależności od tego, w którym fragmencie linii nastąpiło jej zasłonięcie. W praktyce oznacza to, że podobnie nieprawidłowe zaparkowanie może zostać sklasyfikowane odmiennie, jeżeli przekroczenie nastąpi bliżej lub dalej względem środka analizowanego obszaru ROI. Kolejne ograniczenie dotyczy zastosowanej segmentacji barwnej. Algorytm identyfikuje linię na podstawie zakresu HSV odpowiadającego jasnym, słabo nasyconym barwom. W sytuacji, gdy pojazd posiada elementy o zbliżonych parametrach barwy i jasności, może dojść do częściowej błędnej klasyfikacji pikseli, co wpływa na wyznaczony wskaźnik ciągłości. Jednocześnie testy potwierdziły stabilność metody w obecności cieni. Rzucenie cienia przez model pojazdu na linię parkingową nie powodowało istotnej zmiany wartości wskaźnika ciągłości.

Metoda zapewnia zatem poprawną ocenę parkowania w warunkach kontrolowanych, przy stałej geometrii kamery i powtarzalnym układzie sceny, jednak jej uniwersalność w bardziej zróżnicowanych warunkach jest ograniczona.

3.7 Automatyzacja inspekcji parkingu z użyciem Nav2

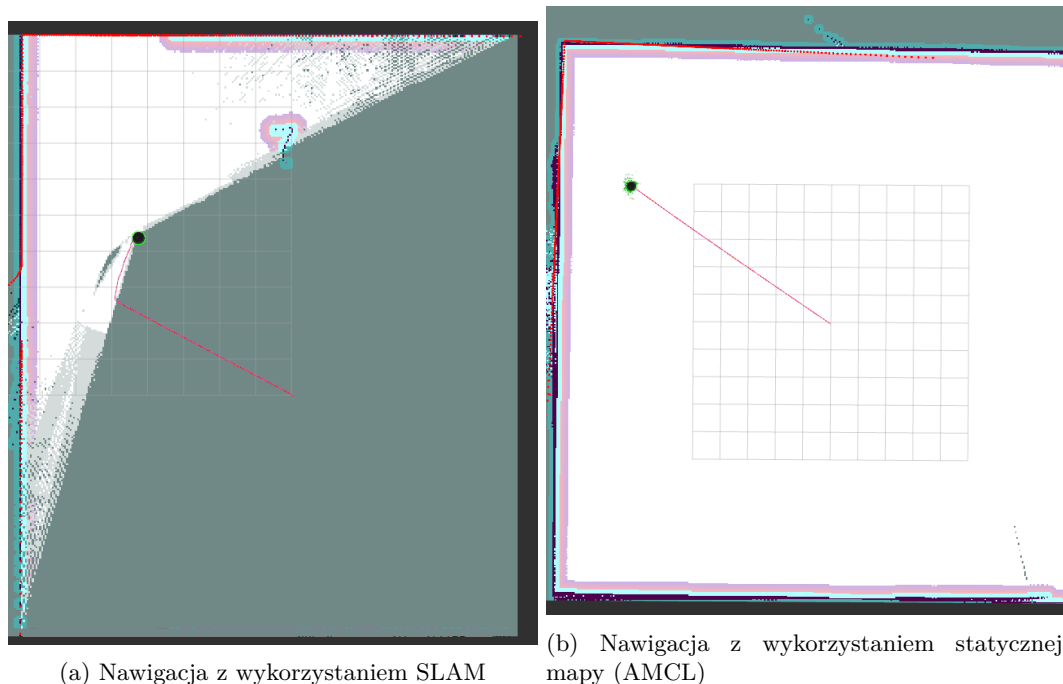
Po zaimplementowaniu i przetestowaniu poszczególnych modułów detekcji, opisanego w sekcji 3.6.2 - węzła analizy ciągłości linii parkingowej oraz w sekcji 3.5.3 - węzła rozpoznawania tablic rejestracyjnych. Do zautomatyzowania procesu inspekcji konieczne było opracowanie algorytmu koordynującego współpracę między tymi węzłami, a autonomicznie poruszającym się robotem. W tym celu zaimplementowano węzeł sterujący `parking_inspector_nav_node`. Węzeł ten integruje trzy główne podsystemy: autonomiczną nawigację działającą na podstawie Nav2, detekcję nieprawidłowego parkowania oraz rozpoznawanie tablic rejestracyjnych.

Architektura węzła bazuje na asynchronicznej komunikacji z wykorzystaniem procedur akcji (actions) systemu ROS 2 do komunikacji z Nav2 oraz usług (services) do wywoływania jednorazowych detekcji w węzłach podrzędnych. Podejście to pozwala na nieblokujące się wzajemnie

wykonywanie zadań nawigacyjnych i detekcyjnych.

3.7.1 Zmapowanie przestrzeni roboczej

Moduł Nav2, którego architekturę i algorytm działania opisano w sekcji 2.7, realizuje planowanie trajektorii za pomocą globalnej i lokalnej mapy kosztów. W celu zapewnienia optymalnego planowania trajektorii przez moduł Nav2 w środowisku symulacyjnym przedstawionym w sekcji 3.2.2, usunięto z niego pojazdy, a następnie przeprowadzono mapowanie przestrzeni za pomocą algorytmu SLAM, wykorzystując dane z czujników robota (LiDAR, IMU, enkoder). Wygenerowaną mapę zajętości zapisano i wykorzystano w kolejnych testach jako statyczną globalną mapę kosztów, z lokalizacją realizowaną przez algorytm AMCL⁶. Takie podejście umożliwiło uzyskanie bardziej optymalnych trajektorii (czerwona linia) w porównaniu do lokalnego mapowania w czasie rzeczywistym, co można zobaczyć na rysunku 3.12.



Rysunek 3.12: Porównanie planowania trajektorii przez Nav2 dla przejazdu z punktu $(-7.5, -5)$ do punktu $(0, 0)$ przy użyciu różnych metod lokalizacji

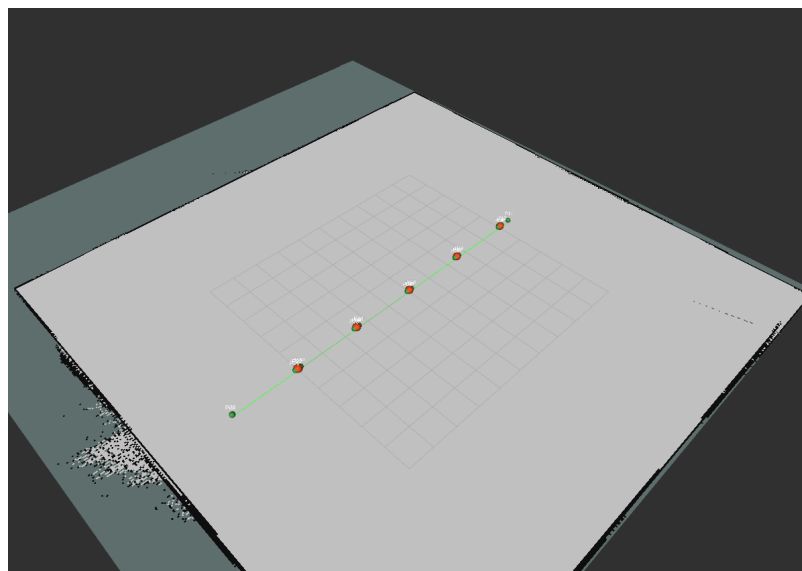
3.7.2 Definicja punktów inspekcyjnych (waypoints)

Trasa inspekcji danego parkingu definiowana jest w zewnętrznym pliku w formacie JSON, zawierającym uporządkowaną listę waypointów, czyli punktów, które po kolei ma osiągnąć robot, z daną konfiguracją przestrzenną. Każdy waypoint opisany jest następującymi polami:

- **name** - unikalna nazwa identyfikująca punkt,
- **x, y, z** - współrzędne pozycji w układzie mapy,
- **qx, qy, qz, qw** - kwaternion orientacji docelowej robota,
- **is_inspection_point** - flaga logiczna decydująca, czy w danym punkcie należy przeprowadzić detekcję.

Przykładową mapę świata można zobaczyć na rysunku 3.13. Punkty inspekcyjne zaznaczono kolorem pomarańczowym, a punkty pośrednie kolorem zielonym.

⁶ AMCL (Adaptive Monte Carlo Localization) - algorytm probabilistyczny służący do lokalizacji robota mobilnego w przestrzeni dwuwymiarowej. Wykorzystuje filtr cząsteczkowy do śledzenia pozycji i orientacji robota względem znanej, statycznej mapy otoczenia.



Rysunek 3.13: Mapa zajętości wraz z naniesionymi waypointami

3.7.3 Usługa sterująca realizacją punktów (Nav2)

Węzeł `parking_inspector_nav_node` udostępnia usługę ROS 2 typu `std_srvs/Trigger` pod adresem `/parking_inspector/start_mission` za pomocą której operator może zainicjować misję patrolu parkingu przez robota. Po jej wywołaniu węzeł sekwencyjnie przetwarza listę waypointów, wykorzystując klienta akcji `NavigateToPose` do komunikacji z Nav2, który odpowiada za znalezienie najbardziej optymalnej, bezkolizyjnej trasy do kolejnego punktu. Po wysłaniu celu nawigacyjnego węzeł rejestruje funkcje zwrotne (*callbacks*), które są wywoływane odpowiednio po akceptacji celu przez serwer Nav2 oraz po zakończeniu nawigacji do niego - czyli osiągnięciu pozycji z założonymi tolerancjami.

Wykorzystano trzypoziomowy łańcuch wywołań zwrotnych (*callback*):

1. `nav_response_callback` - zweryfikowanie, czy cel został zaakceptowany przez serwer Nav2,
2. `nav_result_callback` - osiągnięcie celu przez robota i decyzja o dalszym działaniu w zależności od typu waypointa,
3. wywołania detekcji lub przejście do następnego waypointa.

3.7.4 Problemy z dokładnością pozycji i orientacji

Podczas pierwszych testów zidentyfikowano dwa główne problemy związane z realizacją punktów inspekcyjnych przez Nav2.

Kalibracja parametrów dokładności Nav2

Domyślne parametry tolerancji pozycjonowania w Nav2 okazały się niewystarczające dla potrzeb projektu. Algorytm detekcji linii parkingowej z uwagi na sztywno zadeklarowany ROI jest bardzo czuły na niedokładną pozycję robota. Wymaga, aby kamera znajdowała się na osi analizowanej linii z dokładnością rzędu kilku centymetrów, a orientacja robota była zbliżona do prostopadłej względem linii. Przekroczenie tych tolerancji objawiało się przesunięciem obszaru ROI poza obraz linii, co prowadziło do fałszywych detekcji naruszeń. W związku z tym konieczne było dostrojenie parametrów kontrolera Nav2, w szczególności zmniejszenie tolerancji kątowej i liniowej w konfiguracji `goal_checker`, sprawdzającej czy obiekt sterowany osiągnął pozycję docelową.

Punkty pośrednie dla stabilizacji ustawienia robota

Pomimo dostrojenia parametrów Nav2, bezpośrednie wysłanie robota do pozycji inspekcyjnej (znajdującej się na osi linii parkingowej) wielokrotnie skutkowało niedostateczną dokładnością orientacji - robot docierał do celu, lecz jego kamera nie była zorientowana prostopadle do analizowanej linii. Problem ten był szczególnie widoczny w przypadku waypointów wymagających obrotu robota o kąt bliski 180° względem kierunku dojazdu. Rozwiązaniem okazało się wprowadzenie rozróżnienia waypointów na inspekcyjne oraz pośrednie, za pomocą wcześniej wspomnianej flagi logicznej: `is_inspection_point`, punkty pośrednie wymuszają na robocie wykonanie dodatkowego

manewru korekcyjnego, co zwiększa dokładność pozycji inspekcyjnej. W punkcie pośrednim robot nie wykonuje żadnej detekcji, a jedynie stabilizuje swoją orientację, po czym kontynuuje nawigację do kolejnego punktu.

3.7.5 Proces algorytmu dla pojedynczego punktu

Pełny cykl działania węzła `parking_inspector_nav_node` dla pojedynczego punktu inspekcyjnego składa się z następujących etapów:

1: Nawigacja. Węzeł wysyła współrzędne celu do serwera Nav2 za pośrednictwem akcji `NavigateToPose`. Robot autonomicznie planuje trasę i realizuje dojazd do wskazanego waypointa, omijając przeszkody wykryte przez LiDAR.

2: Klasyfikacja punktu. Po dotarciu do celu węzeł sprawdza wartość flagi `is_inspection_point`. Jeżeli punkt nie jest punktem inspekcyjnym, węzeł natychmiast przechodzi do nawigacji w kierunku następnego waypointa. W przypadku punktu inspekcyjnego następuje zatrzymanie robota na czas określony parametrem `detection_wait_time` (domyślnie 2 sekundy), co pozwala na stabilizację obrazu z kamery.

3: Detekcja linii parkingowej. Węzeł wywołuje metodę `/parking_detector/detect_once` węzła `parking_line_detector_node`. Usługa ta wykonuje jednorazową analizę ciągłości linii na bieżącym obrazie z kamery i zwraca wynik w formacie JSON zawierający ocenę (DOBRZE/ZLE), wartość wskaźnika ciągłości oraz ścieżkę do zapisanego obrazu z wynikiem detekcji.

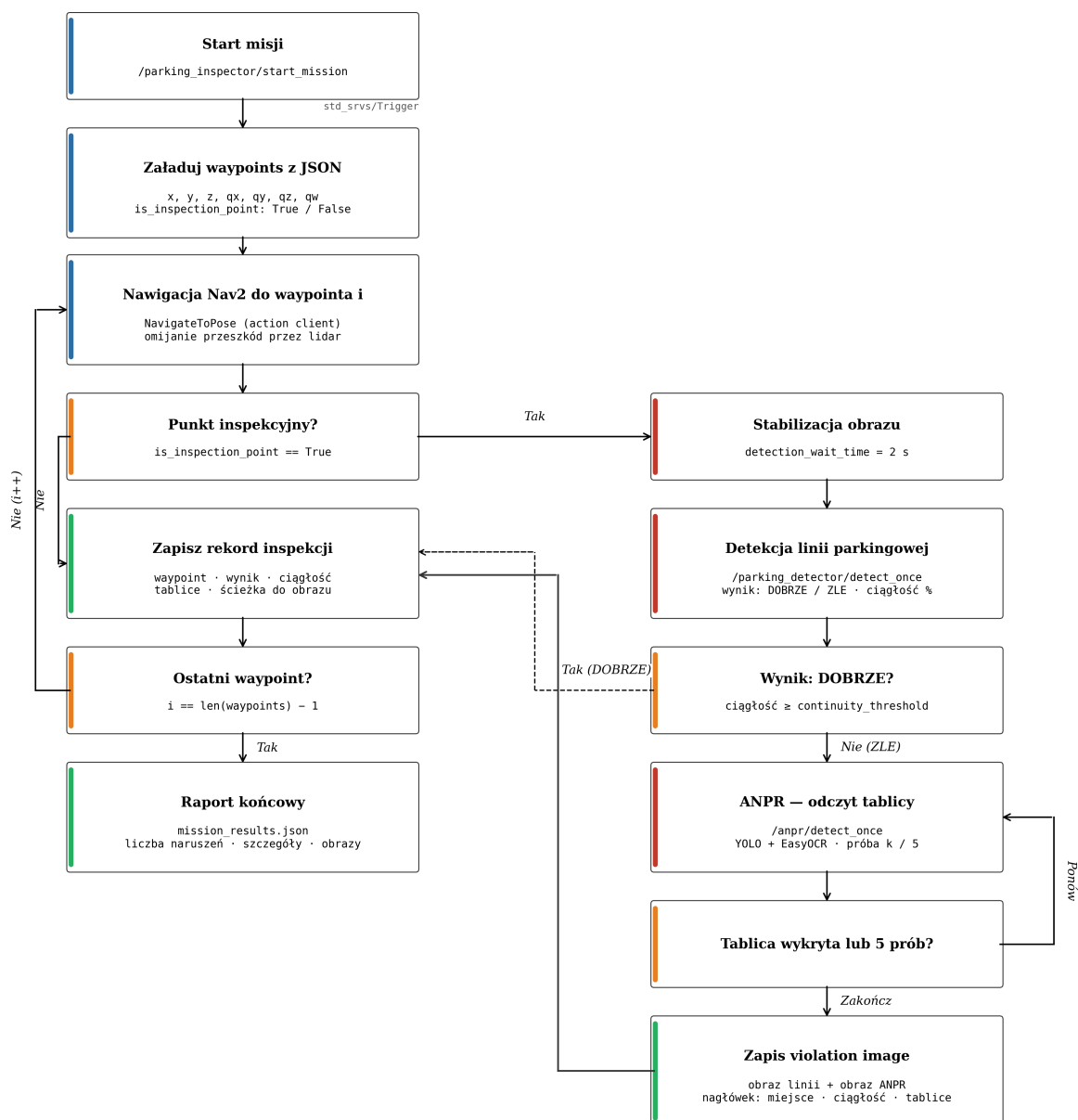
4a: Wynik pozytywny. Jeżeli detekcja zwróci wynik DOBRZE, co oznacza że ciągłość linii jest powyżej ustalonego progu, wynik inspekcji zostaje zapisany i węzeł przechodzi do następnego waypointa. Odczyt tablicy rejestracyjnej nie jest w tym przypadku wykonywany.

4b: Wykrycie naruszenia linii. w przypadku wyniku ZLE węzeł inicjuje proces rozpoznawania tablic rejestracyjnych poprzez metodę `/anpr/detect_once` węzła `anpr_node`. Aby zwiększyć szanse na prawidłową detekcję tablicy rejestracyjnej, węzeł wykonuje do 5 kolejnych wywołań metody ANPR z przerwą 0,5 sekundy między próbami. Proces kończy się wraz z pierwszym udanym odczytem tablicy lub po wyczerpaniu limitu prób. Wszystkie unikalne numery tablic wykryte w kolejnych próbach są zapisywane.

5: Zapis obrazu nieprawidłowego zaparkowania. Po zakończeniu procesu ANPR węzeł generuje obraz (*violation image*), łączący wyniki detekcji przerywania linii jak i tablicy rejestracyjnej. Obraz ten tworzony jest poprzez połączenie obrazów diagnostycznych z detektora linii i z detektora tablic. Tak przygotowany obraz zapisywany jest w katalogu wyjściowym i stanowi dokumentację fotograficzną.

6: Przejście do następnego punktu i podsumowanie. Po zakończeniu inspekcji danego punktu węzeł przechodzi do kolejnego waypointa. Po przejechaniu przez wszystkie punkty generowane jest podsumowanie misji w formacie JSON, zawierające liczbę skontrolowanych miejsc, liczbę wykrytych naruszeń, szczegółowe wyniki dla każdego punktu (w tym wartości ciągłości, odczytane tablice oraz ścieżki do obrazów ukazujących naruszenia). Podsumowanie publikowane jest w logach systemu oraz zapisywane do pliku.

Schemat blokowy procesu inspekcji parkingu



Rysunek 3.14: Schemat blokowy pełnego procesu inspekcji parkingu.

3.7.6 Komunikacja między węzłami

Tabela 3.5 zestawia interfejsy komunikacyjne wykorzystywane przez węzeł sterujący do interakcji z pozostałymi komponentami systemu.

Tabela 3.5: Interfejsy komunikacyjne węzła parking_inspector_nav_node

Interfejs	Typ	Kierunek	Opis
navigate_to_pose	Action	Klient → Nav2	Nawigacja do celu
/parking_detector/detect_once	Service	Klient → Detektor	Jednorazowa analiza linii
/anpr/detect_once	Service	Klient → ANPR	Jednorazowy odczyt tablicy
/parking_inspector/start_mission	Service	Serwer	Start misji (Trigger)

3.7.7 Testy zintegrowanego systemu inspekcji

Po wyznaczeniu optymalnych parametrów poszczególnych modułów przeprowadzono ewaluację całego systemu inspekcji parkingu działającego w trybie autonomicznym. Celem testów było

określenie skuteczności systemu jako całości, tj. łącznej zdolności do poprawnej klasyfikacji stanu miejsca parkingowego oraz identyfikacji pojazdu naruszającego granice miejsca.

Scenariusz testowy

Przygotowano środowisko symulacyjne zawierające 10 punktów inspekcyjnych, z konfiguracją opisaną w tabeli 3.6. Na trasie robota (rysunek 3.13) została dodana przeszkoda w postaci żółtego stożka o wymiarach $r = 0,15 \text{ m}$ oraz $h = 0,6 \text{ m}$, wygenerowany świat symulacji można zobaczyć na rysunku 3.15.

Tabela 3.6: Konfiguracja punktów inspekcyjnych w teście integracyjnym

Punkt	Stan rzeczywisty	Pojazd	Tablica (GT)
P1	Puste	—	—
P2	Naruszenie	Pickup	PZ 55999
P3	Naruszenie	Pickup	PZ 55999
P4	Prawidłowe	Pickup	WA 3476H
P5	Prawidłowe	Pickup	WA 3476H
P6	Puste	—	—
P7	Naruszenie	Hatchback	GD 1289B
P8	Naruszenie	Hatchback	GD 1289B
P9	Prawidłowe	Hatchback	WPR 01DB
P10	Prawidłowe	Hatchback	WPR 01DB



Rysunek 3.15: Świat w Gazebo, użyty w testach

Robot wykonał pełny przejazd inspekcyjny zgodnie z algorytmem opisanym w sekcji 3.7.5 oraz z parametrami wyznaczonymi w sekcji 3.6.3 i 3.5.4 (tabela 3.7).

Tabela 3.7: Parametry systemu użyte w teście integracyjnym

Parametr	Wartość	Opis
roi_y_start_pct	0,32	Początek obszaru ROI w osi pionowej (procent wysokości obrazu)
roi_y_end_pct	0,64	Koniec obszaru ROI w osi pionowej
roi_x_center	0,50	Położenie środka ROI w osi poziomej
roi_x_width_pct	0,032	Szerokość analizowanego wycinka jako procent szerokości obrazu
hsv_lower	[0, 0, 125]	Dolna granica progowania koloru linii w przestrzeni HSV - jasny odcień szarego
hsv_upper	[180, 30, 255]	Górna granica progowania koloru linii w przestrzeni HSV - kolor biały
min_white_pct	0,05	Minimalny procentowy udział pikseli linii w pojedynczym wierszu ROI
continuity_threshold	0,85	Minimalny udział wierszy zawierających linię wymagany do uznania parkowania za poprawne
YOLO confidence (Y)	0,5	Próg uznania obszaru za obszar zawierający tablicę rejestracyjną przez YOLO
OCR confidence threshold (C)	0,5	Próg pewności rozpoznania tekstu przez Easy-OCR

Metodyka

Ocenę systemu przeprowadzono w dwóch etapach, odpowiadających architekturze systemu inspekcji.

Etap 1 - Detekcja naruszenia linii parkingowej. Moduł analizy ciągłości linii parkingowej realizuje binarną klasyfikację stanu miejsca postojowego: **ZLE** (naruszenie wykryte) lub **DOBRE** (brak naruszenia). Wynik klasyfikacji porównano ze stanem rzeczywistym każdego punktu inspekcyjnego, co pozwoliło na zbudowanie pełnej macierzy pomyłek:

- **TP** - system wykrył naruszenie i rzeczywiście pojazd przekroczył linię,
- **FP** - system wykrył naruszenie, ale pojazd był zaparkowany prawidłowo lub miejsce było puste,
- **FN** - system nie wykrył naruszenia mimo jego faktycznego wystąpienia,
- **TN** - system nie wykrył naruszenia i rzeczywiście parking był prawidłowy lub miejsce puste.

Na tej podstawie obliczono:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad \text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (3.2)$$

Etap 2 - Rozpoznawanie tablicy rejestracyjnej. Moduł ANPR uruchamiany jest wyłącznie w przypadku wykrycia naruszenia przez Etap 1 (wynik **ZLE**). Dla każdego takiego przypadku oceniono, czy tablica rejestracyjna pojazdu została:

- odczytana poprawnie (zgodność z wartością referencyjną),
- odczytana z błędem (odległość Levenshteina $d_L > 0$),
- nieodczytana (brak detekcji tablicy w 5 próbach).

Wyniki - Etap 1: Detekcja naruszenia linii

Wyniki klasyfikacji stanu miejsc parkingowych dla poszczególnych punktów inspekcyjnych przedstawiono w tabeli 3.8.

Tabela 3.8: Wyniki detekcji naruszenia linii parkingowej

Punkt	Stan rzeczyw.	Wynik systemu	Ciągłość	Klasyfikacja
P1	Puste	DOBRZE	96,5%	TN
P2	Naruszenie	ZLE	50,8%	TP
P3	Naruszenie	DOBRZE	87,8%	FN
P4	Prawidłowe	ZLE	27,4%	FP
P5	Prawidłowe	DOBRZE	93,5%	TN
P6	Puste	DOBRZE	95,0%	TN
P7	Naruszenie	DOBRZE	85,1%	FN
P8	Naruszenie	ZLE	57,0%	TP
P9	Prawidłowe	DOBRZE	93,0%	TN
P10	Prawidłowe	DOBRZE	93,3%	TN

Macierz pomyłek oraz obliczone metryki przedstawiono w tabeli 3.9.

Tabela 3.9: Macierz pomyłek i metryki - detekcja naruszenia linii

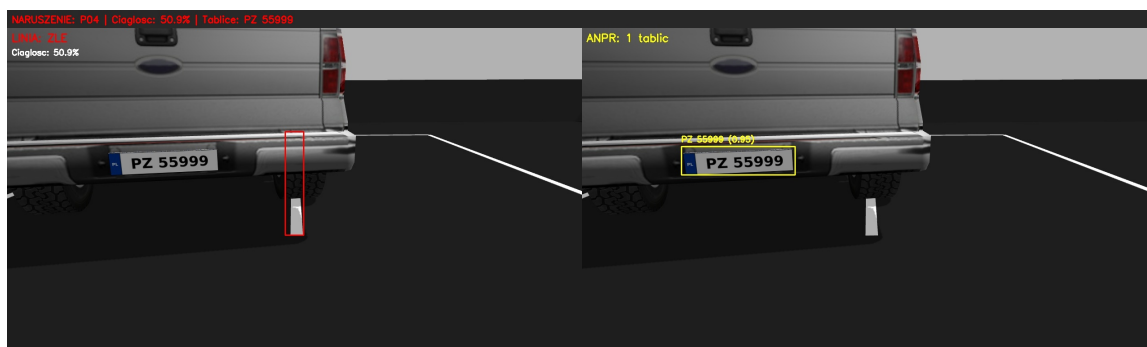
Metryka	Wartość	Metryka	Wartość
TP	2	Precision	66,6%
FP	1	Recall	50,0%
FN	2	Accuracy	70,0%
TN	5		

Wyniki - Etap 2: Rozpoznawanie tablic rejestracyjnych

Moduł ANPR został uruchomiony dla 3 punktów, w których Etap 1 zwrócił wynik ZLE (P2, P8 oraz P4). Wyniki odczytu tablic przedstawiono w tabeli 3.10, a wygenerowane *violation_image* pokazane zostały na rysunku 3.16.

Tabela 3.10: Wyniki rozpoznawania tablic rejestracyjnych w teście integracyjnym

Punkt	Klasyfik.	Odczyt ANPR	GT	d_L	Wynik
P2	TP	PZ 55999	PZ 55999	0	Poprawny
P4	FP	WA 3476H	WA 3476H	0	Poprawny
P8	TP	GD 1289B	GD 1289B	0	Poprawny



(a) *violation_image* - P2



(b) *violation_image* - P4

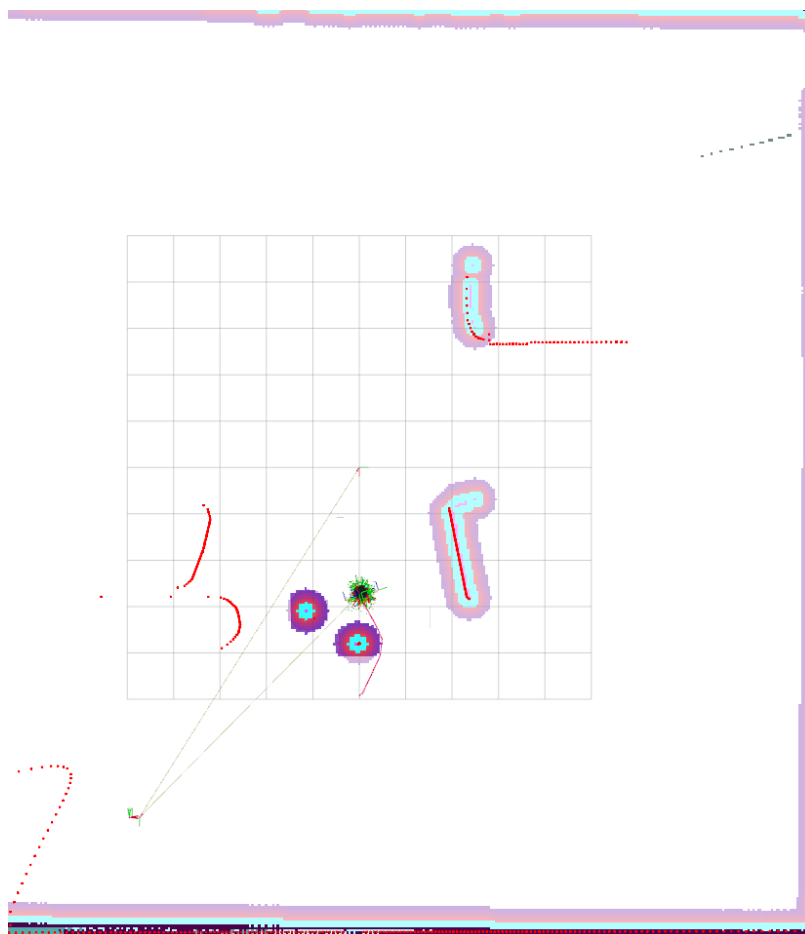


(c) *violation_image* - P8

Rysunek 3.16: Dokumentacja fotograficzna dla wykonanych testów.

Omijanie przeszkód przez Nav2

Na rysunku 3.17 widać, że algorytm Nav2 poprawnie wyznaczył ścieżkę (czerwona linia) omijającą przeszkodę – trajektoria została poprowadzona wokół obiektu wykrytego przez LiDAR. Należy jednak zwrócić uwagę na fałszywą detekcję – LiDAR zarejestrował dodatkową, nieistniejącą przeszkodę. Pomimo tego, robot prawidłowo poruszał się po wyznaczonej trasie, omijając rzeczywiste przeszkody.



Rysunek 3.17: Wytyczenie trasy wokół przeszkody przez Nav2

Analiza wyników

Jedyny przypadek FP (punkt P4) wynikał z niedokładności pozycjonowania robota przez moduł Nav2, problem ten był omawiany w sekcji 3.7.4. Niewystarczająca dokładność dojazdu do punktu inspekcyjnego spowodowała przesunięcie obszaru ROI poza całkowity obrys linii, przypadek ten ukazany jest na rysunku 3.16b. W dwóch przypadkach system nie wykrył naruszenia linii mimo jego faktycznego wystąpienia. W obu przypadkach pojazd przekraczał linię parkingową w niewielkim stopniu, a zmierzony wskaźnik ciągłości nieznacznie przekraczał próg klasyfikacji (85%), wyniki dla reszty punktów były zgodne z oczekiwanymi. Przyjęte wcześniej parametry modułu odpowiedzialnego za prawidłową detekcję tablic rejestracyjnych okazały się skuteczne.

Kalibracja parametrów

Na podstawie analizy przyczyn błędów zmieniono parametry:

1. **Szerokości ROI** (`roi_x_width_pct`) - zwiększono o 10% wartości z 0,032 na 0,035, szerszy obszar analizy jest mniej wrażliwy na drobne przesunięcia kamery względem osi linii, co pozwala kompensować niedokładności pozycjonowania modułu Nav2.
2. **Progu ciągłości** (`continuity_threshold`) - zwiększono próg z 0,85 na 0,88 w celu wyeliminowania błędów typu FN.

Wyniki - po kalibracji

Dla zmienionych parametrów powtórzono test przy zachowaniu wcześniejszego scenariusza, otrzymane wyniki zaprezentowano w tabeli 3.11 i 3.12, a utworzona na ich podstawie macierz pomylek jest dostępna w tabeli 3.13.

Tabela 3.11: Wyniki detekcji naruszenia linii parkingowej - po kalibracji

Punkt	Stan rzeczyw.	Wynik systemu	Ciągłość	Klasyfikacja
P1	Puste	DOBRZE	94,8%	TN
P2	Naruszenie	ZLE	48,7%	TP
P3	Naruszenie	DOBRZE	88,2%	FN
P4	Prawidłowe	DOBRZE	95,2%	TN
P5	Prawidłowe	DOBRZE	92,6%	TN
P6	Puste	DOBRZE	94,8%	TN
P7	Naruszenie	ZLE	81,2%	TP
P8	Naruszenie	ZLE	55,6%	TP
P9	Prawidłowe	DOBRZE	92,1%	TN
P10	Prawidłowe	DOBRZE	93,4%	TN

Tabela 3.12: Wyniki rozpoznawania tablic rejestracyjnych w teście integracyjnym - po kalibracji

Punkt	Klasyfik.	Odczyt ANPR	GT	d_L	Wynik
P2	TP	PZ 55999	PZ 55999	0	Poprawny
P7	TP	WA 3476H	WA 3476H	0	Poprawny
P8	TP	GD 1289B	GD 1289B	0	Poprawny

Tabela 3.13: Macierz pomyłek i metryki - detekcja naruszenia linii - po kalibracji

Metryka	Wartość	Metryka	Wartość
TP	3	Precision	100%
FP	0	Recall	75,0%
FN	1	Accuracy	90,0%
TN	6		

Analiza wyników po kalibracji

Porównanie metryk przed i po kalibracji (tabela 3.14) wskazuje na znaczącą poprawę skuteczności systemu.

Tabela 3.14: Porównanie metryk detekcji linii przed i po kalibracji

Metryka	Przed kalibracją	Po kalibracji
TP	2	3
FP	1	0
FN	2	1
TN	5	6
Precision	66,7%	100,0%
Recall	50,0%	75,0%
Accuracy	70,0%	90,0%

Zmiana parametrów pozwoliła całkowicie wyeliminować detekcje fałszywie pozytywne, jednocześnie liczba błędów fałszywie negatywnych zmniejszyła się z 2 do 1. Jedynym nierozpoznanym naruszeniem pozostał punkt P3, dla którego wskaźnik ciągłości wyniósł 88,2%, wartość nieznacznie przekraczająca próg decyzyjny na poziomie 88%. Moduł ANPR we wszystkich przypadkach uruchomienia zwrócił poprawne odczyty tablic rejestracyjnych dla parametrów dobranych w sekcji 3.5.4

Należy podkreślić, że przeprowadzona kalibracja została dostosowana do konkretnego scenariusza testowego, a przy zmianie go - parametry mogą wymagać ponownej kalibracji. W celu uzyskania najbardziej uniwersalnej konfiguracji parametrów wskazane byłoby przygotowanie kilku różnych scenariuszy testowych i dobranie parametrów minimalizujących średni błąd dla każdego z nich. Takie podejście pozwoliłoby na wyznaczenie konfiguracji bardziej odpornej na zmienne warunki pracy.

3.8 Decyzja o sposobie raportowania naruszeń

3.8.1 Brak jednoznacznego przypisania naruszenia do pojazdu w przypadku dwóch aut

Zastosowana metoda detekcji działająca na podstawie analizy ciągłości linii parkingowej posiada istotne ograniczenie w kontekście identyfikacji sprawcy naruszenia. Linia parkingowa oddziela dwa sąsiednie miejsca postojowe, w związku z czym jej przerwanie może być spowodowane przez pojazd znajdujący się po dowolnej stronie linii. W sytuacji, gdy oba miejsca są zajęte, system nie jest w stanie jednoznacznie wskazać, który z pojazdów przekroczył obrys swojego miejsca. Obie tablice rejestracyjne mogą zostać odczytane, jednak automatyczne przypisanie naruszenia do konkretnego pojazdu wymaga dodatkowej analizy geometrycznej lub zastosowania bardziej zaawansowanych metod detekcji.

3.8.2 Minimalnie inwazyjne podejście: zapis dowodu dla operatora

Z uwagi na opisane ograniczenie oraz możliwość wystąpienia fałszywych detekcji, przyjęto podejście minimalnie inwazyjne. W trakcie testów zaobserwowano przypadki kwalifikowane jako przerwanie linii przez pojazd mimo prawidłowo zaparkowanych pojazdów. Automatyczne nakładanie kar w takich sytuacjach mogłoby prowadzić do niesprawiedliwego traktowania użytkowników parkingu. W celu ograniczenia wpływu niedoskonałości systemu zdecydowano, że system nie podejmuje automatycznej decyzji o przypisaniu naruszenia do konkretnego pojazdu, lecz dokumentuje sytuację poprzez zapis obrazów z potencjalnymi naruszeniami (*violation image*). Ostateczna ocena pozostaje w gestii operatora, który na podstawie zgromadzonej dokumentacji fotograficznej może jednoznacznie zidentyfikować, czy rzeczywiście doszło do naruszenia oraz który pojazd go dokonał. Podejście to wymaga wprowadzić interwencji operatora w końcowej fazie, ale dzięki temu ogranicza ryzyko niesłusznego przypisania naruszenia.

Rozdział 4

Podsumowanie

4.1 Wnioski końcowe

Końcowa postać systemu odbiega w pewnych aspektach od pierwotnych założeń projektowych. Algorytm klasyfikacji prawidłowości parkowania, zamiast planowanej metody bazującej na sieciach neuronowych, działa na podstawie analizy barwnej w wyznaczonym obszarze zainteresowania (ROI) z wykorzystaniem segmentacji w przestrzeni HSV. Podejście to, choć skuteczne w kontrolowanych warunkach, posiada ograniczenia wpływające na uniwersalność i pełną automatyczność systemu.

Przede wszystkim metoda jest wrażliwa na zmianę środowiska - inne warunki oświetleniowe, odmienna geometria parkingu czy niestandardowe oznakowanie mogą wymagać ponownej kalibracji parametrów. Dodatkowo, jak wykazano w sekcji 3.8.2, system nie jest w stanie jednoznacznie przypisać naruszenia do konkretnego pojazdu w sytuacji, gdy oba sąsiadujące miejsca są zajęte, co wyklucza w pełni automatyczne nakładanie kar i wymaga interwencji operatora.

Istotnym ograniczeniem okazała się również dokładność pozycjonowania robota przez moduł Nav2. Nawet po dostrojeniu parametrów tolerancji, niedokładności w osiąganiu pozycji inspekcyjnych stanowiły główne źródło błędów klasyfikacji, w tym jedyny przypadek detekcji fałszywie pozytywnej w teście integracyjnym. Sam algorytm analizy linii, przy poprawnym ustawieniu kamery, wykazywał wysoką skuteczność.

Pomimo wymienionych ograniczeń, po przeprowadzeniu kalibracji dla danego scenariusza testowego system osiągnął dokładność na poziomie 90% w detekcji naruszeń oraz 100% poprawnych odczytów tablic rejestracyjnych w teście integracyjnym. Wyniki te pozwalają uznać proces za skuteczny. W szczególności wysoką skuteczność wykazał moduł ANPR bazujący na połączeniu YOLO i EasyOCR, co potwierdza, że odpowiednio opracowane metody wizji komputerowej wspomaganej uczeniem głębokim dobrze sprawdzają się w zadaniach lokalizacji i odczytu tablic rejestracyjnych, również w środowisku symulacyjnym.

4.2 Propozycja dalszych prac

Dalszy rozwój platformy może obejmować kilka kierunków, zarówno poprawę istniejących modułów, jak i rozszerzenia funkcjonalności systemu.

W pierwszej kolejności wskazane byłoby zastąpienie algorytmu analizy ciągłości linii metodą działającą na podstawie sieci neuronowych, analogicznie do modułu ANPR. Wytrenowanie modelu typu YOLO do klasyfikacji stanu miejsca parkingowego (prawidłowe/nieprawidłowe) na podstawie obrazu z kamery wyeliminowałoby zależność od sztywnie zdefiniowanego ROI oraz progów barwnych, zwiększając uniwersalność systemu w różnych warunkach.

Zastosowanie robota mobilnego do inspekcji parkingu otwiera szereg możliwości rozbudowy o dodatkowe funkcje:

- Patrol w poszukiwaniu podejrzanych zachowań na parkingu, takich jak próby kradzieży pojazdów.
- Wyszukiwanie pojazdów poszukiwanych przez służby - ciągła analiza tablic rejestracyjnych i weryfikację z zewnętrzną bazą danych.
- Inspekcja stanu technicznego pojazdów, np. wykrywanie wycieków płynów eksploatacyjnych poprzez analizę obrazu podłoża pod pojazdem.
- Dodanie drugiej kamery, umożliwiające jednoczesną analizę miejsc parkingowych po obu stronach jezdni, co skróciłoby czas inspekcji.

- Dostosowanie systemu do pracy w warunkach dynamicznych, uwzględniających ruch pojazdów i pieszych na terenie parkingu.

Ostatecznie, po osiągnięciu zadowalających wyników w symulacji, docelowym krokiem byłoby przeniesienie systemu na fizycznego robota i realizacja koncepcji w środowisku rzeczywistym.

4.3 Podsumowanie

Realizacja projektu umożliwiła zdobycie praktycznego doświadczenia w zakresie budowy i integracji systemów robotycznych oraz wizyjnych w środowisku symulacyjnym. Praca objęła pełen cykl - od konfiguracji środowiska, przez implementację i testowanie poszczególnych modułów, po integrację systemu działającego w trybie autonomicznym. Doświadczenia zdobyte w trakcie pracy nad projektem, obejmujące zarówno umiejętności techniczne, jak i podejście do rozwiązywania złożonych problemów inżynierskich, stanowią solidną podstawę do dalszego rozwoju zawodowego i naukowego w dziedzinie robotyki.

Bibliografia

- [1] DigiKey. *Intro to ROS, Part 1: What is the Robot Operating System (ROS)?* 2025. URL: <https://www.digikey.com/en/maker/tutorials/2025/intro-to-ros-part-1-what-is-the-robot-operating-system-ros> (term. wiz. 10.02.2026).
- [2] S. Al-Batati Abdulrahman, Koubaa Anis i Abdelkader Mohamed. *ROS 2 Key Challenges and Advances: A Survey of ROS 2 Research, Libraries, and Applications*. 2024. URL: <https://www.preprints.org/manuscript/202410.1204/v1#sec2-preprints-121244> (term. wiz. 10.02.2026).
- [3] Aparobot. *What is ROS?* URL: <https://www.aparobot.com/articles/what-is-ros> (term. wiz. 10.02.2026).
- [4] Think Robotics. *ROS2 Tutorial for Beginners: Your Complete Guide to Robot Operating System 2*. 2024. URL: <https://thinkrobotics.com/blogs/tutorials/ros2-tutorial-for-beginners-your-complete-guide-to-robot-operating-system-2> (term. wiz. 10.02.2026).
- [5] Steve Macenski i in. "Robot Operating System 2: Design, Architecture, and Uses in the Wild". W: *arXiv preprint arXiv:2211.07752* (2022). DOI: 10.48550/arXiv.2211.07752.
- [6] Model Prime. *ROS 1 vs ROS 2: What Are the Biggest Differences?* 2023. URL: <https://www.model-prime.com/blog/ros-1-vs-ros-2-what-are-the-biggest-differences> (term. wiz. 10.02.2026).
- [7] Open Source Robotics Foundation. *ROS 2 integration overview — Gazebo Documentation*. 2025. URL: <https://gazebo.org/docs/latest/getstarted/> (term. wiz. 10.02.2026).
- [8] M. S. P. de Melo i in. "Analysis and Comparison of Robotics 3D Simulators". W: *2019 21st Symposium on Virtual and Augmented Reality (SVR)*. IEEE, 2019, s. 242–251.
- [9] A. Harris i J. M. Conrad. "Survey of Popular Robotics Simulators, Frameworks, and Toolkits". W: *Proceedings of IEEE Southeastcon*. IEEE, 2011, s. 243–249.
- [10] S. Ivaldi i in. "Tools for Simulating Humanoid Robot Dynamics: A Survey Based on User Feedback". W: *IEEE-RAS International Conference on Humanoid Robots*. IEEE, 2014, s. 842–849.
- [11] Aleksandar Haber. *How to Install and Run Gazebo Harmonic inside of ROS2 Jazzy Jalisco*. 2024. URL: <https://aleksandarhaber.com/how-to-install-and-run-gazebo-harmonic-inside-of-ros2-jazzy-jalisco-installation/> (term. wiz. 10.02.2026).
- [12] Jay Wang i in. "How to Pick a Mobile Robot Simulator: A Quantitative Comparison of CoppeliaSim, Gazebo, MORSE and Webots with a Focus on the Accuracy of Motion Simulations". W: *ResearchGate* (2022).
- [13] M. Zwingel i in. "Robotics Simulation – A Comparison of Two State-of-the-Art Solutions". W: *ASIM Conference on Simulation in Production and Logistics*. ASIM, 2022. URL: https://www.asim-gi.org/fileadmin/user_upload_asim/ASIM_Publikationen_OA/AM180/a2033.arep.20_OA.pdf.
- [14] OpenRobotics. *ROS2 wiki - URDF*. 2023. URL: <https://wiki.ros.org/urdf> (term. wiz. 10.02.2026).
- [15] ASTOR. *Benefits of using digital twins of robots or a robotic operation system*. 2025. URL: <https://www.astor.com.pl/en/articles/benefits-of-using-digital-twins-of-robots-or-a-robotic-operation-system/> (term. wiz. 10.02.2026).
- [16] Clearpath Robotics. *TurtleBot 4*. 2024. URL: <https://clearpathrobotics.com/turtlebot-4/> (term. wiz. 12.02.2026).

- [17] Husarion. *ROSBOT Manual and Documentation*. 2024. URL: <https://husarion.com/manuals/rosbot/> (term. wiz. 13.02.2026).
- [18] SMP Robotics. *ALPR Robot – Automatic License Plate Recognition Mobile System*. 2024. URL: https://smprobotics.com/products_autonomous_ugv/automatic-license-plate-recognition-mobile-system/ (term. wiz. 13.02.2026).
- [19] Reduct Store. *Comparison of RViz, Foxglove and Rerun*. 2024. URL: <https://www.reduct.store/blog/comparison-rviz-foxglove-rerun> (term. wiz. 10.02.2026).
- [20] Kenta Takaya i in. “Simulation Environment for Mobile Robots Testing Using ROS and Gazebo”. W: *Proceedings of the 20th International Conference on System Theory, Control and Computing (ICSTCC)*. Sinaia, Romania: IEEE, paź. 2016. URL: <https://fs.unm.edu/ScArt/SimulationEnvironment.pdf>.
- [21] Open Robotics. *RViz User Guide*. 2024. URL: <https://wiki.ros.org/rviz/UserGuide> (term. wiz. 10.02.2026).
- [22] Morgan Quigley and Brian Gerkey i William D. Smart. *Programming Robots with ROS*. O'Reilly Media, 2015.
- [23] Durrant-Whyte Hugh i Bailey Tim. “Simultaneous Localization and Mapping: Part I”. W: *IEEE Robotics & Automation Magazine* 13.2 (2006), s. 99–110.
- [24] Josep Aulinas i in. “The SLAM Problem: A Survey”. W: *Proceedings of the 2008 International Conference on Informatics in Control, Automation and Robotics (ICINCO)*. IOS Press, 2008, s. 363–371. DOI: 10.3233/978-1-58603-925-7-363.
- [25] Sebastian Thrun. “Robotic Mapping: A Survey”. W: *Exploring Artificial Intelligence in the New Millennium*. Red. Gerhard Lakemeyer i Bernhard Nebel. Morgan Kaufmann, 2002. ISBN: 1558608117.
- [26] Xiaobin Xu i in. “A Review of Multi-Sensor Fusion SLAM Systems Based on 3D LiDAR”. W: *Remote Sensing* 14.12 (2022), s. 2835. DOI: 10.3390/rs14122835.
- [27] Michael Montemerlo i in. “Junior: The Stanford Entry in the Urban Challenge”. W: *Journal of Field Robotics* 25.9 (2008), s. 569–597.
- [28] Jesse Levinson i in. “Towards Fully Autonomous Driving: Systems and Algorithms”. W: *Proceedings of the IEEE Intelligent Vehicles Symposium (IV)*. Baden-Baden, Germany, 2011, s. 163–168.
- [29] Giorgio Grisetti, Cyrill Stachniss i Wolfram Burgard. “Improved Techniques for Grid Mapping With Rao-Blackwellized Particle Filters”. W: *IEEE Transactions on Robotics* 23.1 (2007), s. 34–46.
- [30] Takafumi Taketomi, Hideaki Uchiyama i Sei Ikeda. “Visual SLAM algorithms: a survey from 2010 to 2016”. W: *IPSJ Transactions on Computer Vision and Applications* 9 (2017), s. 16. DOI: 10.1186/s41074-017-0027-2.
- [31] Reinhard Klette, Andreas Koschan i Karsten Schlüns. *Computer Vision: Three-Dimensional Data from Images*. 1 wyd. Springer, 1998.
- [32] David Nistér. “A Minimal Solution to the Generalised 3-Point Pose Problem”. W: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. T. 1. 2004, s. 560–567.
- [33] Ji Zhang i Sanjiv Singh. “LOAM: Lidar Odometry and Mapping in Real-Time”. W: *Proceedings of the Robotics: Science and Systems Conference (RSS)*. 12–14 July 2014. Berkeley, CA, USA, lip. 2014.
- [34] Carlos Campos i in. “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial, and Multimap SLAM”. W: *IEEE Transactions on Robotics* (2021). DOI: 10.1109/TR0.2021.3075644.
- [35] Steve Macenski i Ivona Jambrecic. “SLAM Toolbox: SLAM for the dynamic world”. W: *Journal of Open Source Software* 6.61 (2021), s. 2783. DOI: 10.21105/joss.02783. URL: <https://doi.org/10.21105/joss.02783>.
- [36] Open Robotics. *Navigation2 Documentation*. 2024. URL: <https://docs.nav2.org/> (term. wiz. 12.02.2026).

- [37] Ines Haouala. "Autonomous Navigation in Agriculture Scenarios using Nav2". Prac. mag. University of Genova, DIBRIS - Department of Computer Science, Bioengineering, Robotics i System Engineering, lip. 2025. URL: https://www.researchgate.net/profile/Ines_Haouala/publication/393662284_Autonomous_Navigation_in_Agriculture_Scenarios_using_Nav2/links/6874fb824d336a436746e80e/Autonomous-Navigation-in-Agriculture-Scenarios-using-Nav2.pdf.
- [38] Richard Szeliski. *Computer Vision: Algorithms and Applications*. 2 wyd. Springer, 2022. ISBN: 978-3-030-34372-9.
- [39] Pedro F. Felzenszwalb i in. "Object Detection with Discriminatively Trained Part-Based Models". W: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 32.9 (2010), s. 1627–1645.
- [40] Kah Kay Sung i Tomaso Poggio. "Example-Based Learning for View-Based Human Face Detection". W: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 20.1 (2002), s. 39–51.
- [41] Christian Wojek i in. "Pedestrian Detection: An Evaluation of the State of the Art". W: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 34.4 (2012), s. 743–761.
- [42] H. Kobatake i Y. Yoshinaga. "Detection of Spicules on Mammogram Based on Skeleton Analysis". W: *IEEE Transactions on Medical Imaging* 15.3 (1996), s. 235–245.
- [43] Zhong-Qiu Zhao i in. "Object Detection with Deep Learning: A Review". W: *arXiv preprint arXiv:1807.05511* (2018). URL: <https://arxiv.org/abs/1807.05511>.
- [44] Yangqing Jia i in. "Caffe: Convolutional Architecture for Fast Feature Embedding". W: *Proceedings of the ACM International Conference on Multimedia*. 2014.
- [45] Alex Krizhevsky, Ilya Sutskever i Geoffrey E. Hinton. "ImageNet Classification with Deep Convolutional Neural Networks". W: *Advances in Neural Information Processing Systems (NeurIPS)*. 2012.
- [46] Zhe Cao i in. "Realtime Multi-Person 2D Pose Estimation Using Part Affinity Fields". W: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2017.
- [47] Shuo Yang i Ram Nevatia. "A Multi-Scale Cascade Fully Convolutional Network Face Detector". W: *Proceedings of the International Conference on Pattern Recognition (ICPR)*. 2016.
- [48] Chenyi Chen i in. "DeepDriving: Learning Affordance for Direct Perception in Autonomous Driving". W: *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*. 2015.
- [49] Xiaozhi Chen i in. "Multi-View 3D Object Detection Network for Autonomous Driving". W: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2017.
- [50] David G. Lowe. "Distinctive Image Features from Scale-Invariant Keypoints". W: *International Journal of Computer Vision* 60.2 (2004), s. 91–110.
- [51] Navneet Dalal i Bill Triggs. "Histograms of Oriented Gradients for Human Detection". W: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2005.
- [52] Rainer Lienhart i Jochen Maydt. "An Extended Set of Haar-Like Features for Rapid Object Detection". W: *Proceedings of the International Conference on Image Processing (ICIP)*. 2002.
- [53] Corinna Cortes i Vladimir Vapnik. "Support-Vector Networks". W: *Machine Learning* 20.3 (1995), s. 273–297.
- [54] Yoav Freund i Robert E. Schapire. "A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting". W: *Journal of Computer and System Sciences* 55.1 (1997), s. 119–139.
- [55] Ross Girshick i in. "Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation". W: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2014.
- [56] Shaoqing Ren i in. "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks". W: *Advances in Neural Information Processing Systems (NeurIPS)*. 2015, s. 91–99.
- [57] Joseph Redmon i in. "You Only Look Once: Unified, Real-Time Object Detection". W: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016.

- [58] IBM. *Convolutional Neural Networks (CNNs)*. 2024. URL: <https://www.ibm.com/think/topics/convolutional-neural-networks> (term. wiz. 12.02.2026).
- [59] DigitalOcean Community. *Faster R-CNN Explained: Object Detection*. 2023. URL: <https://www.digitalocean.com/community/tutorials/faster-r-cnn-explained-object-detection> (term. wiz. 12.02.2026).
- [60] Ayse Aybilge Murat i Mustafa Servet Kiran. "A comprehensive review on YOLO versions for object detection". W: *Engineering Science and Technology, an International Journal* (2025). ISSN: 2215-0986. URL: <https://www.sciencedirect.com/science/article/pii/S2215098625002162> (term. wiz. 10.02.2026).
- [61] Nicolas Carion i in. "End-to-End Object Detection with Transformers". W: *arXiv preprint arXiv:2005.12872* (2020). URL: <https://arxiv.org/abs/2005.12872> (term. wiz. 12.02.2026).
- [62] IBM. *Optical Character Recognition (OCR)*. 2024. URL: <https://www.ibm.com/think/topics/optical-character-recognition> (term. wiz. 12.02.2026).
- [63] Amardeep Singh, Kshitij Bacchuwar i Akshay Bhasin. "A Survey of OCR Applications". W: *International Journal of Machine Learning and Computing* 2.3 (2012), s. 314–318.
- [64] Augmented Startups. *Automatic Number Plate Recognition with EasyOCR and OpenCV*. 2023. URL: <https://medium.com/augmented-startups/automatic-number-plate-recognition-with-easy-ocr-and-open-cv-6d121384cdc7> (term. wiz. 12.02.2026).
- [65] AlgoDocs. *What Is OCR and How Does It Work?* 2024. URL: <https://algodocs.com/what-is-ocr-and-how-does-it-work/> (term. wiz. 12.02.2026).
- [66] Ray Smith. "An Overview of the Tesseract OCR Engine". W: (). URL: <https://static.googleusercontent.com/media/research.google.com/pl/pubs/archive/33418.pdf> (term. wiz. 12.02.2026).
- [67] JaidevAI. *EasyOCR Documentation*. n.d. URL: <https://www.jaidev.ai/easyocr/documentation/> (term. wiz. 12.02.2026).
- [68] Raghav Gali. *License Plate Recognition using OpenCV and Tesseract OCR*. 2023. URL: <https://github.com/Raghavgali/License-Plate-Recognition-using-OpenCV-and-Tesseract-OCR> (term. wiz. 17.02.2026).
- [69] Ultralytics. *How to Build an Automatic Number Plate Recognition System using EasyOCR Ultralytics YOLO11*. 2025. URL: <https://www.youtube.com/watch?v=Fym4hoaoreE> (term. wiz. 17.02.2026).
- [70] Ultralytics. *How to do Automatic Number Plate Recognition using Ultralytics YOLO11 and OpenAI GPT-4o-Mini Model*. 2025. URL: <https://www.youtube.com/watch?v=HS3Ta7c0zuI> (term. wiz. 17.02.2026).
- [71] Ultralytics. *Ultralytics YOLO11 Documentation*. 2024. URL: <https://docs.ultralytics.com/models/yolo11/> (term. wiz. 17.02.2026).

Wykaz skrótów i symboli

Skróty

ALPR	Automatic License Plate Recognition (Automatyczne rozpoznawanie tablic rejestracyjnych)
AMCL	Adaptive Monte Carlo Localization (Adaptacyjna lokalizacja Monte Carlo)
ANPR	Automatic Number Plate Recognition (Automatyczne rozpoznawanie numerów rejestracyjnych)
API	Application Programming Interface (Interfejs programistyczny aplikacji)
BT	Behavior Tree (Drzewo zachowań)
CNN	Convolutional Neural Network (Konwolucyjna sieć neuronowa)
CPU	Central Processing Unit (Jednostka centralna procesora)
DDS	Data Distribution Service (Standard komunikacji w ROS 2)
DETR	DEtection TRansformer (Architektura detekcji obiektów oparta na transformerach)
FN	False Negative (Wynik fałszywie ujemny)
FP	False Positive (Wynik fałszywie dodatni)
GPU	Graphics Processing Unit (Procesor graficzny)
GT	Ground Truth (Wartość referencyjna, rzeczywista)
GZ	Gazebo (Środowisko symulacyjne)
HSV	Hue, Saturation, Value (Model barw: odcień, nasycenie, jasność)
IMU	Inertial Measurement Unit (Jednostka inercyjna)
JSON	JavaScript Object Notation (Format wymiany danych tekstowych)
LIO	LiDAR-Inertial Odometry (Odometria lidarowo-inercyjna)
LLM	Large Language Model (Wielki model językowy)
OCR	Optical Character Recognition (Optyczne rozpoznawanie znaków)
OS	Operating System (System operacyjny)
QoS	Quality of Service (Parametry jakości usług komunikacyjnych)
ROI	Region of Interest (Obszar zainteresowania w obrazie)
ROS 2	Robot Operating System 2 (System operacyjny dla robotów, wersja 2)
Rviz	ROS Visualization (Narzędzie do wizualizacji danych w systemie ROS)
SDF	Simulation Description Format (Format opisu symulacji w Gazebo)
SLAM	Simultaneous Localization and Mapping (Jednoczesna lokalizacja i budowanie mapy)
TB4	TurtleBot 4 (Mobilna platforma robotyczna)
TF	Transform Library (Biblioteka transformacji układów współrzędnych)
TN	True Negative (Wynik prawdziwie ujemny)
TP	True Positive (Wynik prawdziwie dodatni)
URDF	Unified Robot Description Format (Format opisu mechaniki robota)
VIO	Visual-Inertial Odometry (Odometria wizyjno-inercyjna)
vSLAM	Visual SLAM (SLAM wizyjny)
XACRO	XML Macros (Narzędzie do parametryzacji plików URDF)
YOLO	You Only Look Once (Architektura sieci do detekcji obiektów w czasie rzeczywistym)

Symbole

N	Częstotliwość analizy klatek obrazu
Y	Próg pewności (confidence threshold) detekcji modelu YOLOv11
C	Próg pewności (confidence threshold) rozpoznawania tekstu przez EasyOCR
d_L	Odległość edycyjna Levenshteina pomiędzy ciągiem odczytanym a wzorcowym
r	Promień obiektu testowego (stożka) w środowisku symulacyjnym
h	Wysokość obiektu testowego (stożka) w środowisku symulacyjnym
x, y, z	Współrzędne położenia w kartezjańskim układzie mapy
q_w, q_x, q_y, q_z	Składowe kwaternionu opisującego orientację robota w przestrzeni
CER	Współczynnik błędów znakowych (Character Error Rate)

Spis rysunków

2.1	Architektura wymiany danych w ROS 2 [5]	12
2.2	Porównanie ROS 1 i ROS 2 [2]	13
2.3	Przykładowa symulacja w Gazebo [11]	14
2.4	Porównanie czterech popularnych symulatorów robotów: Coppeliasim (wcześniej znany jako V-REP), Gazebo, MORSE oraz Webots. [12]	15
2.5	Gazebo czy NIS dla danych zastosowań [13]	15
2.6	Karta katalogowa TurtleBot4 [16]	17
2.7	ROSbot [17]	18
2.8	ALPR Robot [18]	18
2.9	Interfejs graficzny RViz [19]	19
2.10	Schemat blokowy ilustrujący działanie algorytmów SLAM działających na podstawie fuzji sensorycznej [26]	21
2.11	Mapa zajętości budowana w czasie rzeczywistym przez algorytm SLAM	22
2.12	Schemat ilustrujący architekturę Nav2 [36]	23
2.13	Algorytm detekcji R-CNN [59]	25
2.14	Algorytm detekcji YOLO [57]	26
2.15	Odczyt numeru tablicy rejestracyjnej za pomocą OCR [64]	27
2.16	Uogólniony algorytm działania OCR [65]	28
3.1	Parking wraz z modelami samochodów	30
3.2	Wersja parkingu wykorzystywana podczas badań	31
3.3	Porównanie konfiguracji TB4.	32
3.4	Plik PNG tablicy rejestracyjnej użyty jako tekstura	33
3.5	Modele pojazdów z dodanymi modelami tablic rejestracyjnych.	34
3.6	Cztery przykładowe próbki zdjęć wraz z wynikiem użyte w testach.	36
3.7	Schemat blokowy węzła odpowiadającego za detekcję i odczyt tablic rejestracyjnych.	38
3.8	Scenariusz dla walidacji metody <i>is_valid_plate_format()</i>	39
3.9	Scenariusz w Gazebo dla doboru progu pewności OCR.	40
3.10	Konfiguracje przestrzenne pojazdu w testach ANPR	40
3.11	Wyniki analizy parkowania, dla poszczególnych przypadków.	45
3.12	Porównanie planowania trajektorii przez Nav2 dla przejazdu z punktu (-7.5, -5) do punktu (0, 0) przy użyciu różnych metod lokalizacji	46
3.13	Mapa zajętości wraz z naniesionymi waypointami	47
3.14	Schemat blokowy pełnego procesu inspekcji parkingu.	49
3.15	Świat w Gazebo, użyty w testach	50
3.16	Dokumentacja fotograficzna dla wykonanych testów.	53
3.17	Wytyczenie trasy wokół przeszkody przez Nav2	54

Spis tabel

2.1	Kryteria oceny cech jakościowych symulatorów (na podstawie [12])	14
3.1	Wyniki testów rozpoznawania tablic rejestracyjnych	37
3.2	Wyniki ewaluacji modułu ANPR dla 12 przejazdów	41
3.3	Szczegółowe zestawienie błędnych odczytów tablic rejestracyjnych	42
3.4	Empirycznie dobrane parametry algorytmu analizy linii	44
3.5	Interfejsy komunikacyjne węzła parking_inspector_nav_node	49
3.6	Konfiguracja punktów inspekcyjnych w teście integracyjnym	50
3.7	Parametry systemu użyte w teście integracyjnym	51
3.8	Wyniki detekcji naruszenia linii parkingowej	52
3.9	Macierz pomyłek i metryki - detekcja naruszenia linii	52
3.10	Wyniki rozpoznawania tablic rejestracyjnych w teście integracyjnym	52
3.11	Wyniki detekcji naruszenia linii parkingowej - po kalibracji	55
3.12	Wyniki rozpoznawania tablic rejestracyjnych w teście integracyjnym - po kalibracji	55
3.13	Macierz pomyłek i metryki - detekcja naruszenia linii - po kalibracji	55
3.14	Porównanie metryk detekcji linii przed i po kalibracji	55

Załączniki

Pliki projektowe

inz_pack/	
├─ gz_models/	 Modele SDF i tekstury dla Gazebo
├─ Rviz_config/	 Mapy (.pgm, .yaml) i konfiguracje Rviz2
├─ tb4_ws/	 Główny obszar roboczy ROS 2
└─ src/	
├─ parking_inspector_pkg/	 Główna paczka systemowa
├─ config/	 Pliki konfiguracyjne (np. waypoints.json)
├─ launch/	 Pliki uruchomieniowe .launch.py
├─ parking_inspector_pkg/	 Właściwe źródła węzłów Python
├─ anpr_node.py	
├─ parking_line_detector_node.py	
├─ parking_inspector_nav_node.py	
├─ resource/	 Wagi modelu YOLOv11 (.pt)
├─ scripts/	 Skrypty narzędziowe (inicjalizacja misji)
├─ set_initial_pose.py	
├─ start_mission.py	
├─ package.xml	 Definicje zależności paczki
├─ setup.py	 Konfiguracja instalacji setuptools
├─ turtlebot4/	 Model TurtleBot 4
├─ turtlebot4_simulator/	 Oficjalny symulator platformy TB4
├─ turtlebot4_gz_bringup/	 Konfiguracja startowa Gazebo
├─ worlds/	 Definicje światów
├─ yolo_ocr_test/	 Izolowane testy modułów wizyjnych
└─ README.md	 Instrukcja dla plików